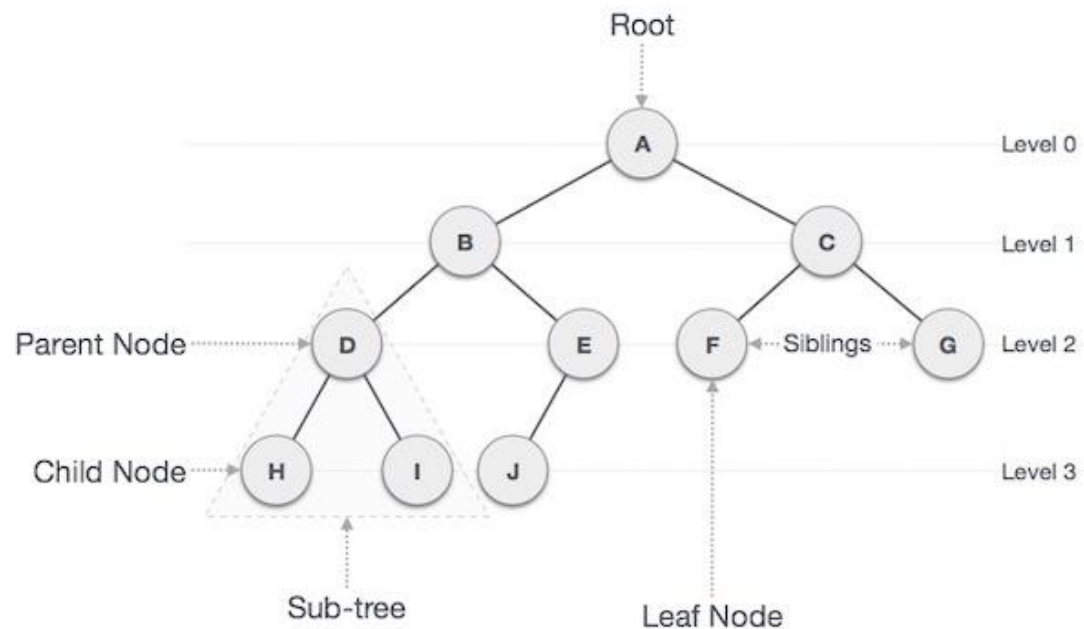


Lecture 03

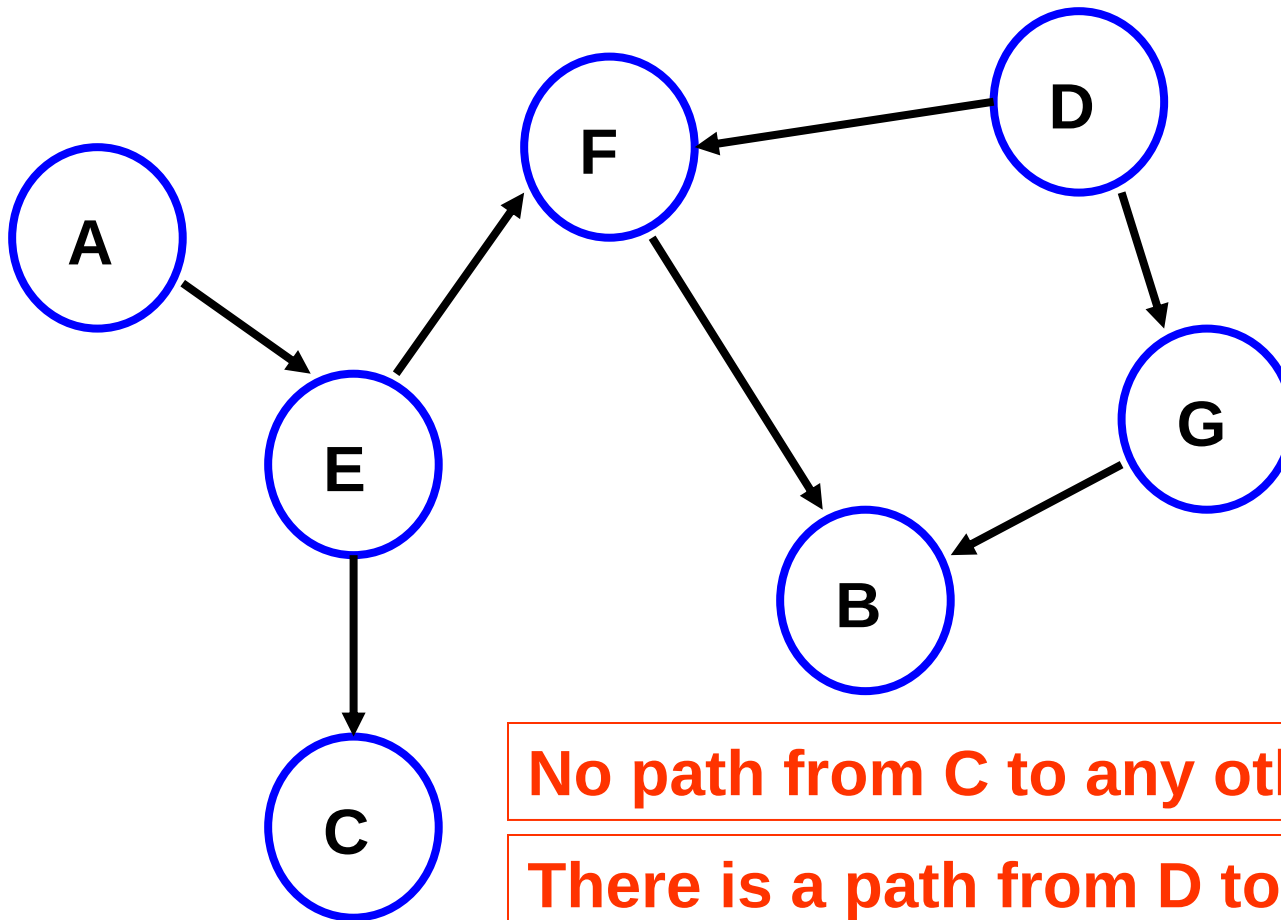
Binary Search Trees



Definition of Tree

- A ***tree*** is a set of linked nodes, such that there is one and only one ***path*** from a unique node (called the ***root*** node) to every other node in the tree.
- A path exists from node A to node B if one can follow a chain of pointers/references to travel from node A to node B.

Paths



No path from C to any other node.

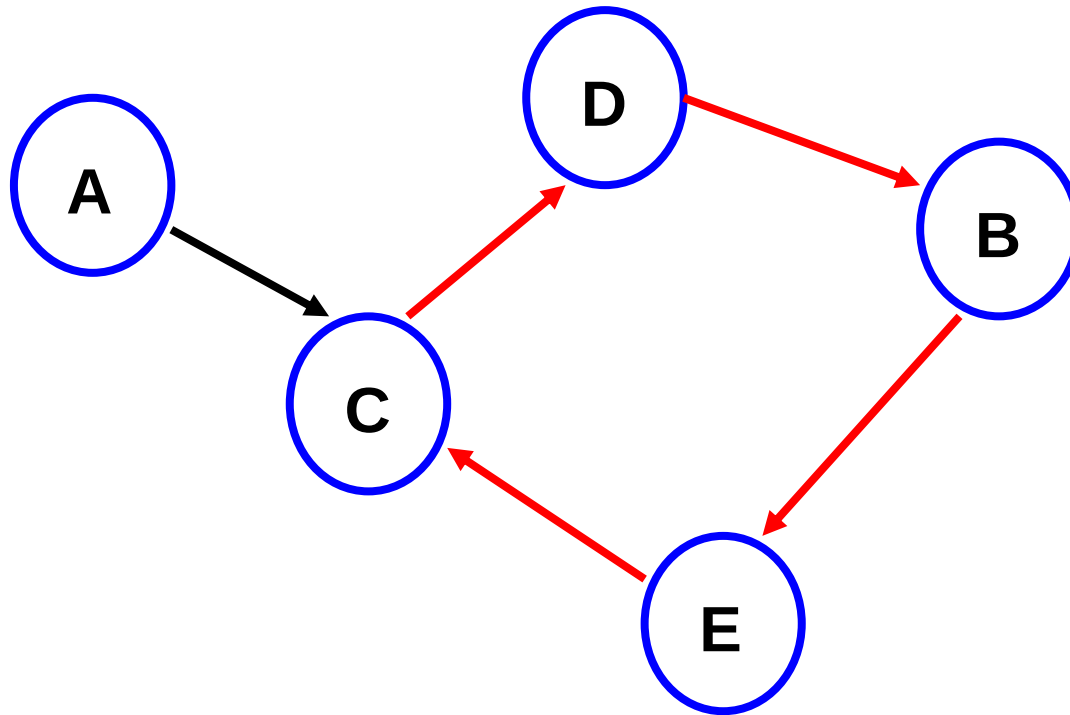
There is a path from D to B.

There is also a second path from D to B.

Cycles

- There is no ***cycle*** (circle of pointers) in a tree.
- Any linked structure that has a cycle would have more than one path from the root node to another node.

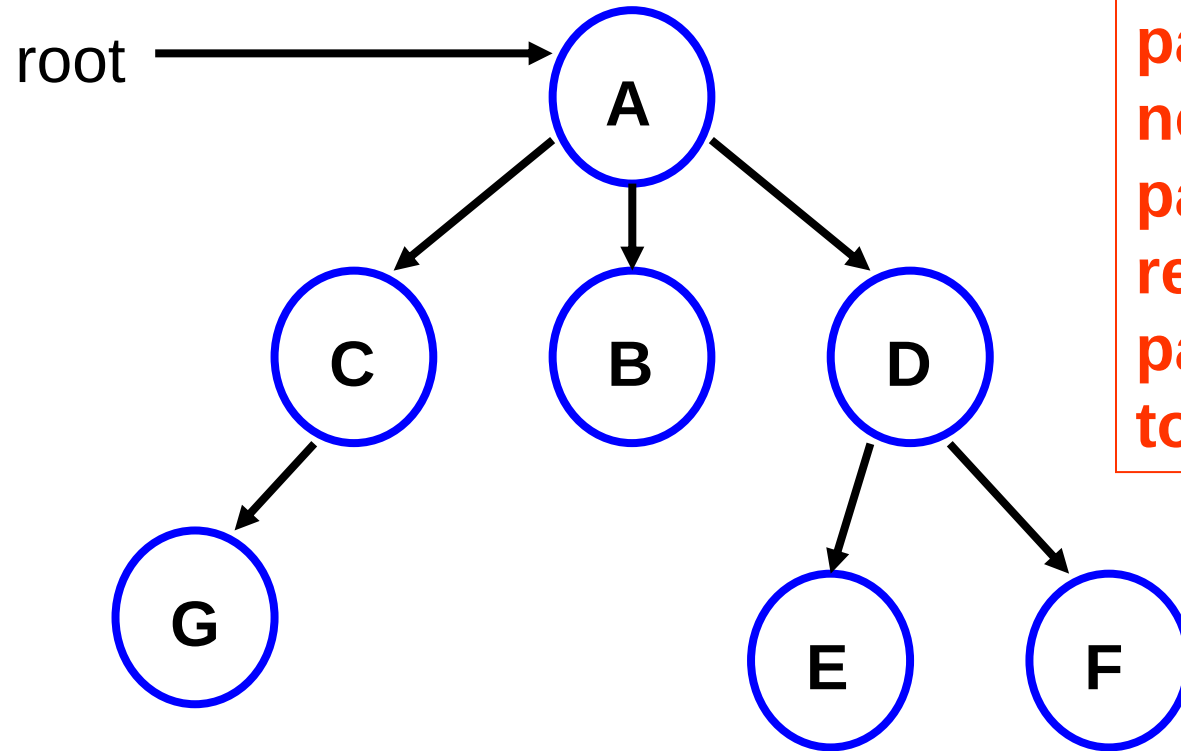
Example of a Cycle



Cycle: $C \rightarrow D \rightarrow B \rightarrow E \rightarrow C$

Tree cannot have a cycle.

Example of a Tree

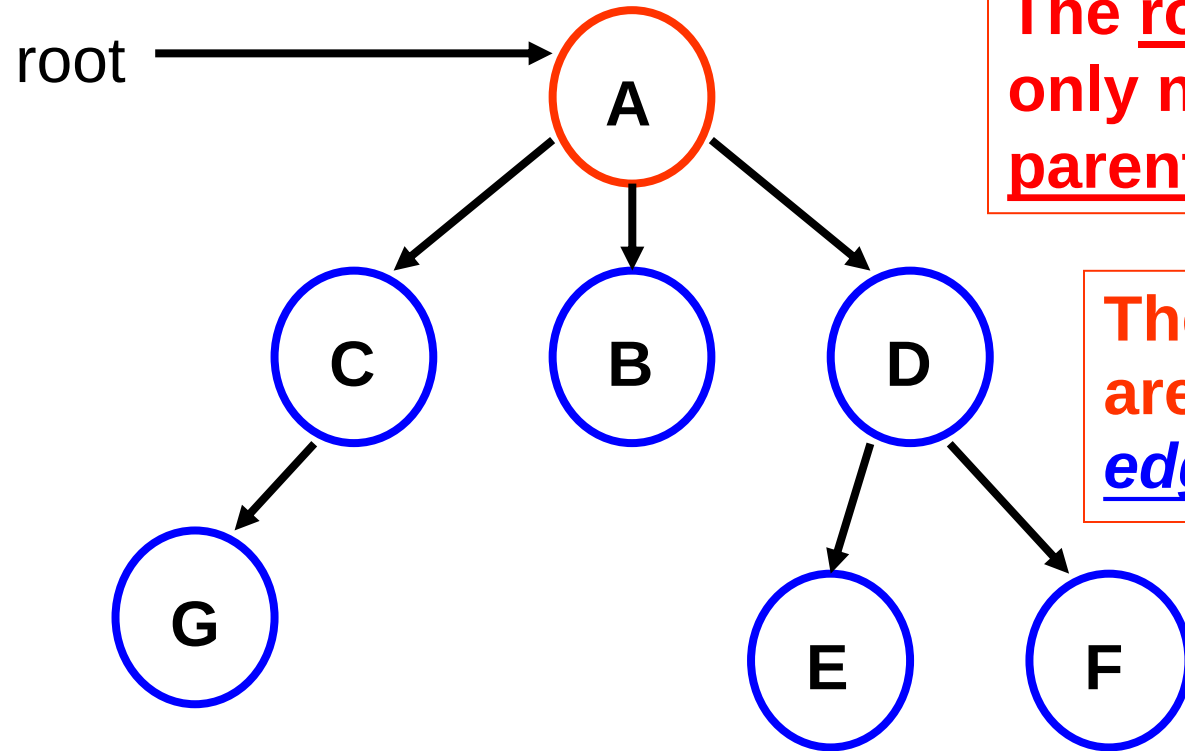


In a tree, every pair of linked nodes have a parent-child relationship (the parent is closer to the root)

For example, C is a parent of G

E and F are children of D

Example of a Tree (cont.)



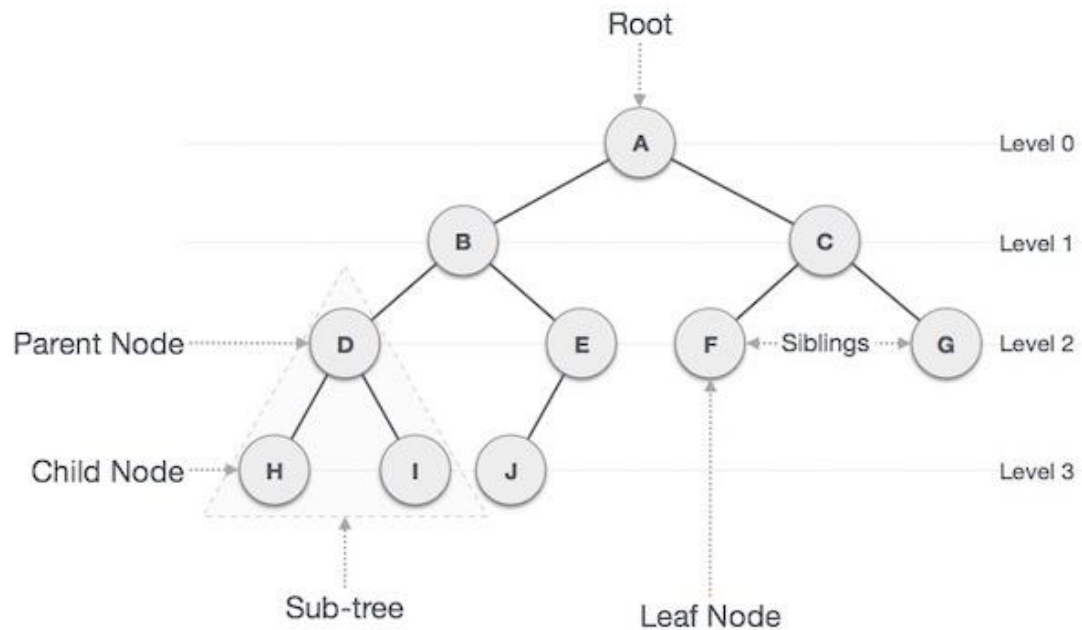
The root node is the only node that has no parent.

The links in a tree are often called edges

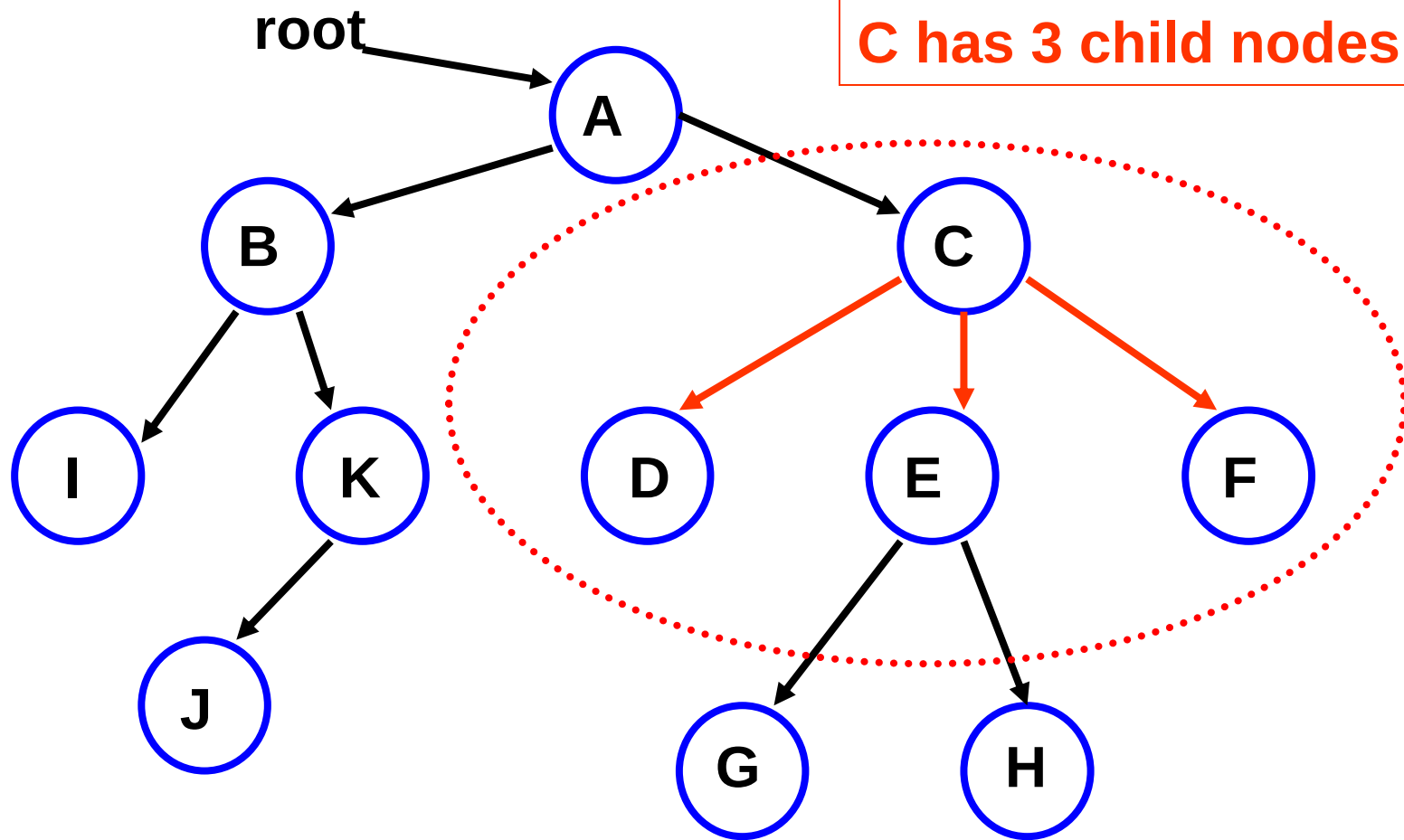
Leaf nodes (or *leaves* for short) have no children, e.g. B, E, F, G

Binary Trees

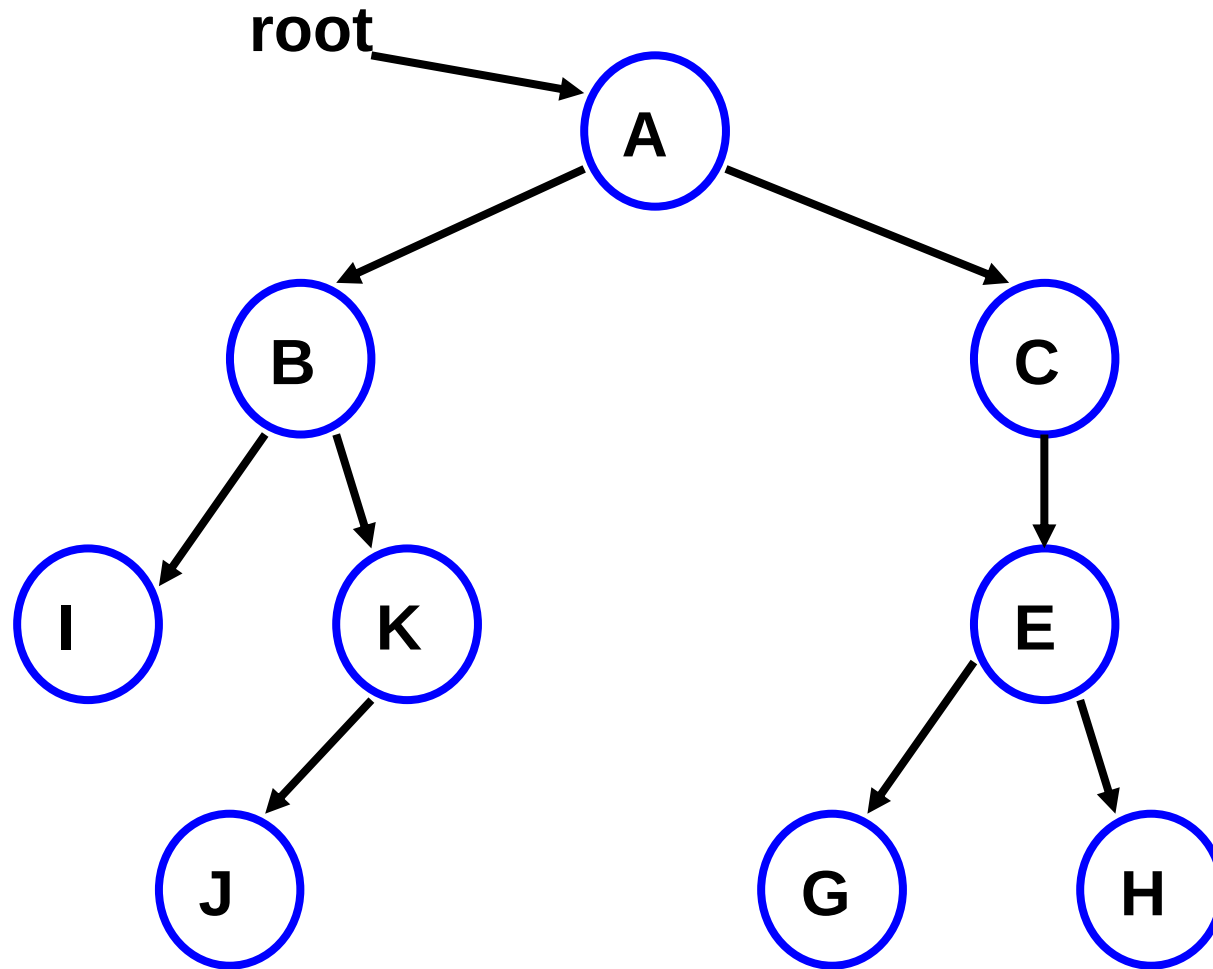
- A **binary tree** is a tree in which each node can only have up to two children...



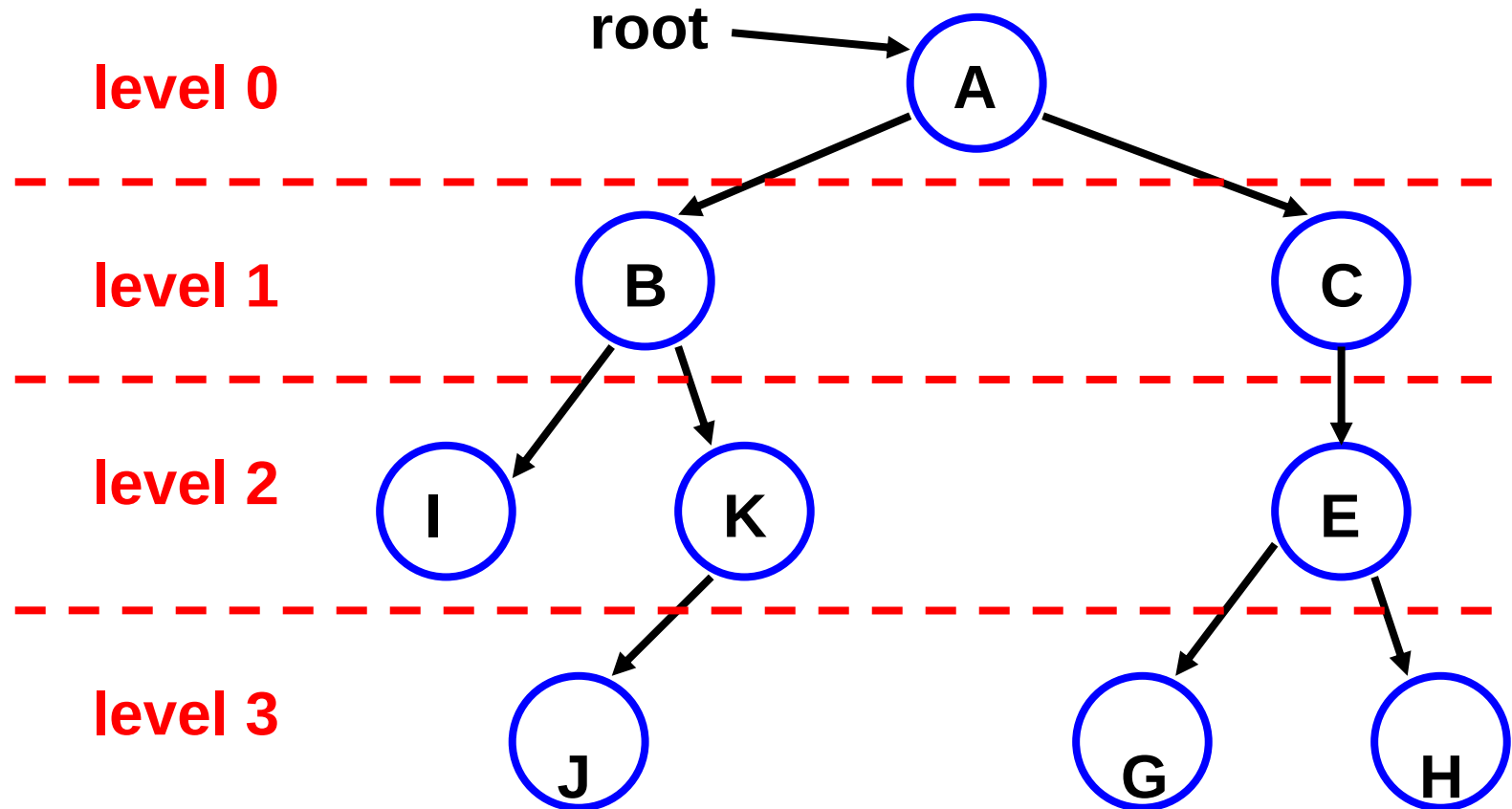
NOT a Binary Tree



Example of a Binary Tree

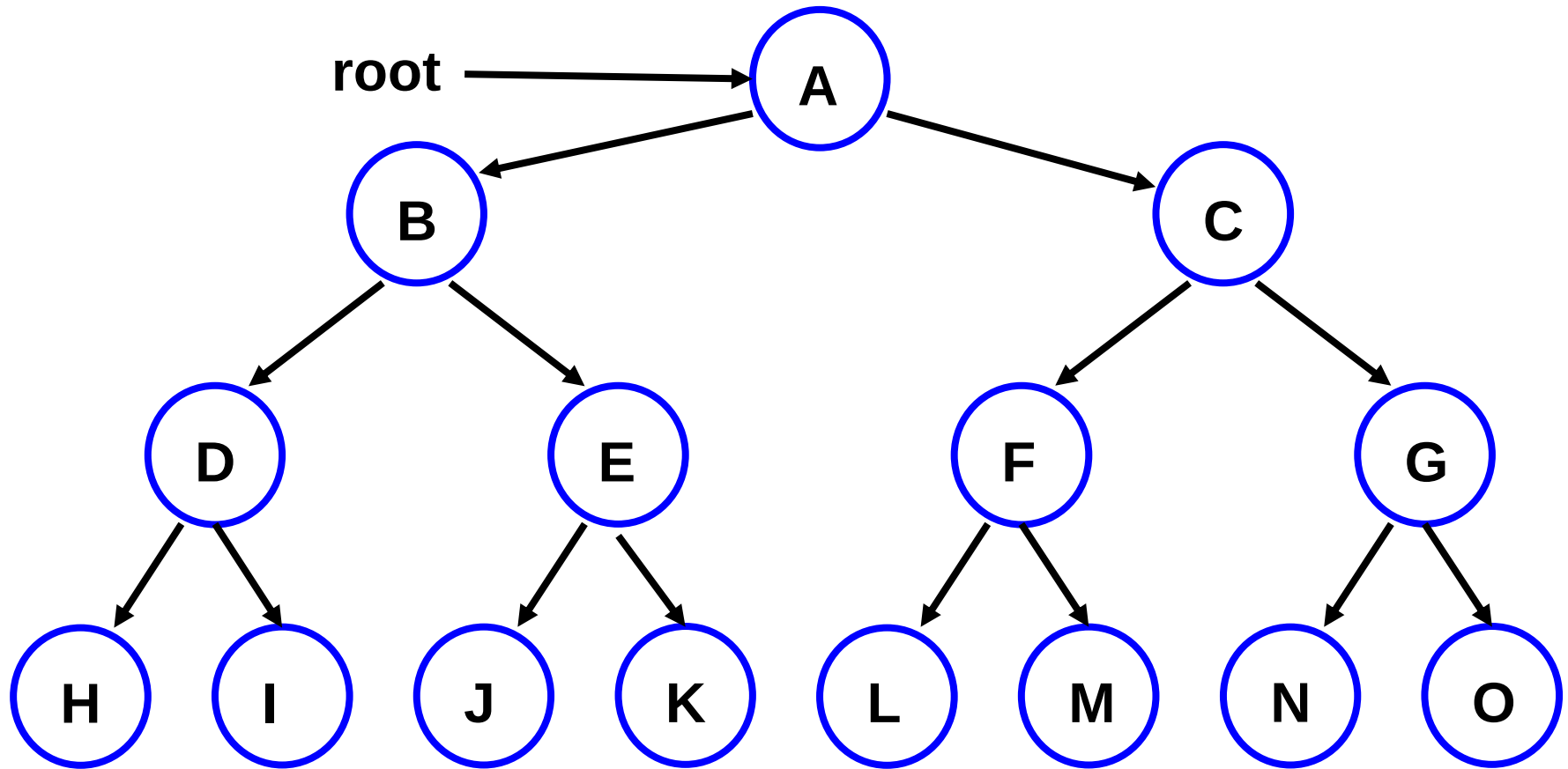


Levels



The *level* of a node is the number of edges in the path from the root node to this node

Full Binary Tree

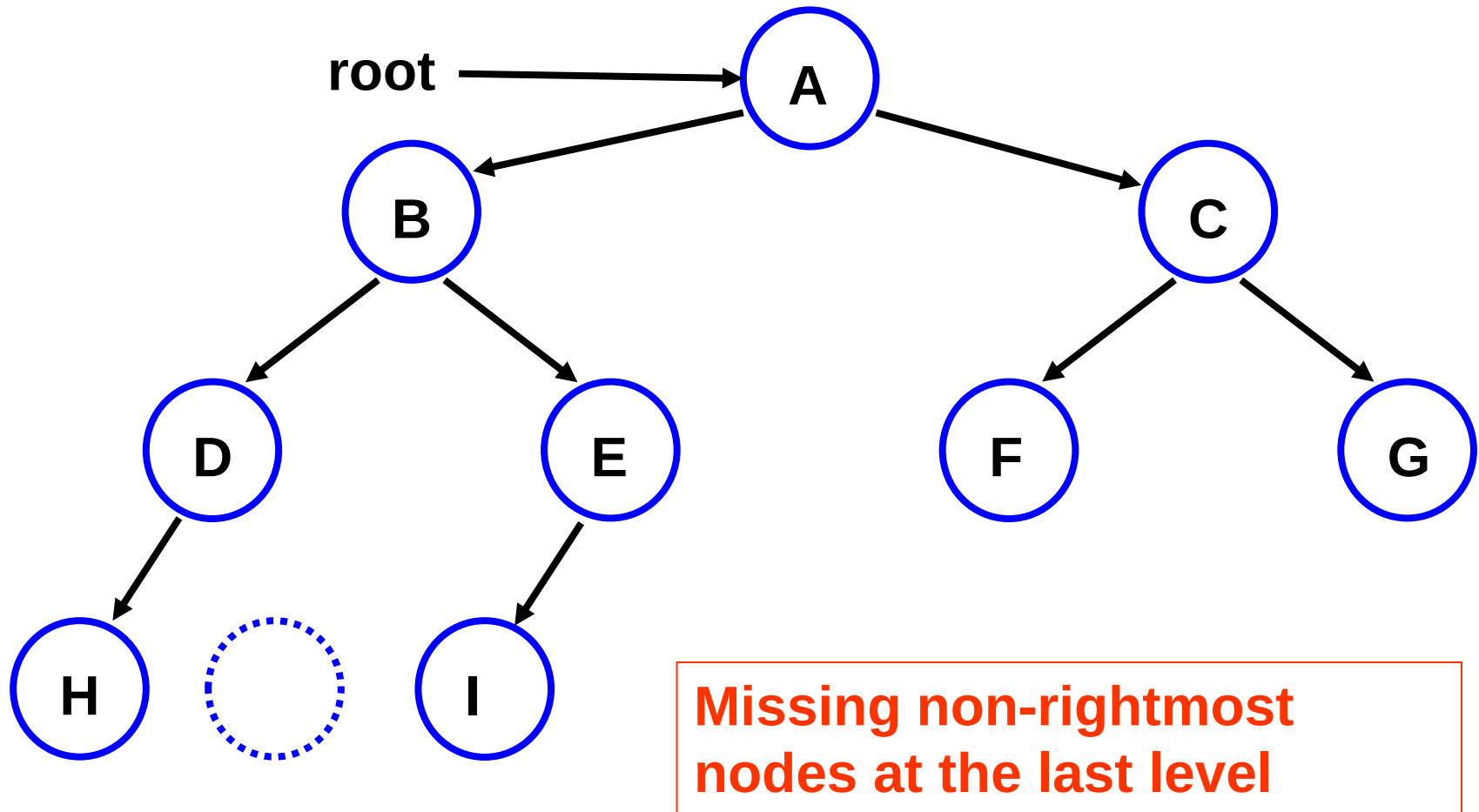


In a *full binary tree*, each node has two children except for the nodes on the last level, which are leaf nodes

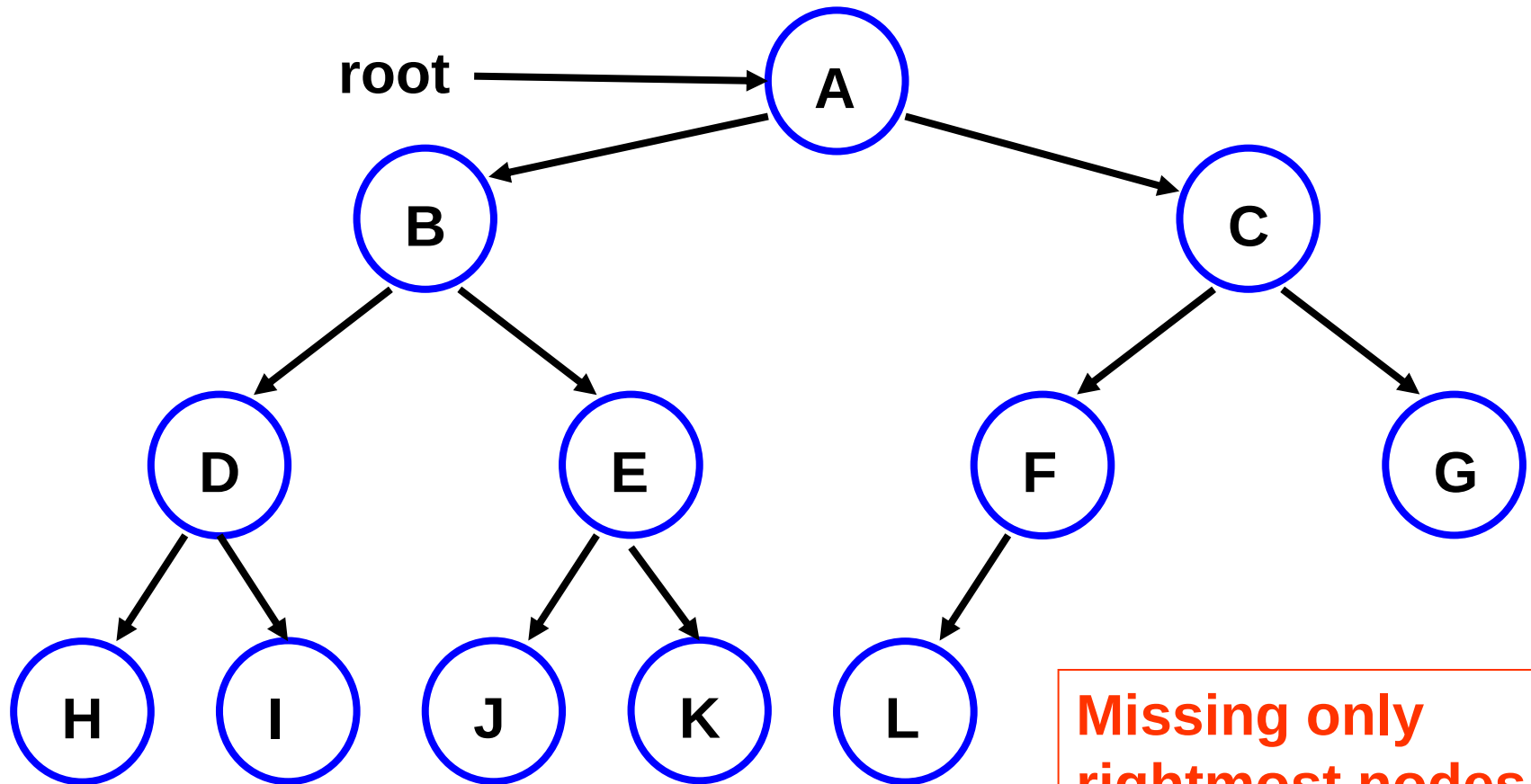
Complete Binary Trees

- A ***complete binary tree*** is a binary tree that is either
 - a full binary tree
 - OR
 - a tree that would be a full binary tree but it is missing the rightmost nodes on the last level

NOT a Complete Binary Trees



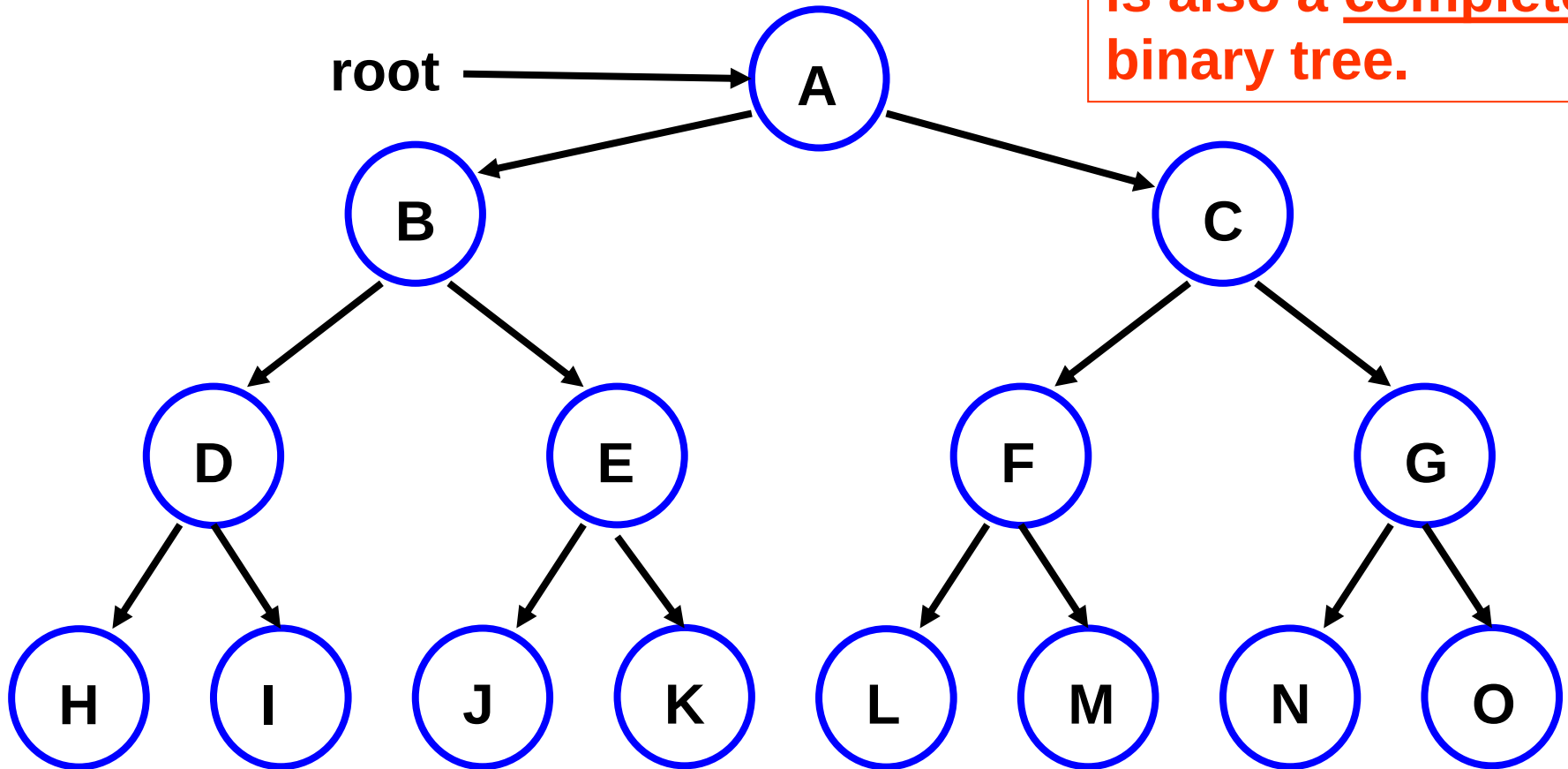
Complete Binary Trees (cont.)



**Missing only
rightmost nodes at
the last level**

Complete Binary Trees (cont.)

A full binary tree is also a complete binary tree.



Binary Search Trees (BST)

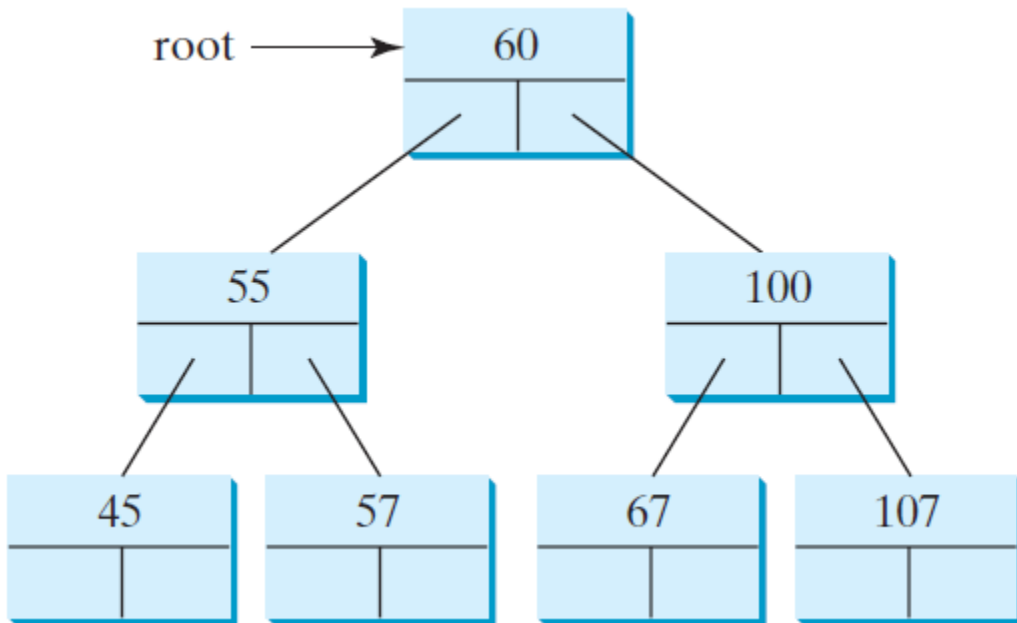
- A **binary search tree** is a binary tree that allows us to search for values that can be anywhere in the tree.
- Usually, we search for a certain key value, and once we find the node that contains it, we retrieve the rest of the info at that node.

Properties of BST

- For any node,
 - the key in its left child (if any) is less than its key.
 - the key in its right child (if any) is greater than or equal to its key.
- $(\text{Left} < \text{Parent} \leq \text{Right})$ or $(\text{Left} \leq \text{Parent} < \text{Right})$

BST Node

```
// Java
class BSTNode {
    int element;
    BSTNode left; // reference/pointer to left node
    BSTNode right; // reference/pointer to right node
}
```



The implementation of a BST usually just maintains a single reference/pointer to the root node.

Inserting Nodes Into a BST

root is null

BST starts off empty

Objects that need to be inserted (only key values are shown):

37, 2, 45, 48, 41, 29, 20, 30, 49, 7

Pseudocode

Algorithm `insertElement (p, e)`

Input: `e` is the element to be inserted under node `p`, without rotation.

Output: if `e` is inserted as the left child of `p` if `e` is less than `p`; else `e` is inserted as the left child of `p`

```
if p == null
```

```
    p.element = e
```

```
else if e < p.element
```

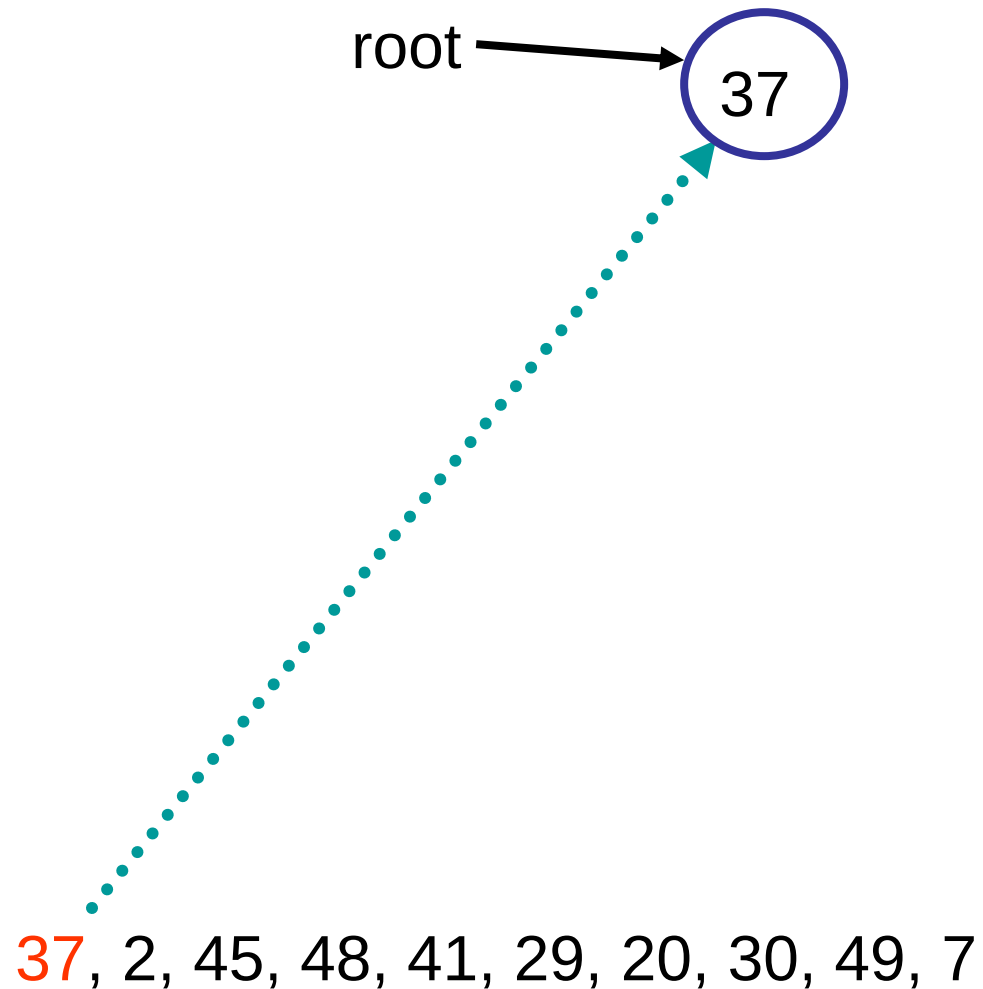
```
    p.left = insertElement (p.left, e)
```

```
else
```

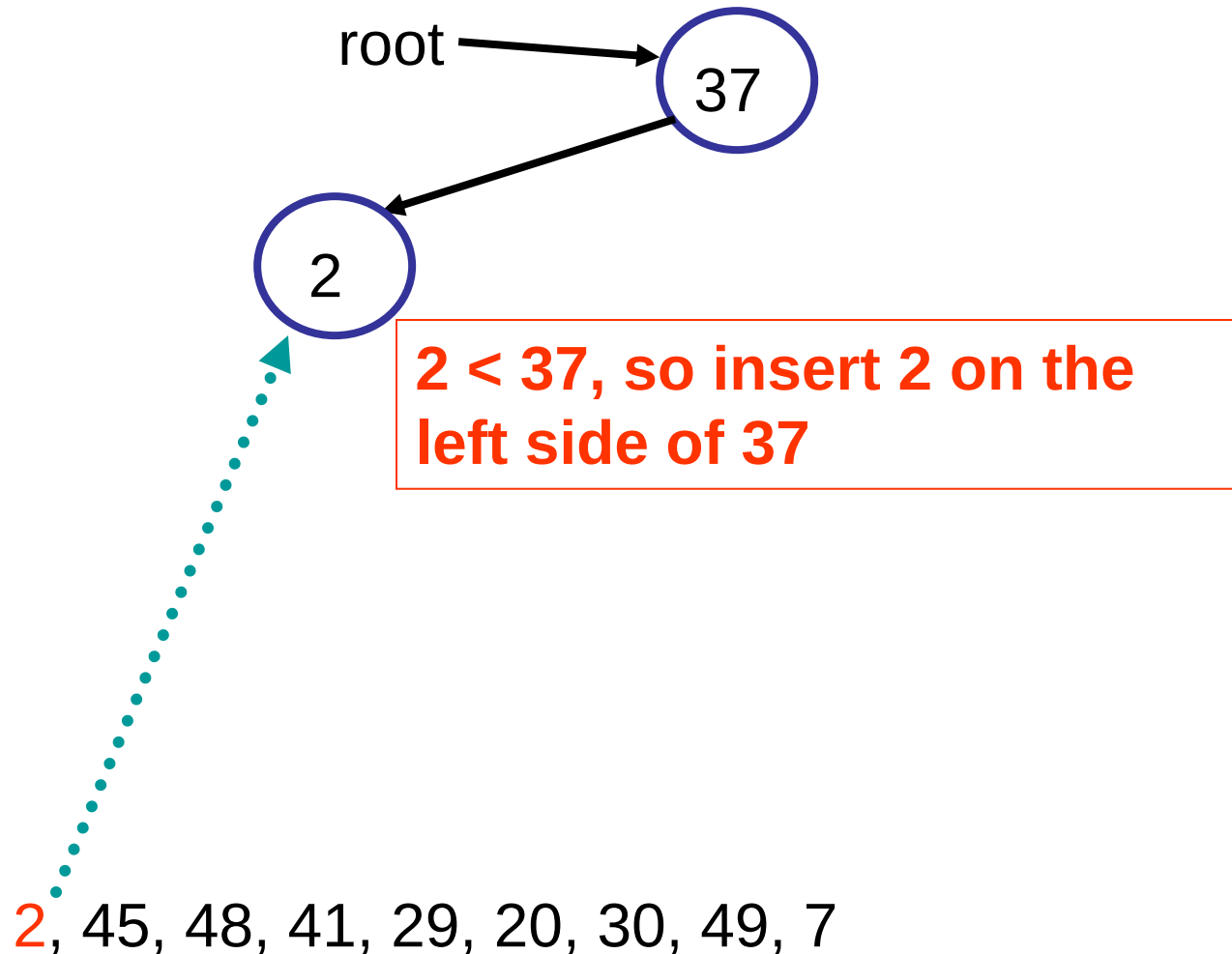
```
    p.right = insertElement (p.right, e);
```

```
return p
```

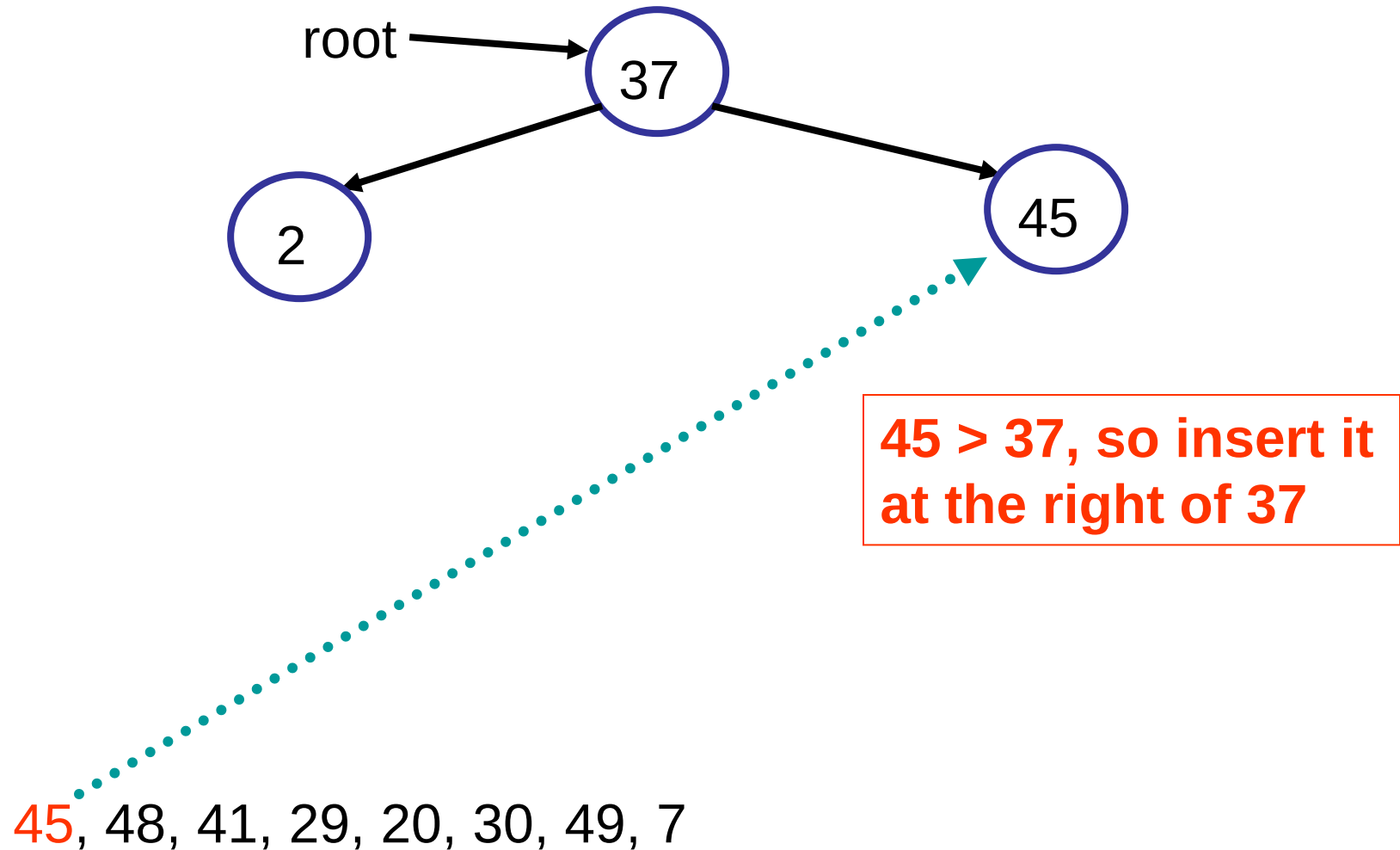
Inserting Nodes Into a BST (cont.)



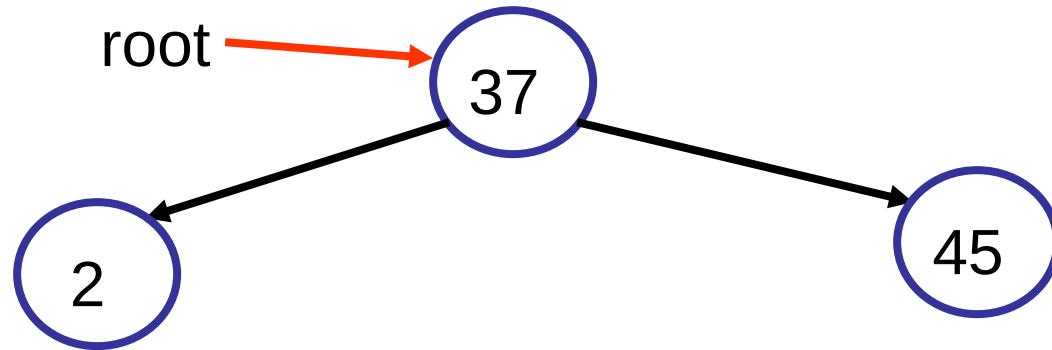
Inserting Nodes Into a BST (cont.)



Inserting Nodes Into a BST (cont.)



Inserting Nodes Into a BST (cont.)

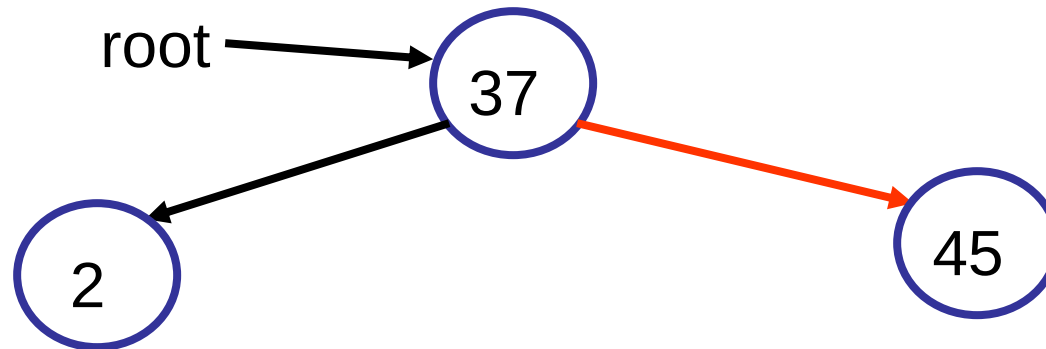


When comparing, we always start at the root node

$48 > 37$, so look to the right

48, 41, 29, 20, 30, 49, 7

Inserting Nodes Into a BST (cont.)

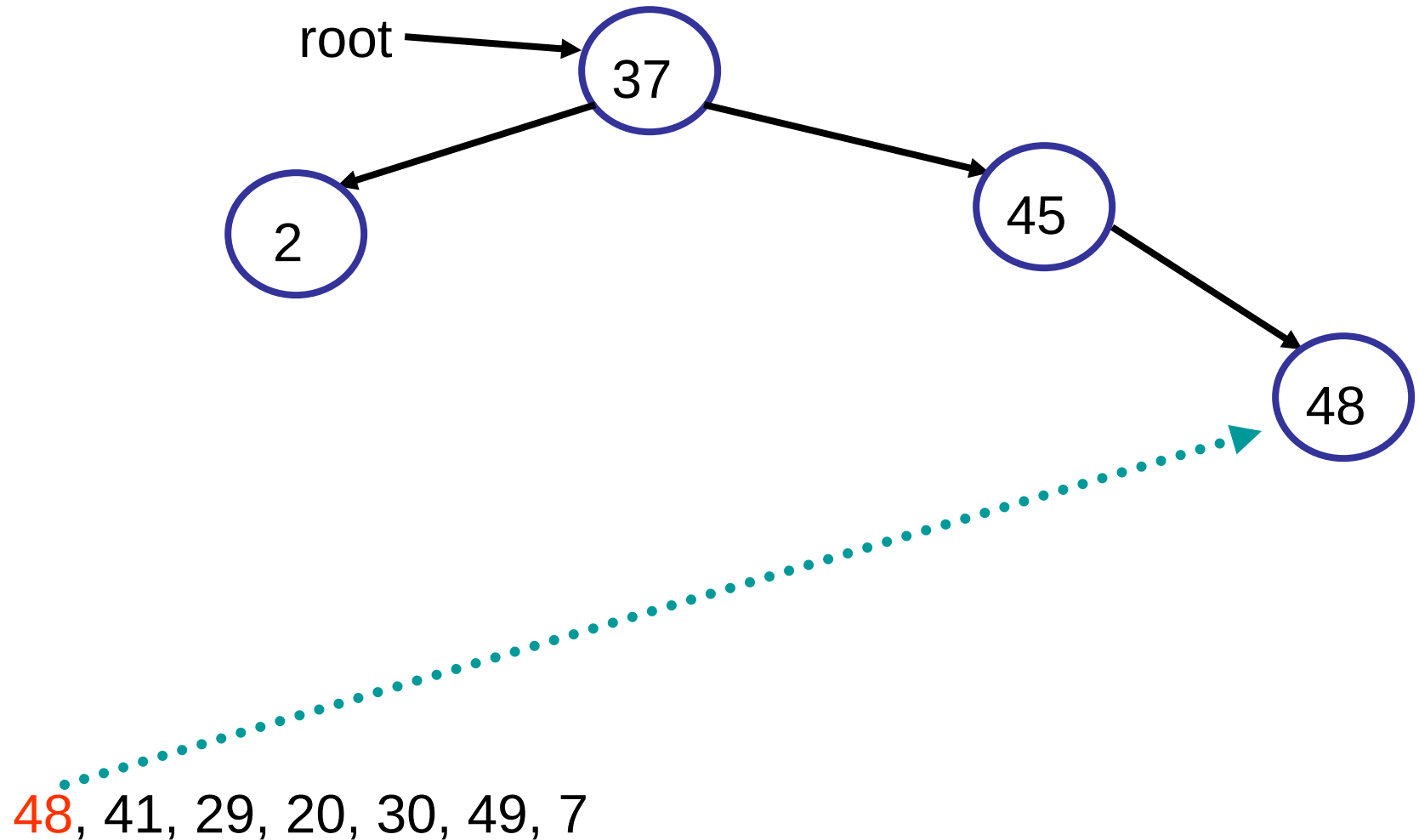


This time, there is a node already to the right of the root node. We then compare 48 to this node

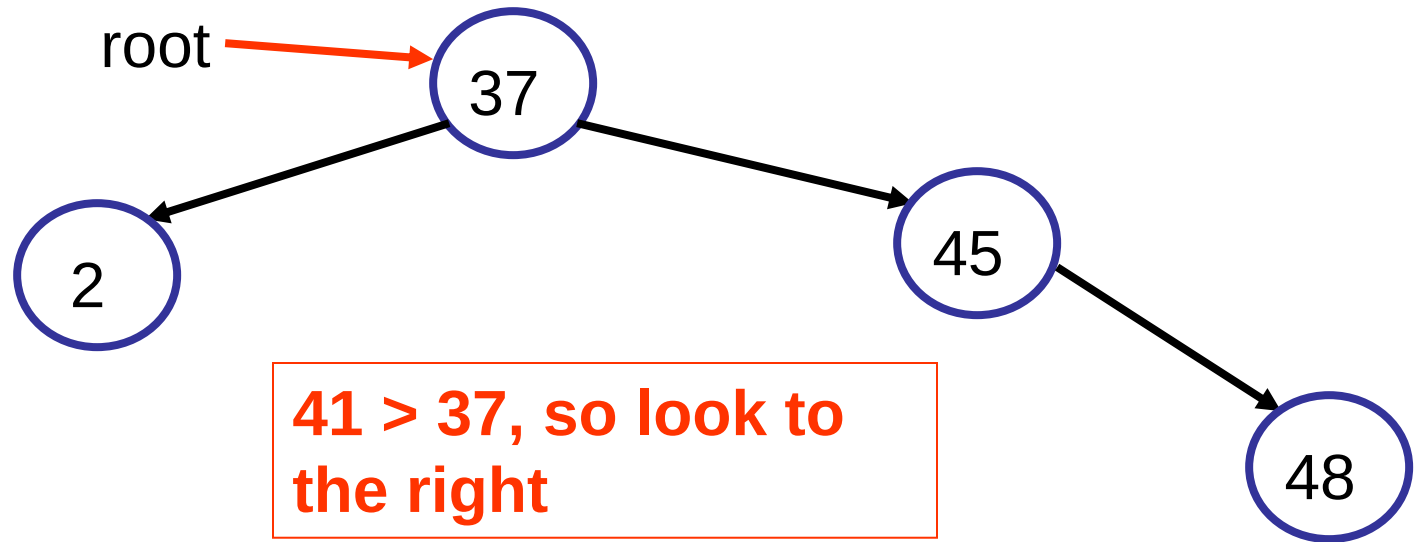
$48 > 45$, and 45 has no right child, so we insert 48 on the right of 45

48, 41, 29, 20, 30, 49, 7

Inserting Nodes Into a BST (cont.)

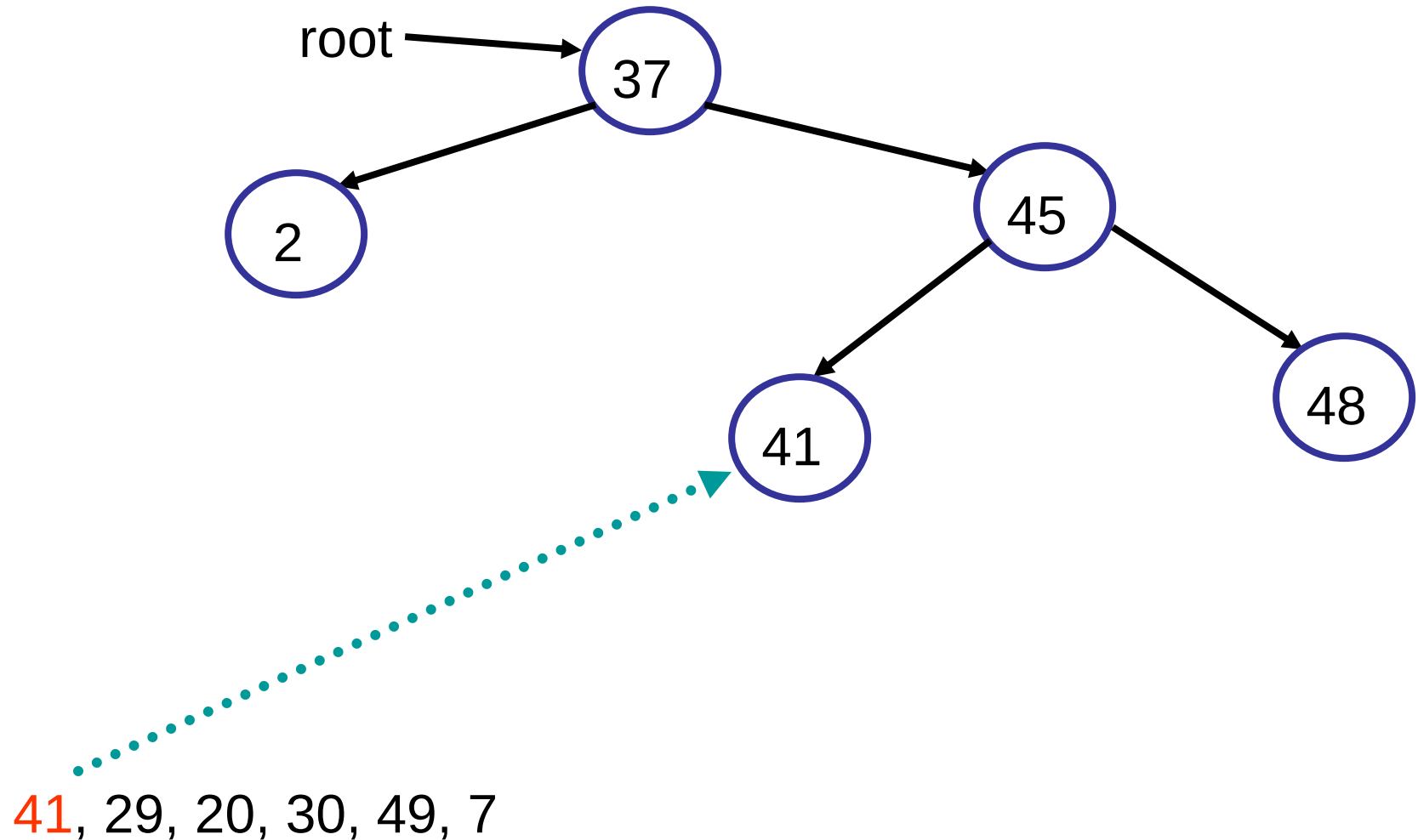


Inserting Nodes Into a BST (cont.)

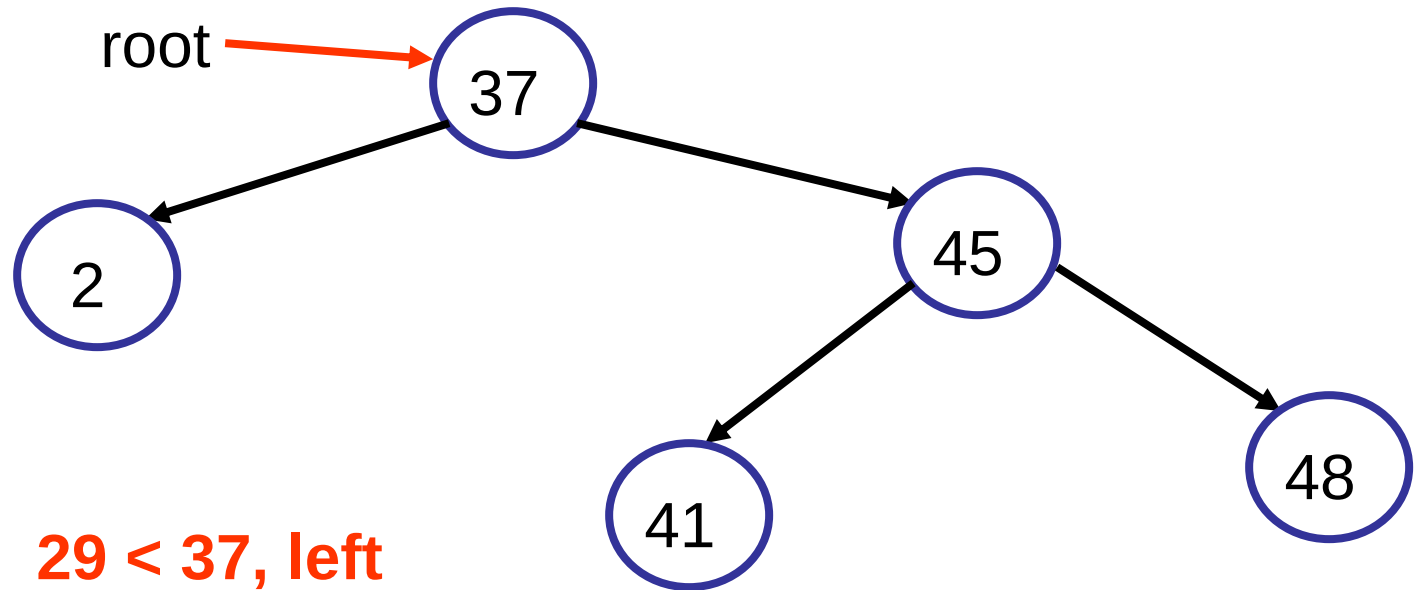


41, 29, 20, 30, 49, 7

Inserting Nodes Into a BST (cont.)

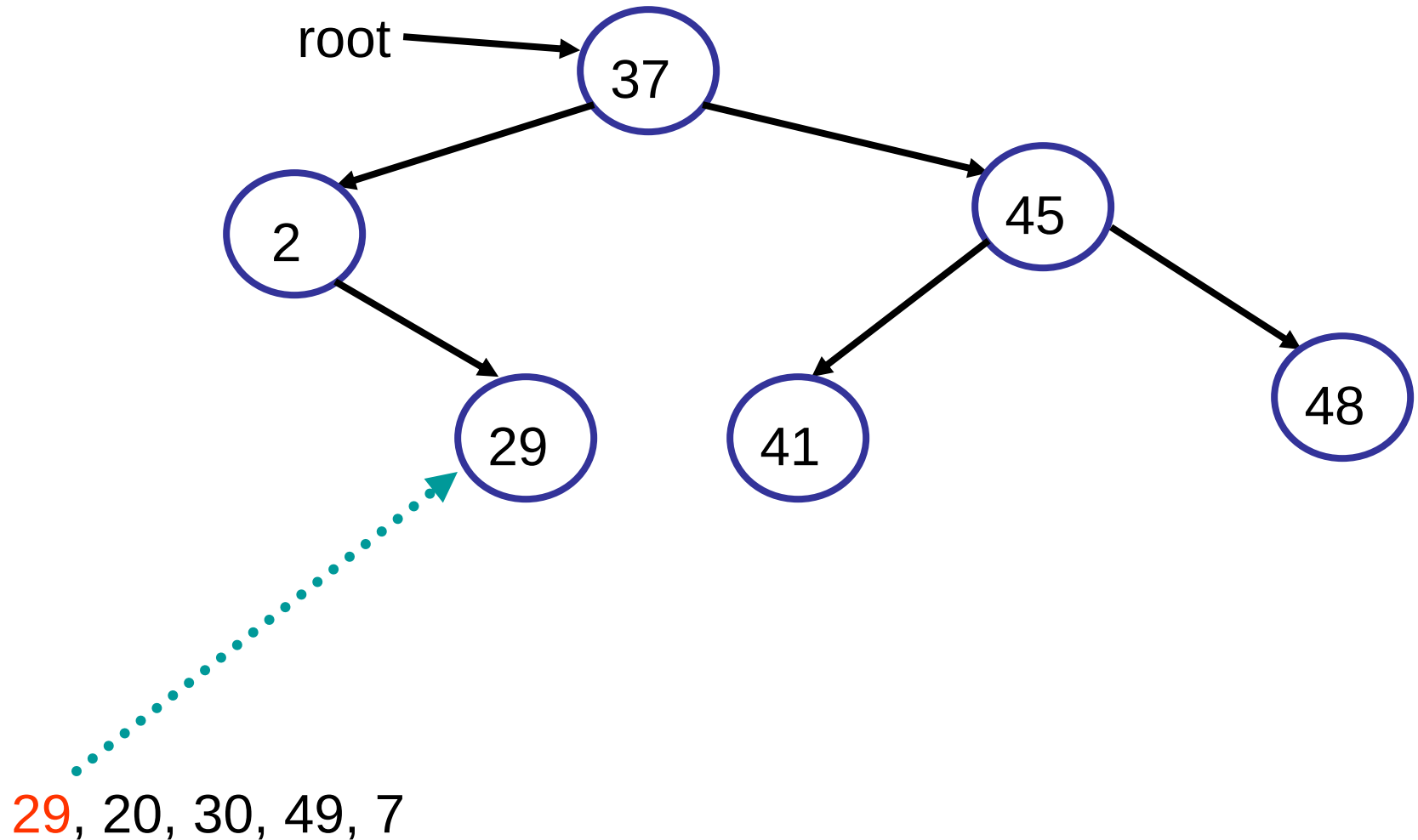


Inserting Nodes Into a BST (cont.)

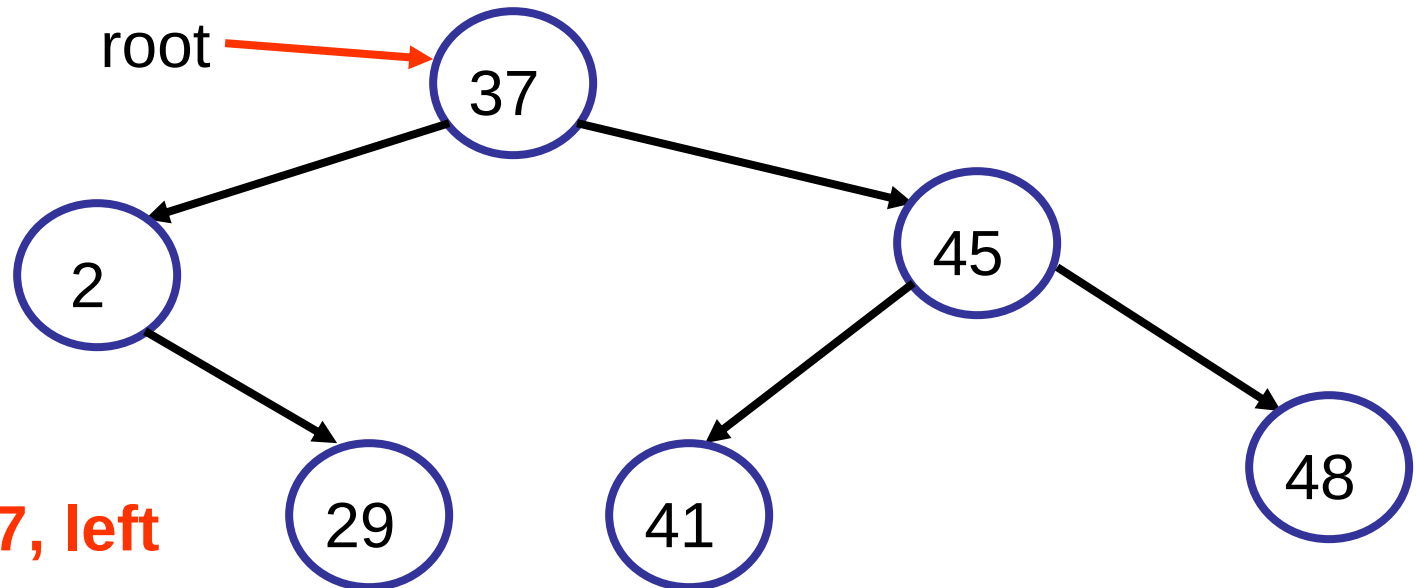


29, 20, 30, 49, 7

Inserting Nodes Into a BST (cont.)



Inserting Nodes Into a BST (cont.)



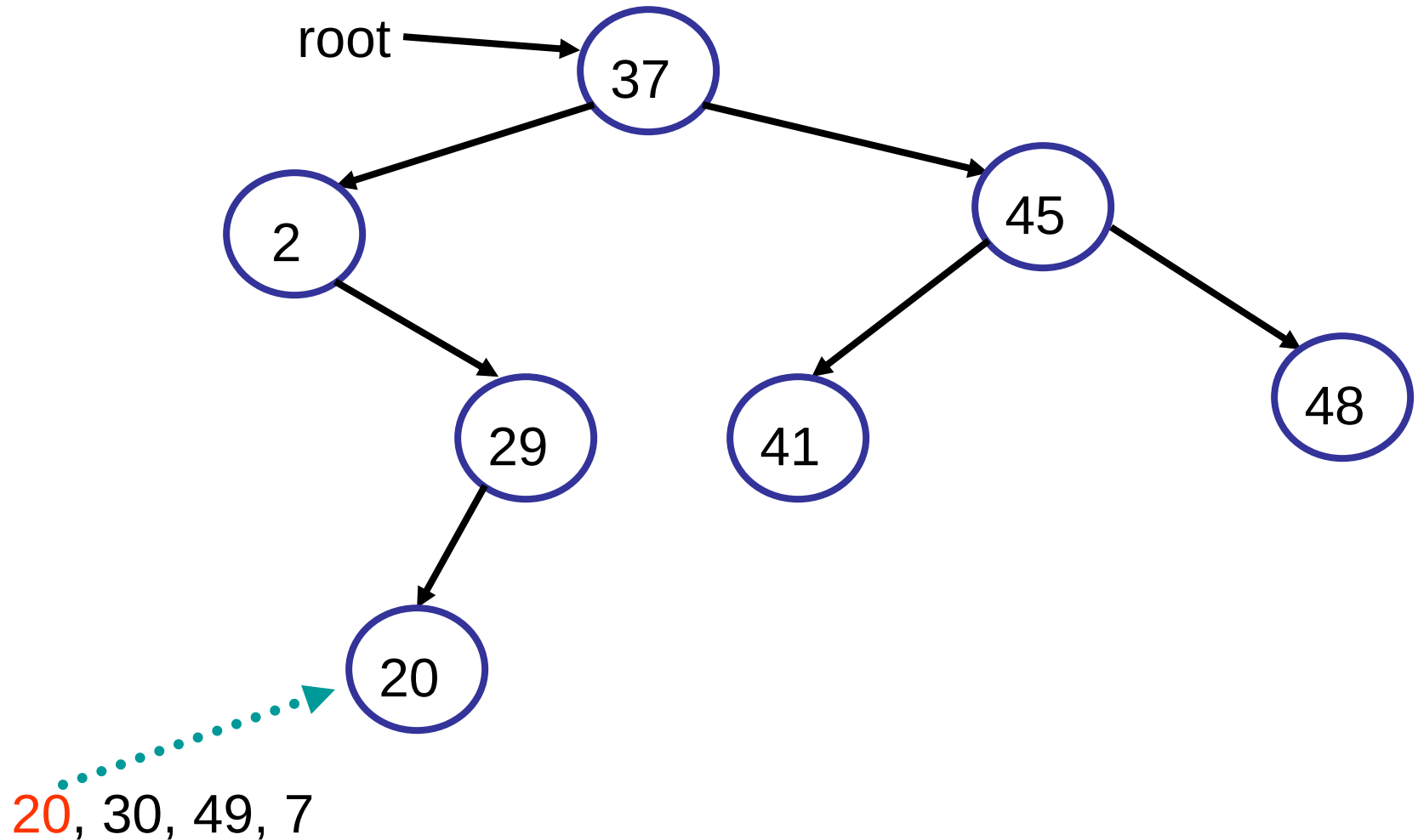
20 < 37, left

20 > 2, right

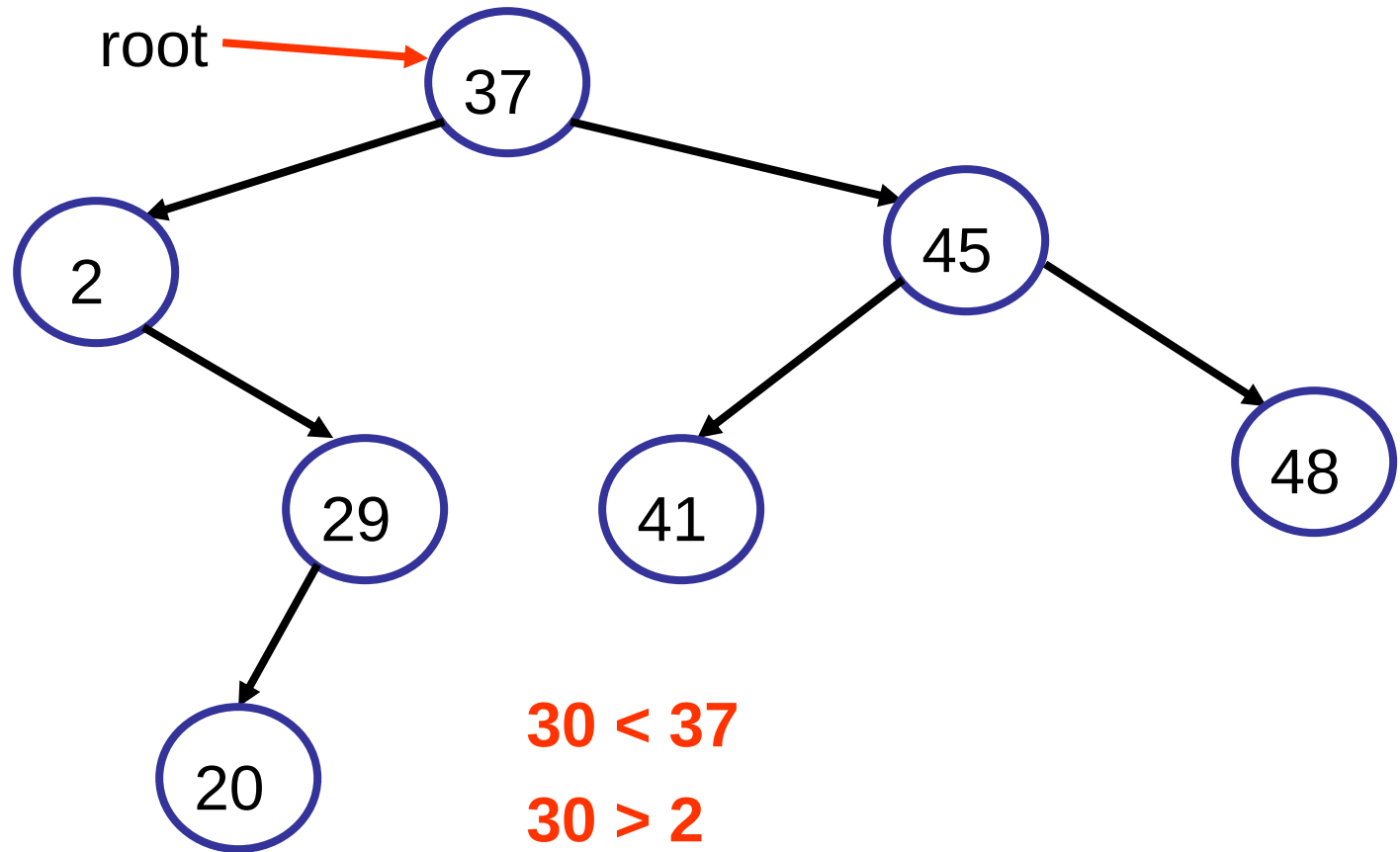
20 < 29, left

20, 30, 49, 7

Inserting Nodes Into a BST (cont.)



Inserting Nodes Into a BST (cont.)



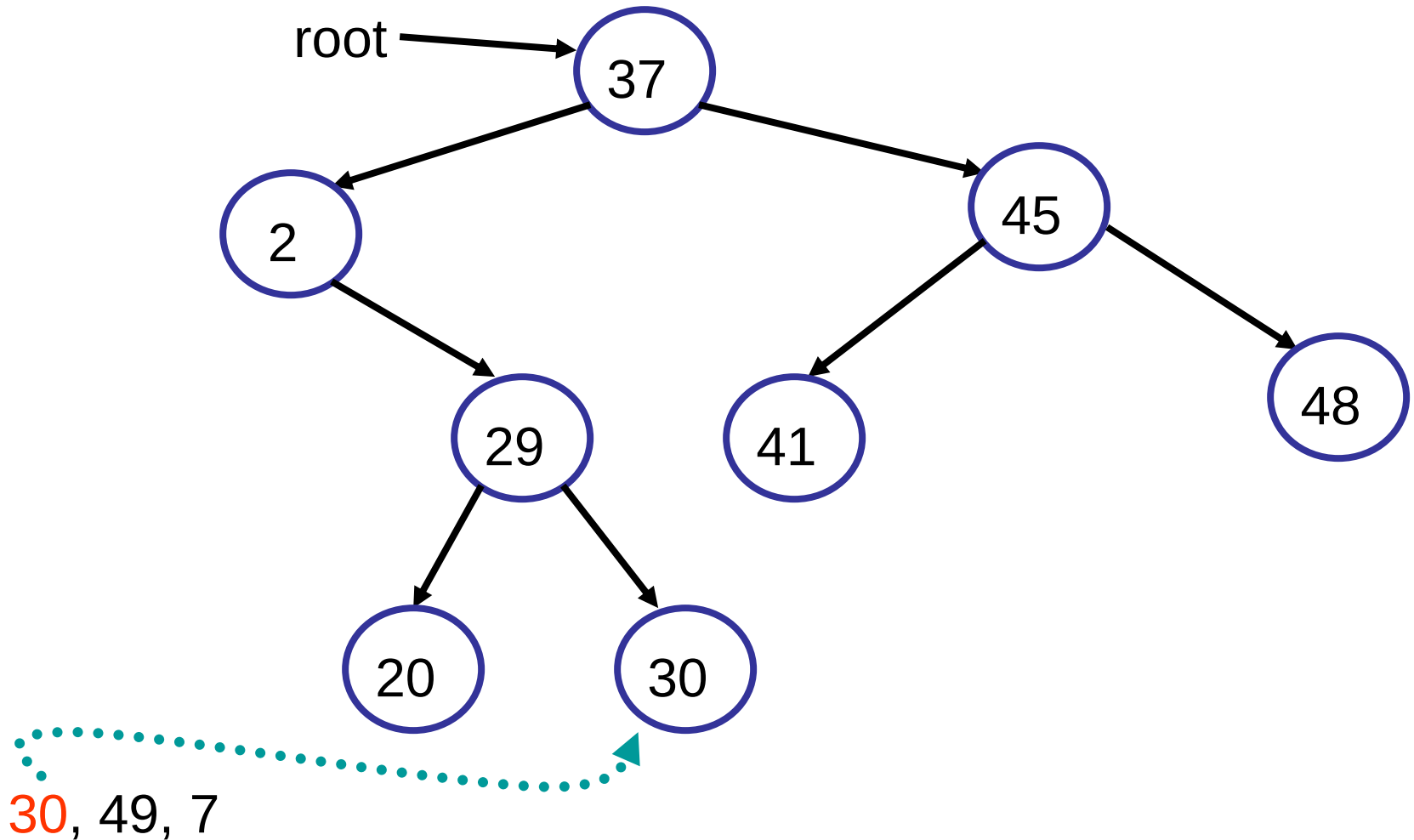
30, 49, 7

$30 < 37$

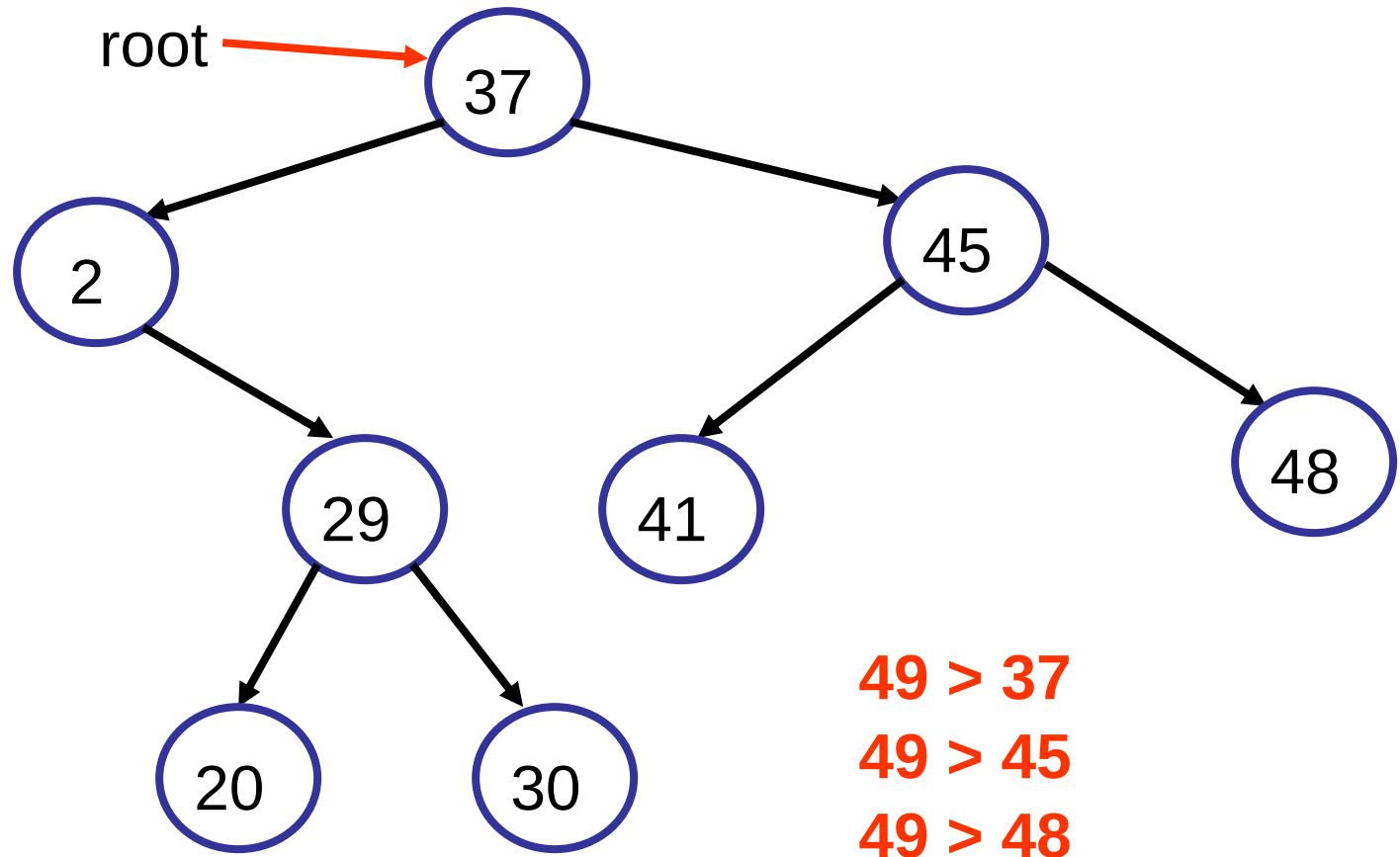
$30 > 2$

$30 > 29$

Inserting Nodes Into a BST (cont.)

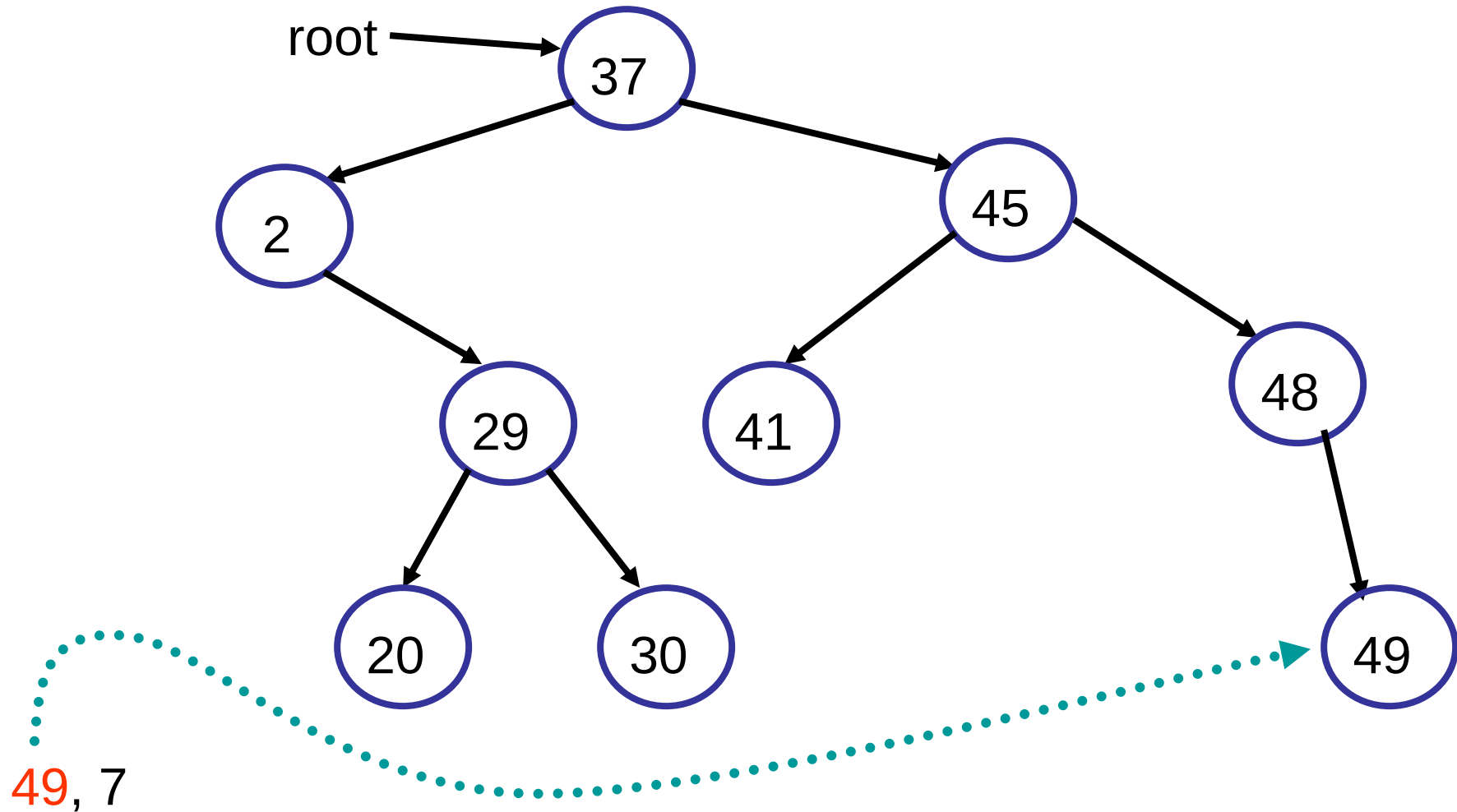


Inserting Nodes Into a BST (cont.)

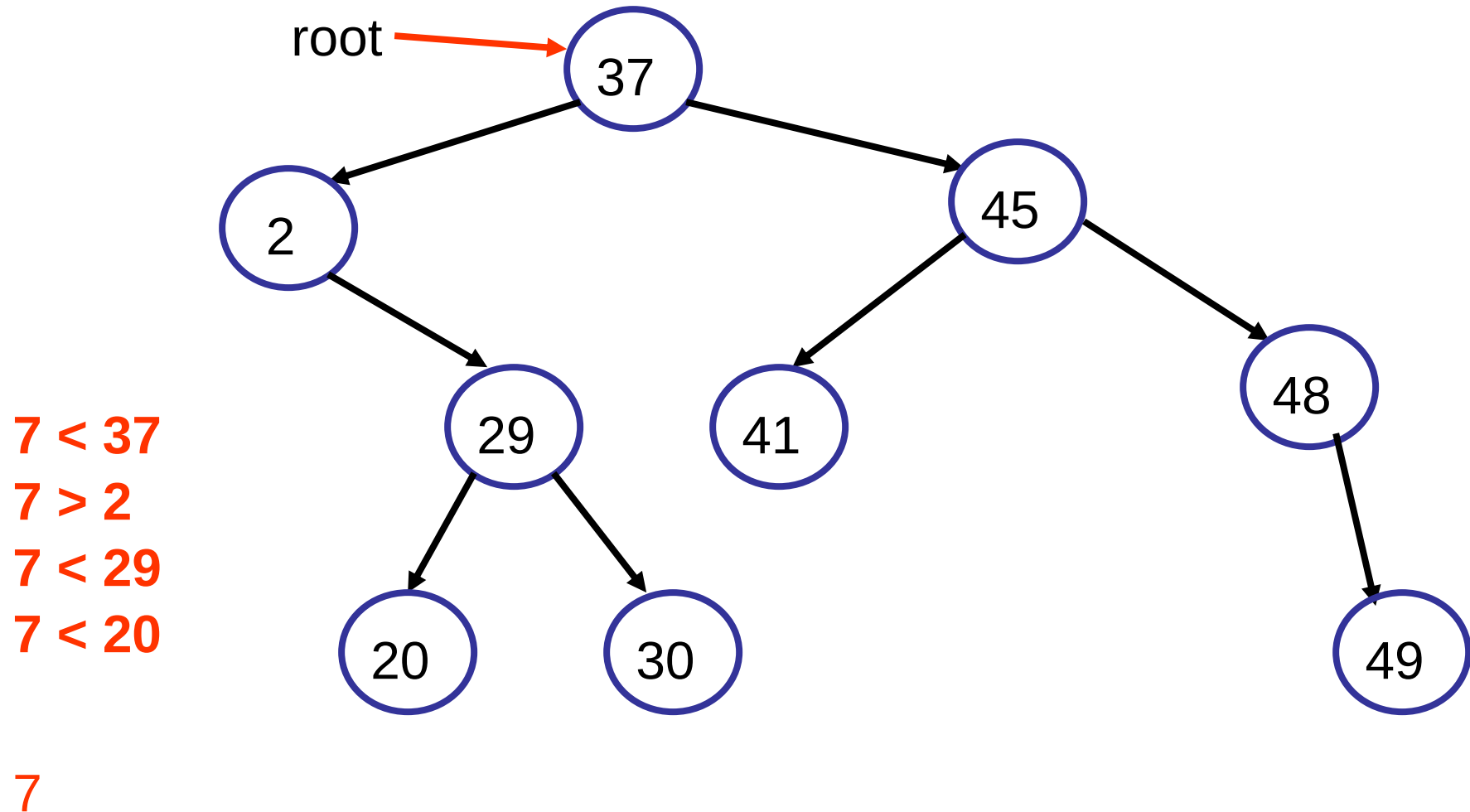


49, 7

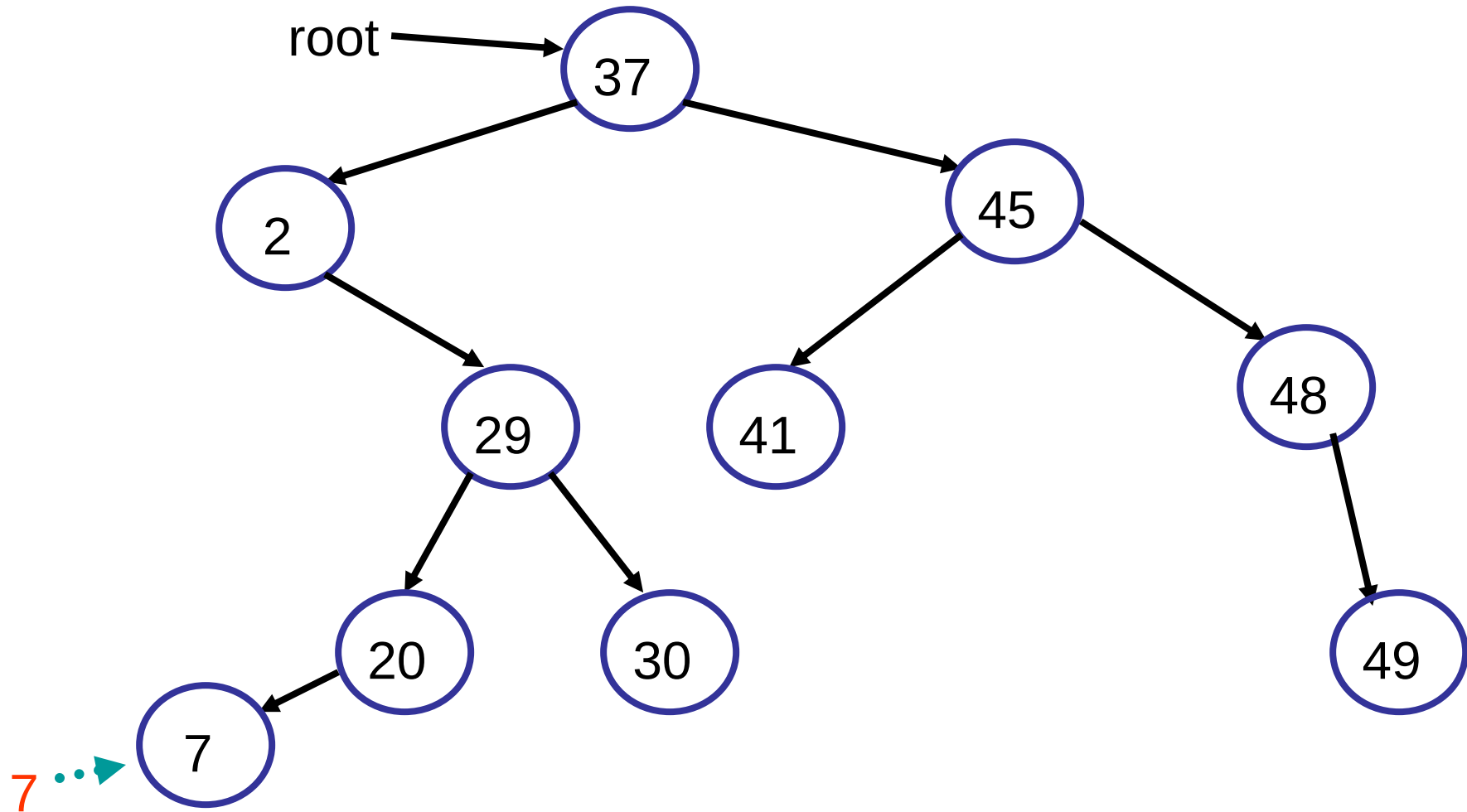
Inserting Nodes Into a BST (cont.)



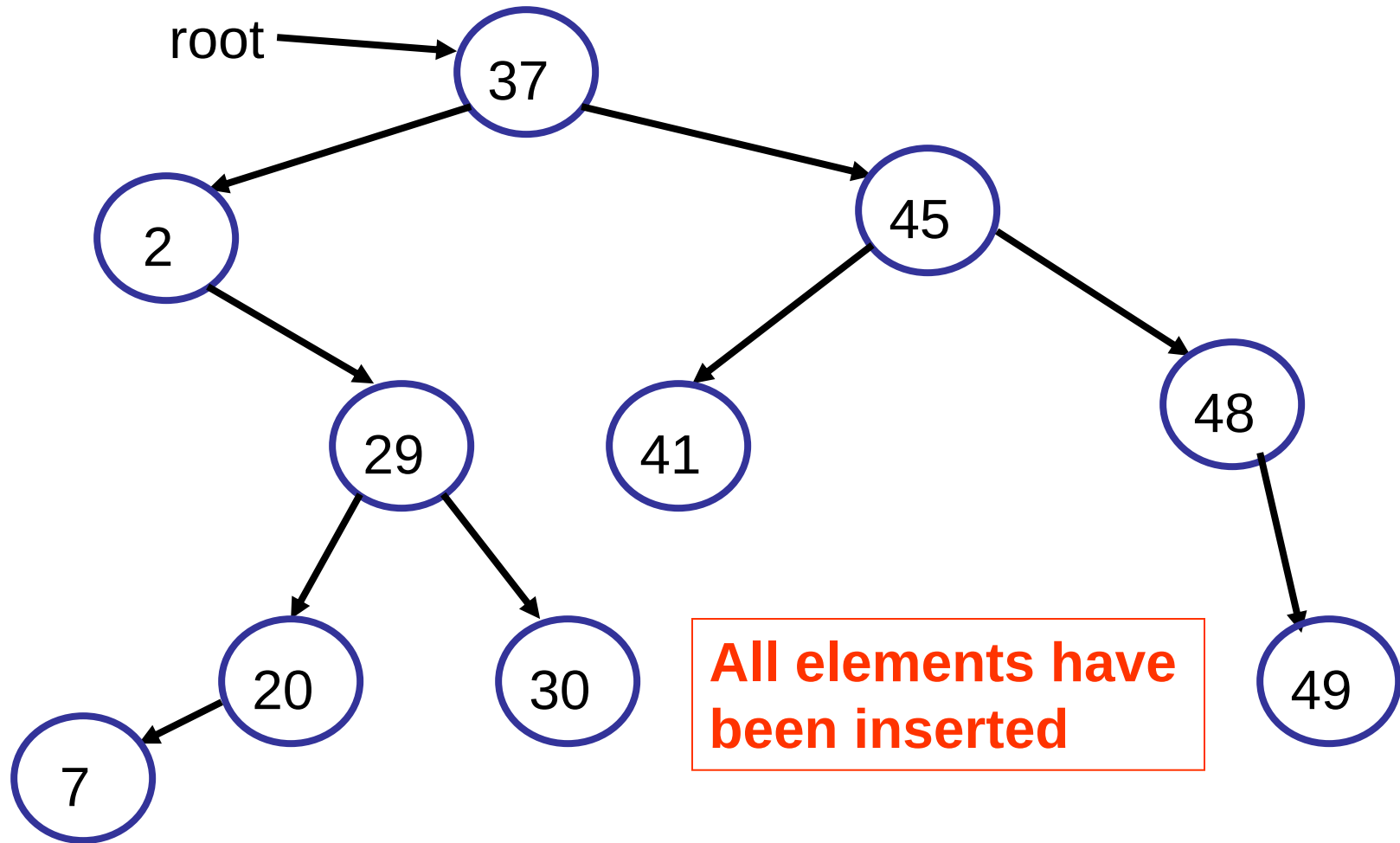
Inserting Nodes Into a BST (cont.)



Inserting Nodes Into a BST (cont.)



Inserting Nodes Into a BST (cont.)



Searching for a Key in a BST

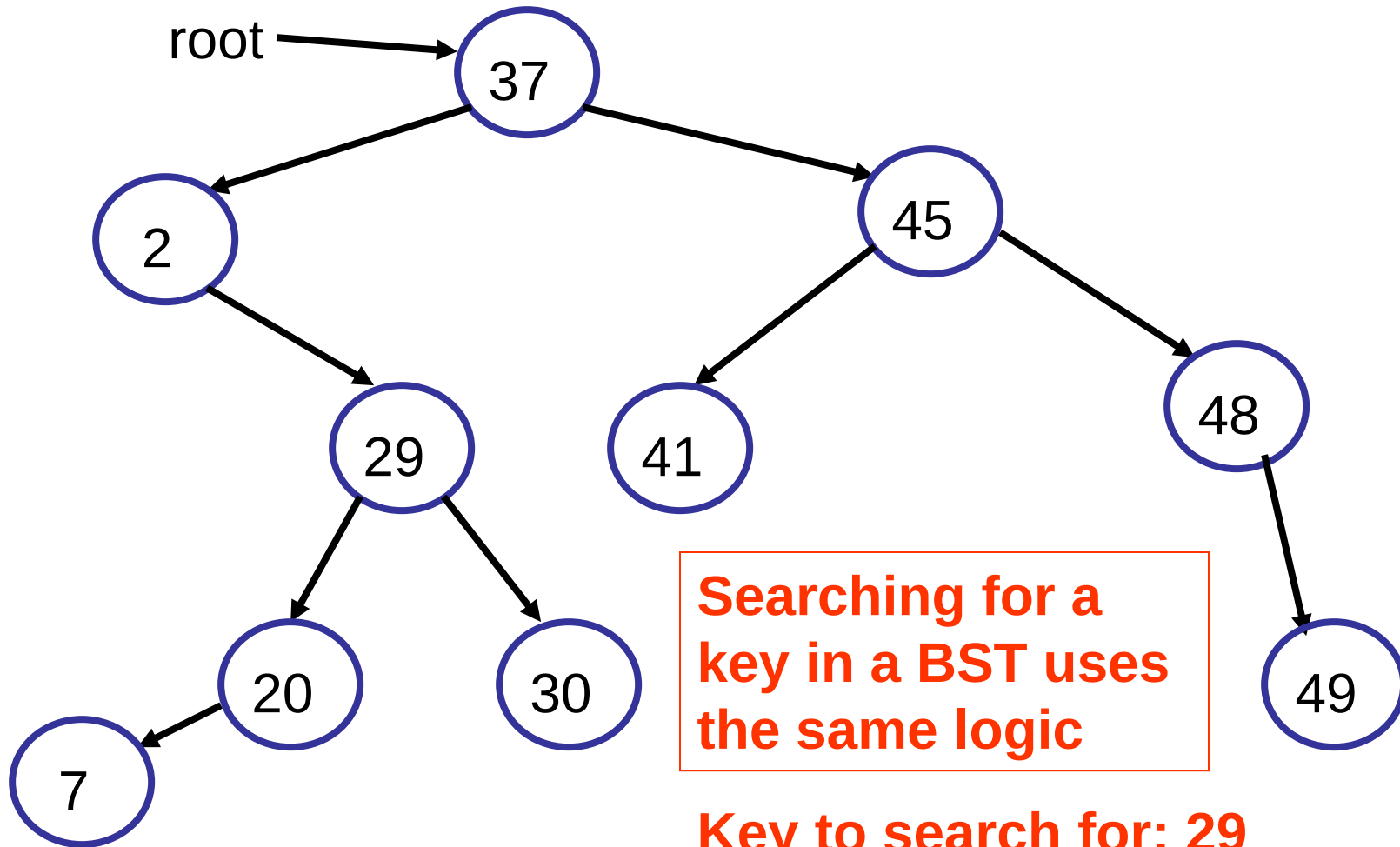
Algorithm `findElement (p, k)`

Input: find target `k` from node `p` and its children.

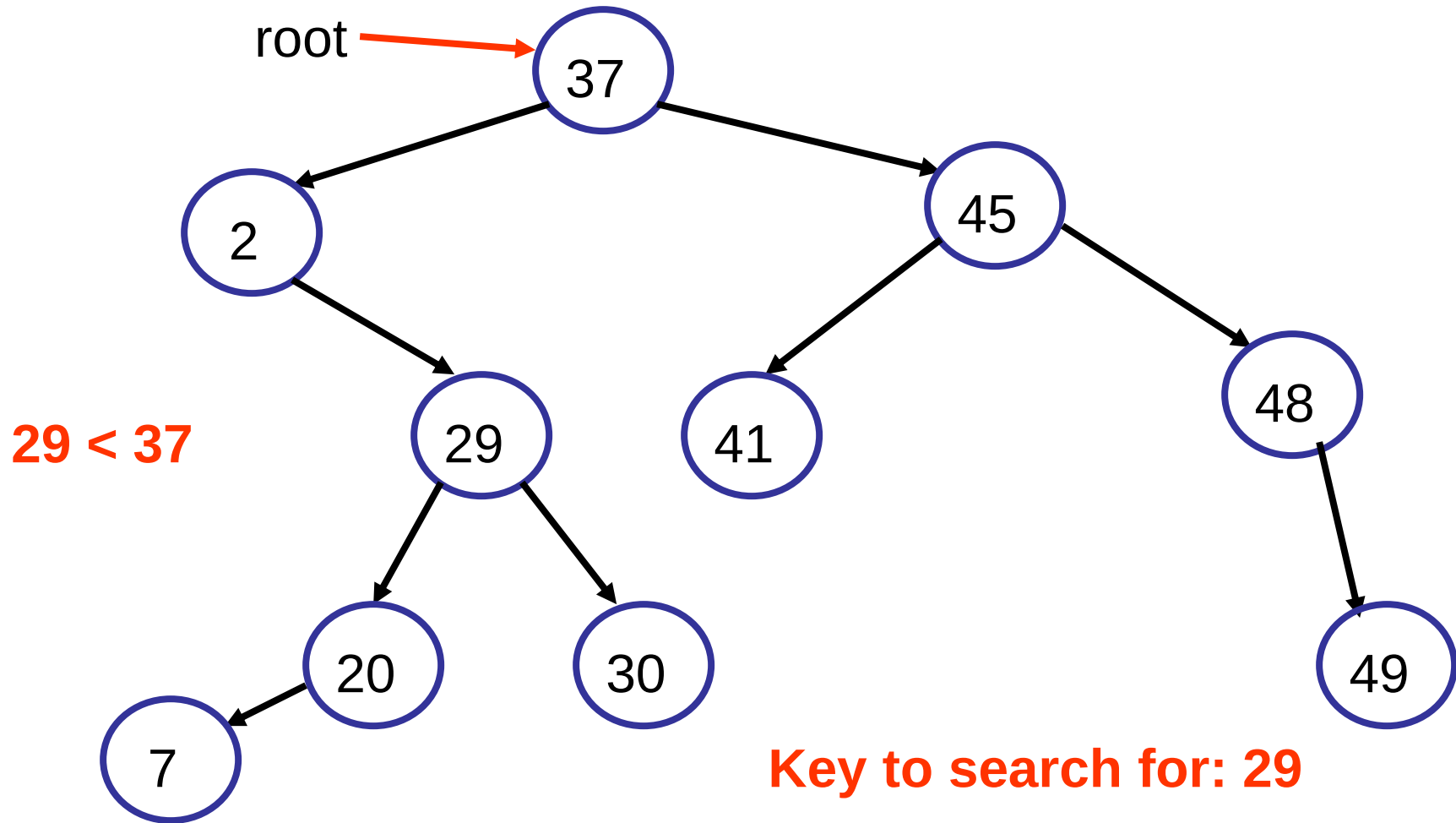
Output: return `p` if `k` is found; else return `NO_SUCH_KEY`.

```
if p == null
    return NO_SUCH_KEY
if k == p.element
    return p
else if k < p.element
    return findElement (k, p.left)
else // k > p.element
    return findElement (k, p.right)
```

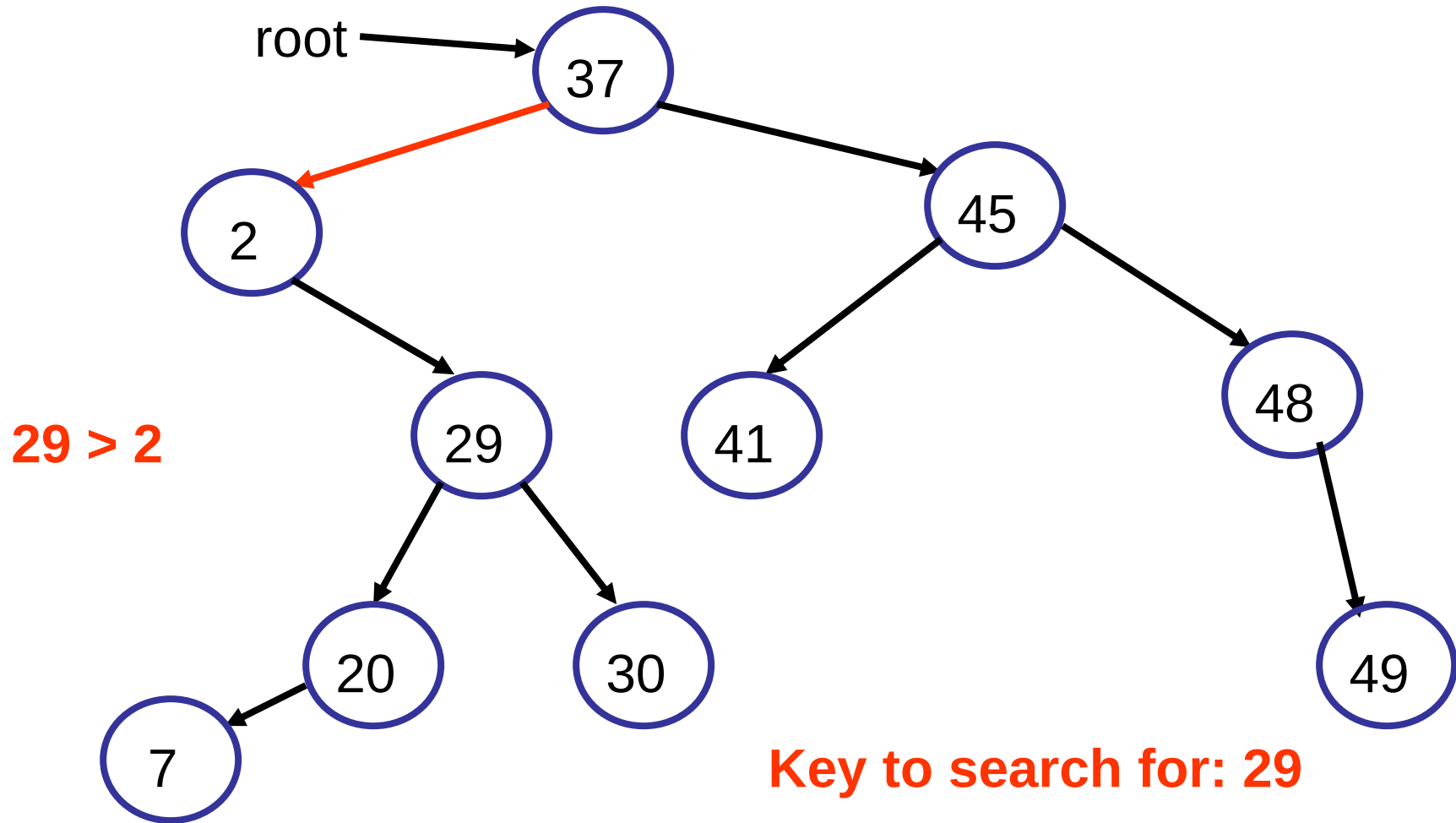
Searching for a Key in a BST



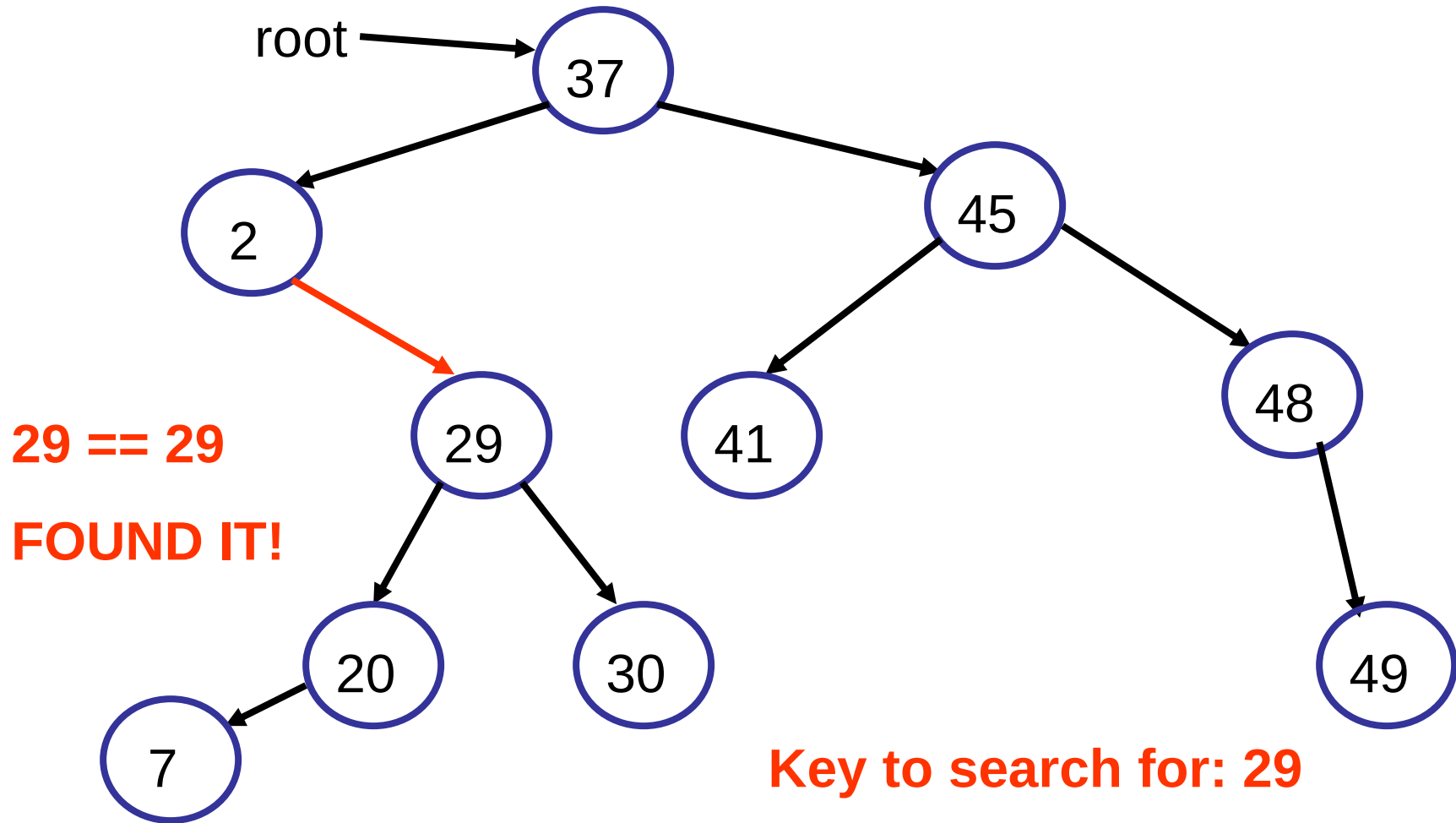
Searching for a Key in a BST (cont.)



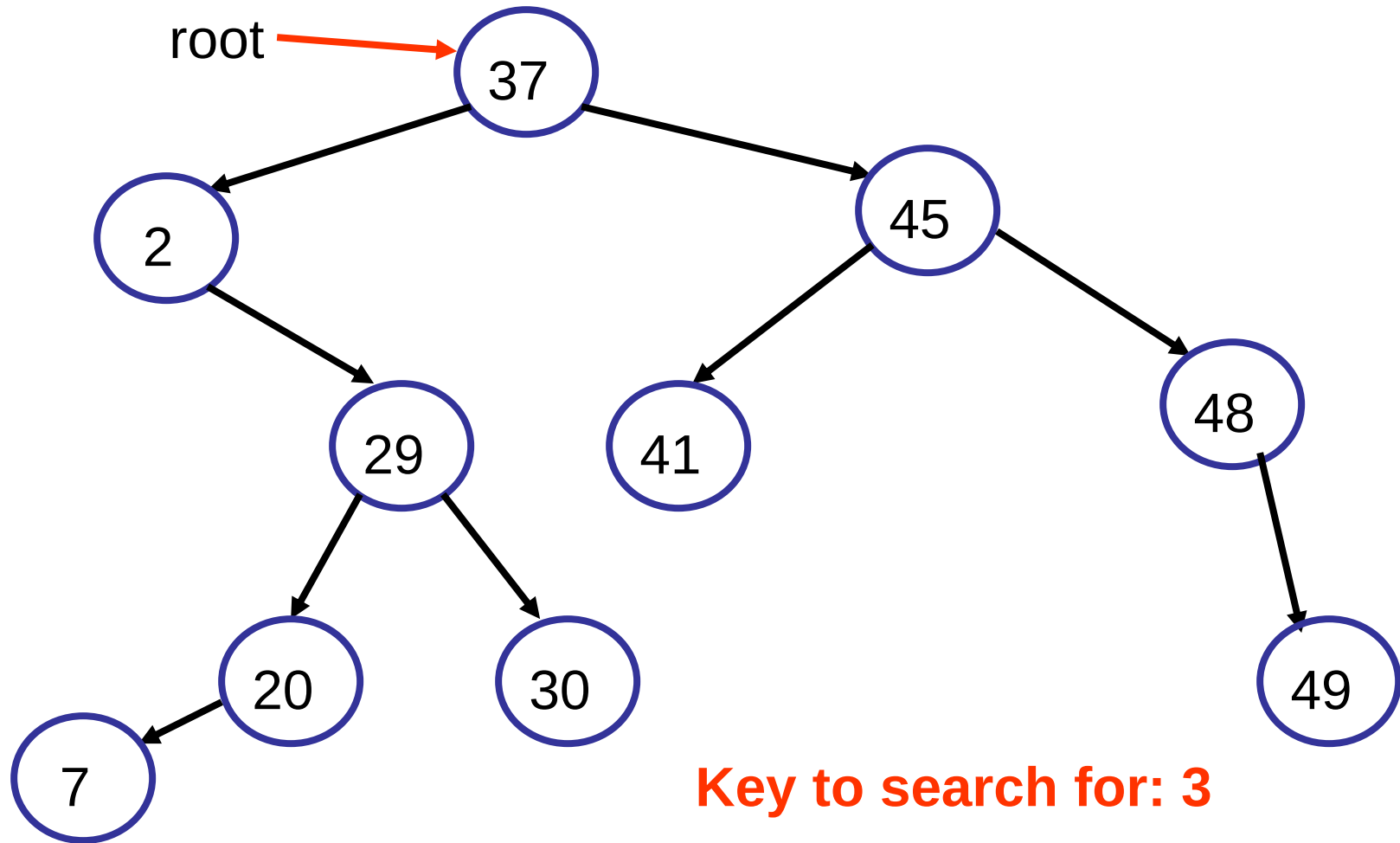
Searching for a Key in a BST (cont.)



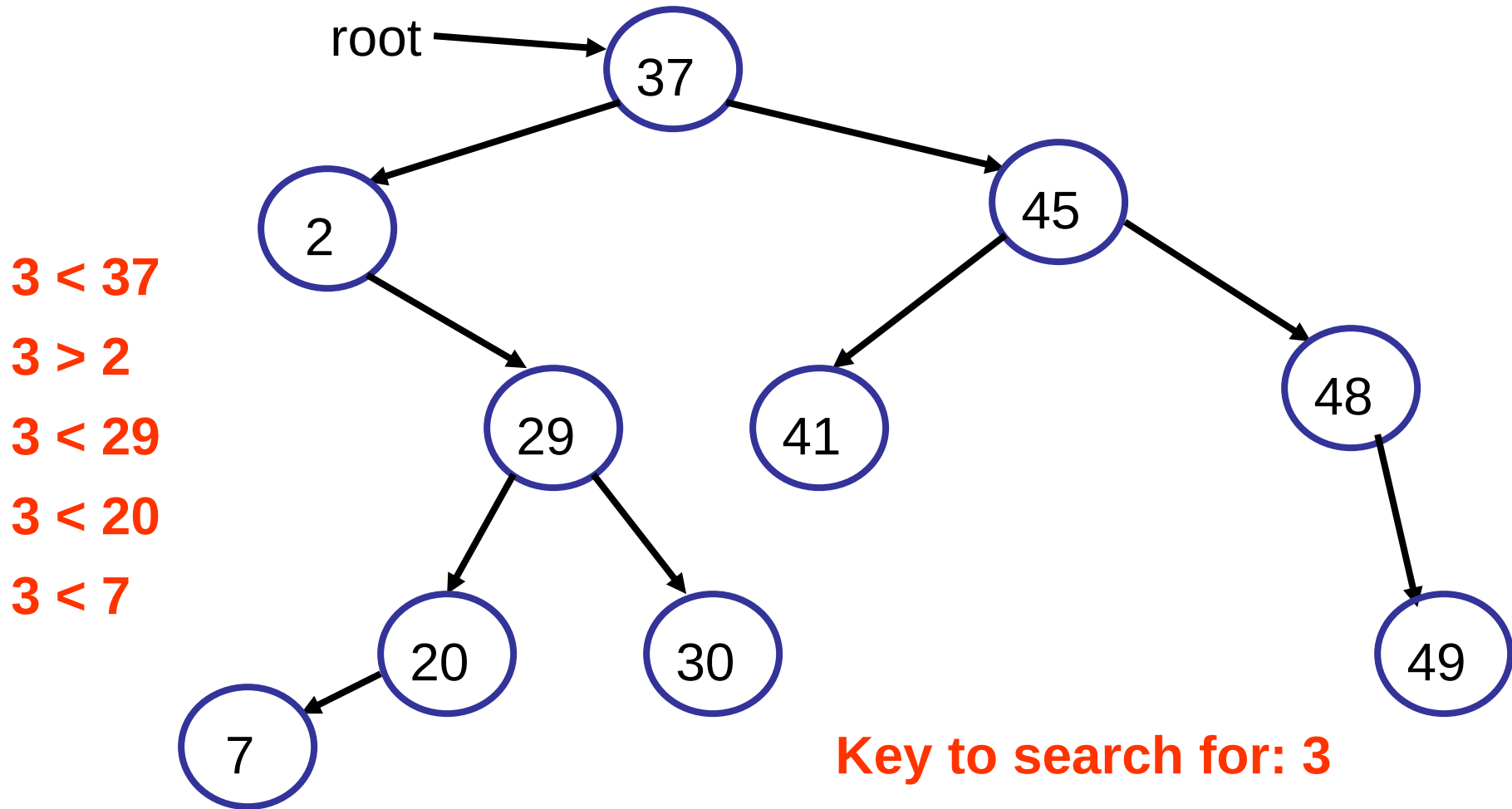
Searching for a Key in a BST (cont.)



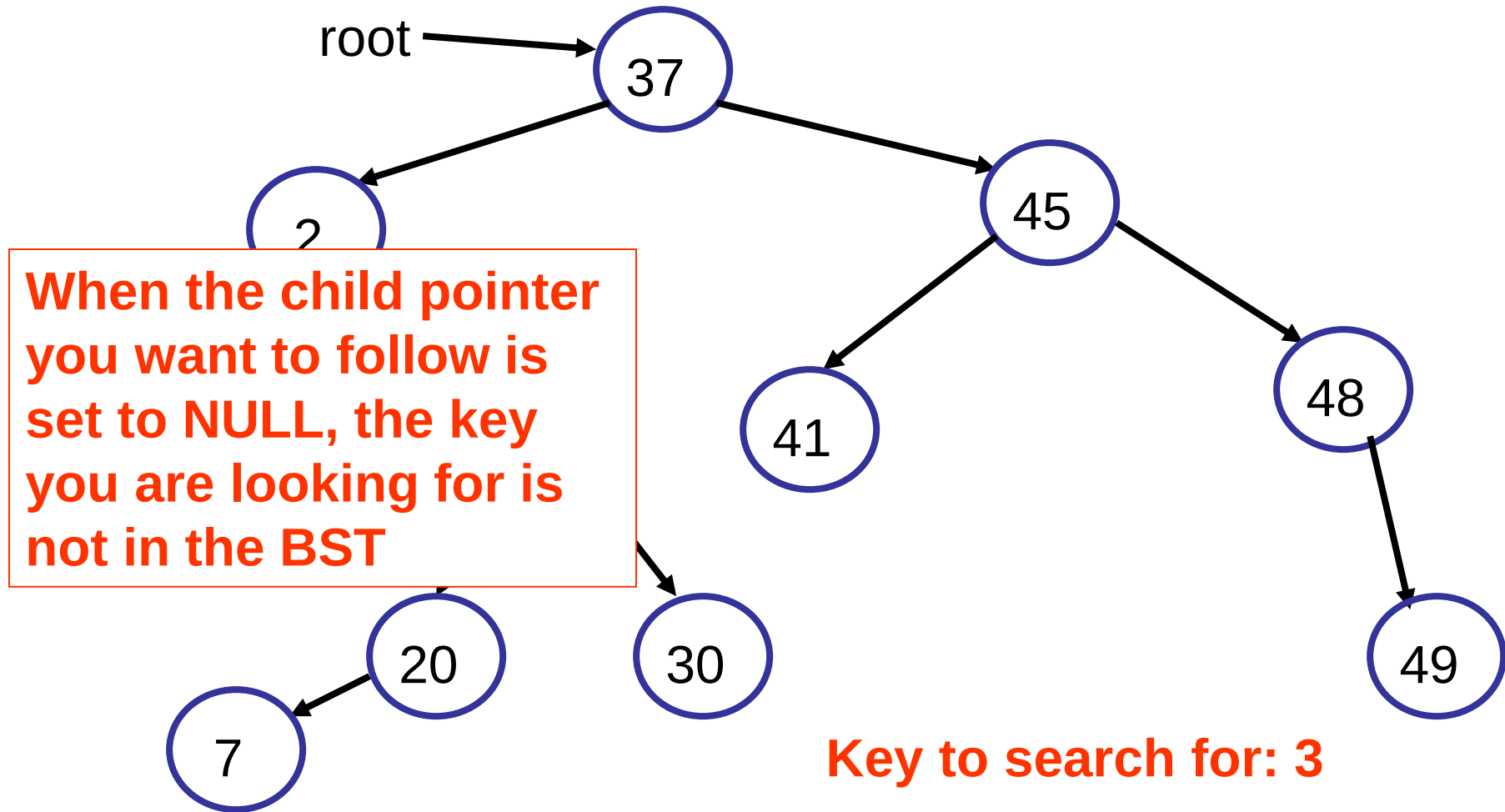
Searching for a Key in a BST (cont.)



Searching for a Key in a BST (cont.)



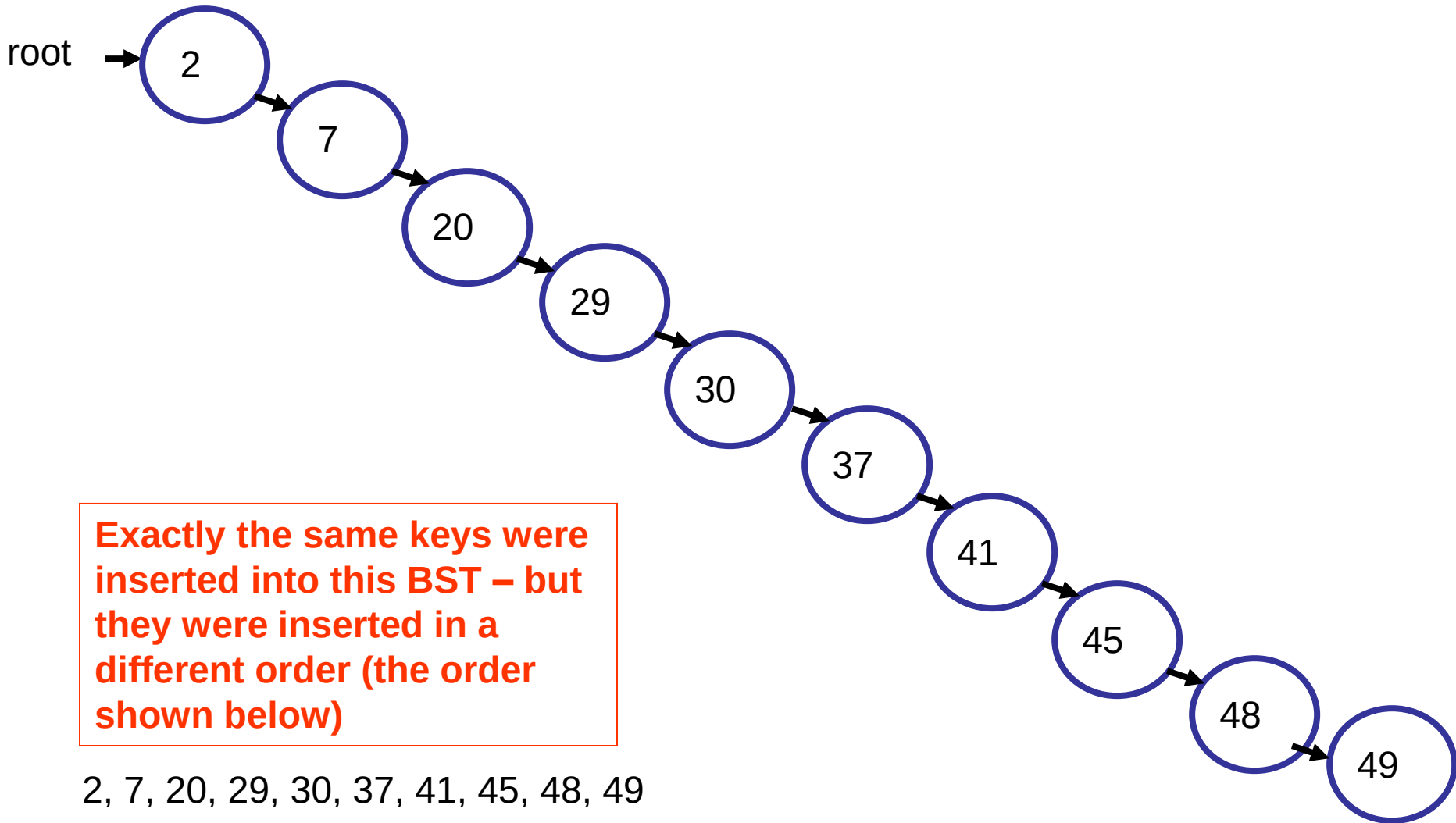
Searching for a Key in a BST (cont.)



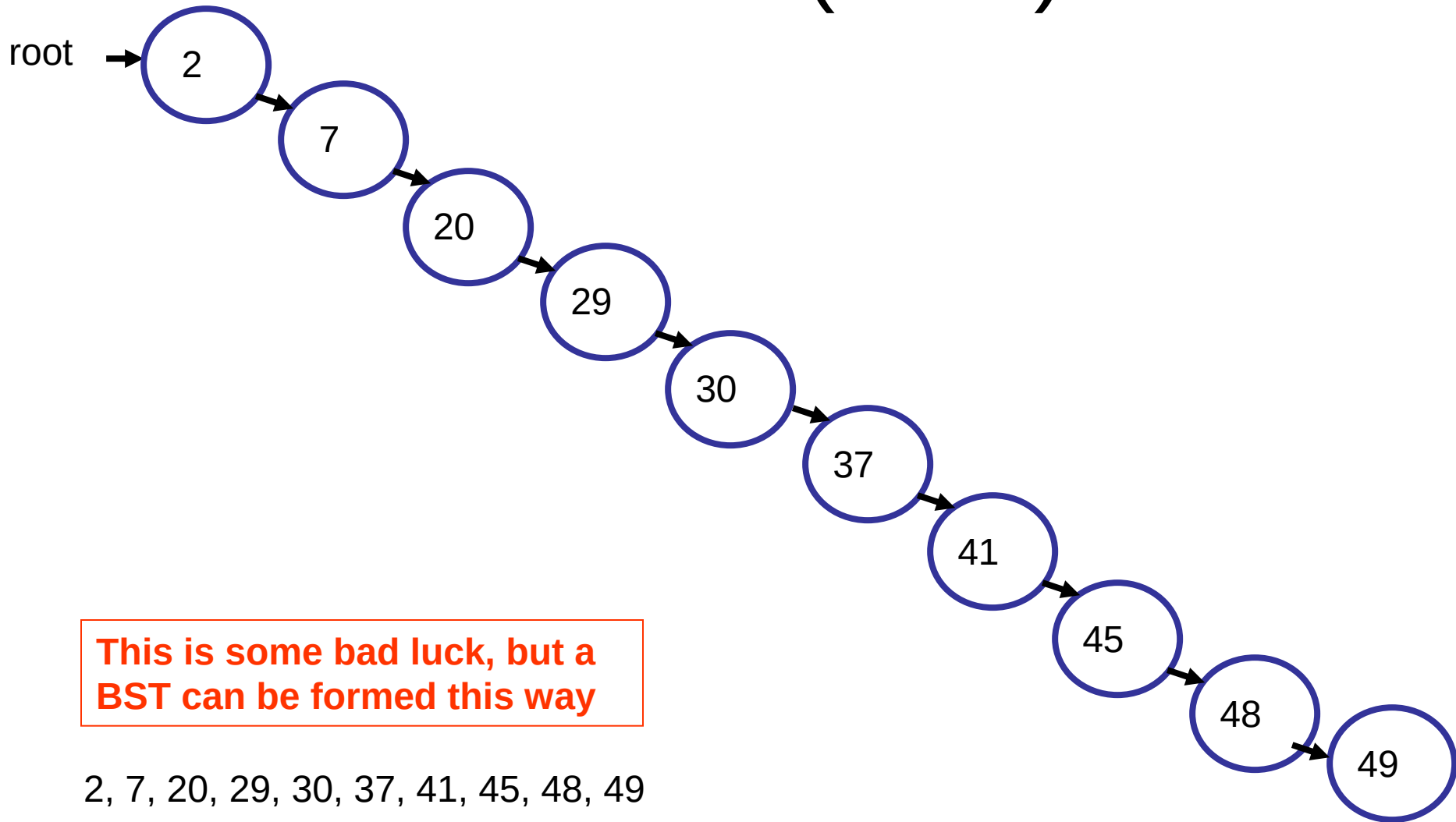
Time Complexities

- If the binary search tree happens to be a complete binary tree:
 - the time for insertion is $\Theta(\lg n)$
 - the time for searching is $O(\lg n)$
 - Searching in array is $O(n)$
- However, we could run into some bad luck...

Bad Luck



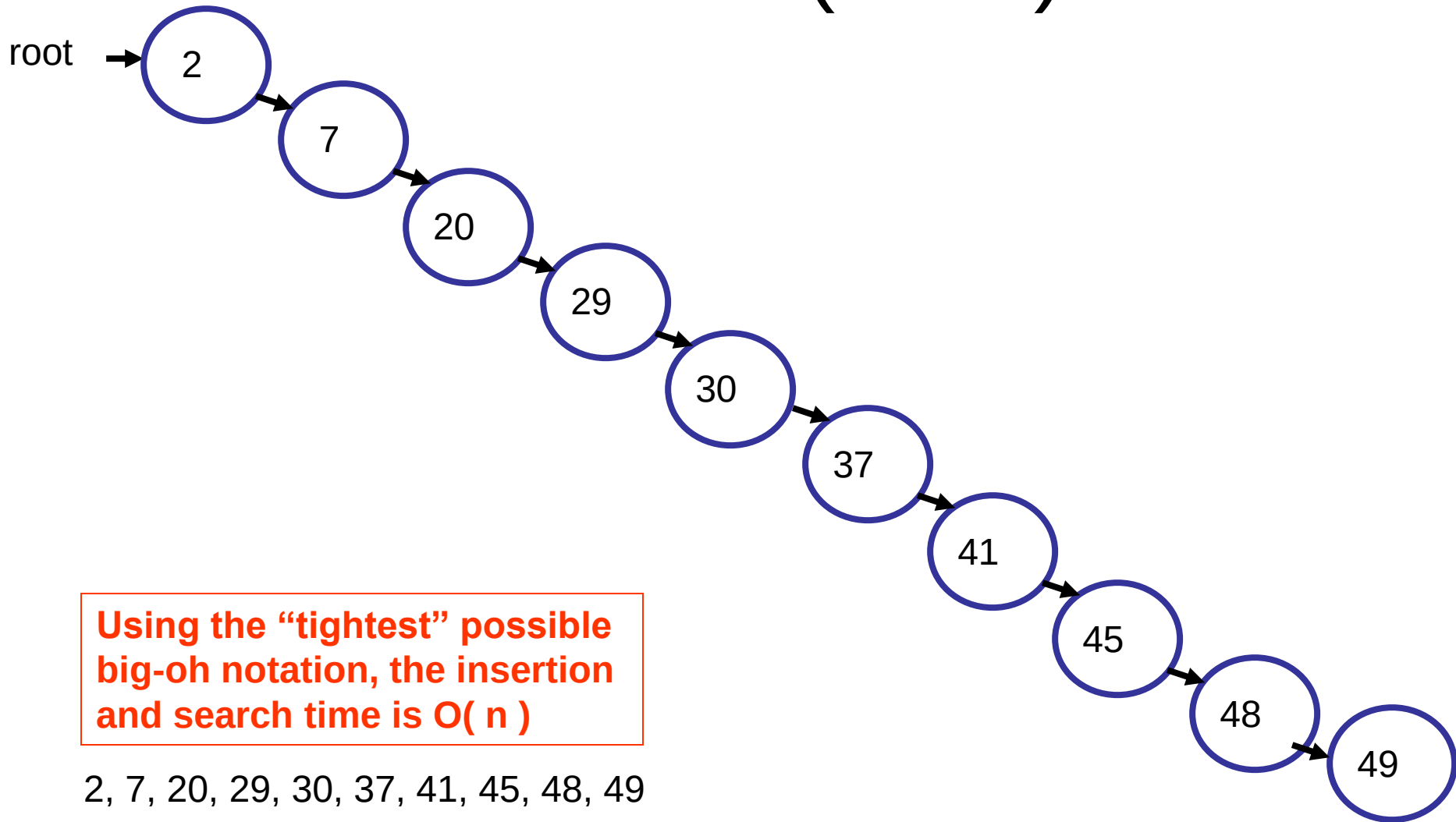
Bad Luck (cont.)



**This is some bad luck, but a
BST can be formed this way**

2, 7, 20, 29, 30, 37, 41, 45, 48, 49

Bad Luck (cont.)



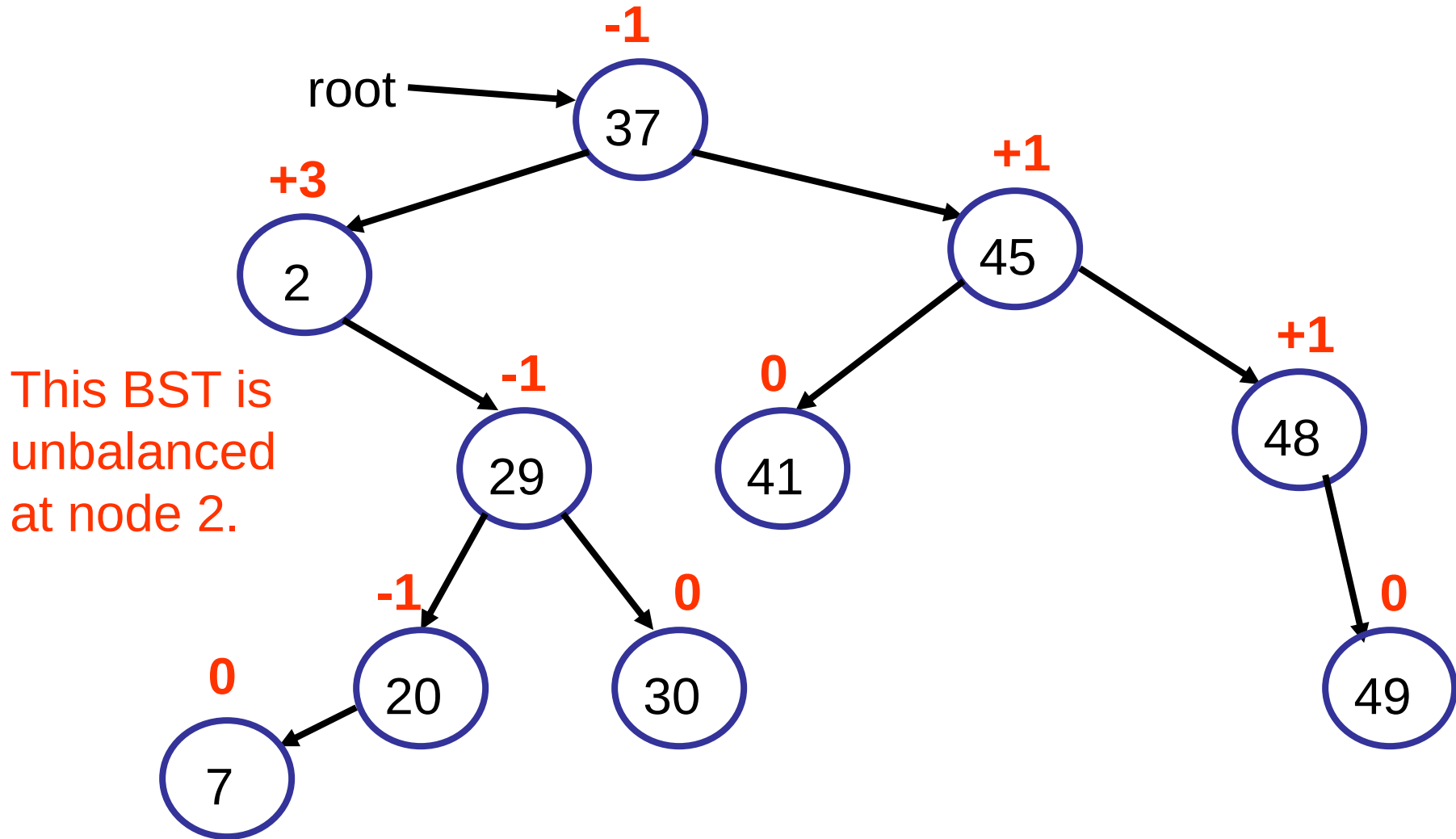
Balance Factor of BST

- In a BST, the ***balance factor*** of a node N is defined as the height of right subtree at N minus the height of left subtree at N .

$$\text{BalanceFactor}(N) = \text{Height}(\text{RightSubtree}(N)) - \text{Height}(\text{LeftSubtree}(N))$$

- If the balance factor of all nodes in a BST is -1, 0, or +1 then the BST is balanced, this BST is also known as ***AVL*** tree. Otherwise, the BST is unbalanced.

Balance Factor Example



Balanced vs. Unbalanced

- **Balanced** BST (AVL)
 - takes $\Theta(\lg n)$ time for insertion, and $O(\lg n)$ time for a search.
 - A complete or full BST is a balanced BST.
- **Unbalanced** BST
 - takes $O(n)$ time for insertion and searching.

Re-balancing a BST

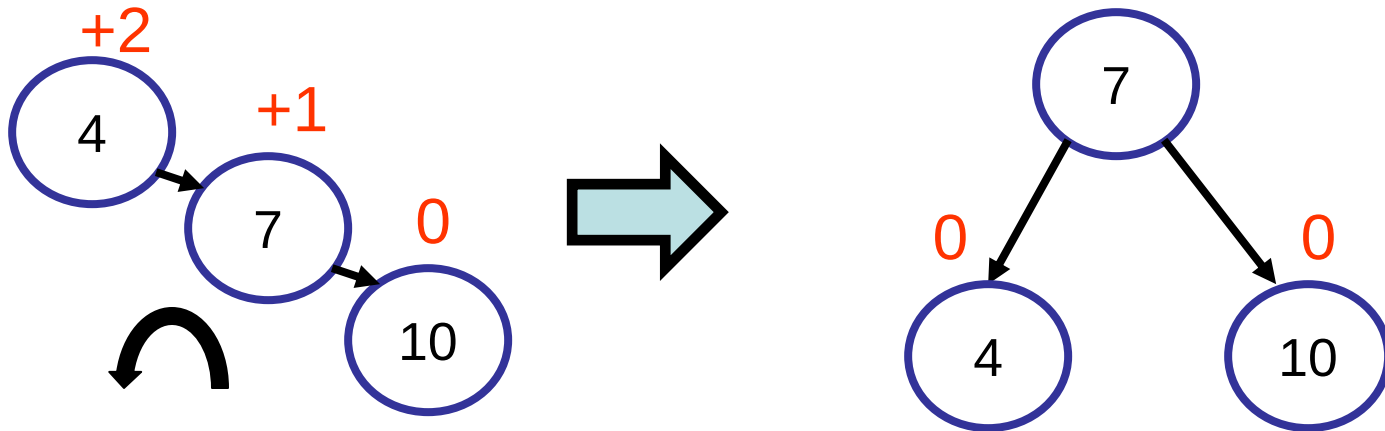
- A balanced BST is generally preferred over an unbalanced one as the former performs significantly faster than the latter.
- A BST can become unbalanced during the insertion/deletion of a node. When the BST becomes unbalanced, rotation is performed to make the BST balance again.

Rotating a BST

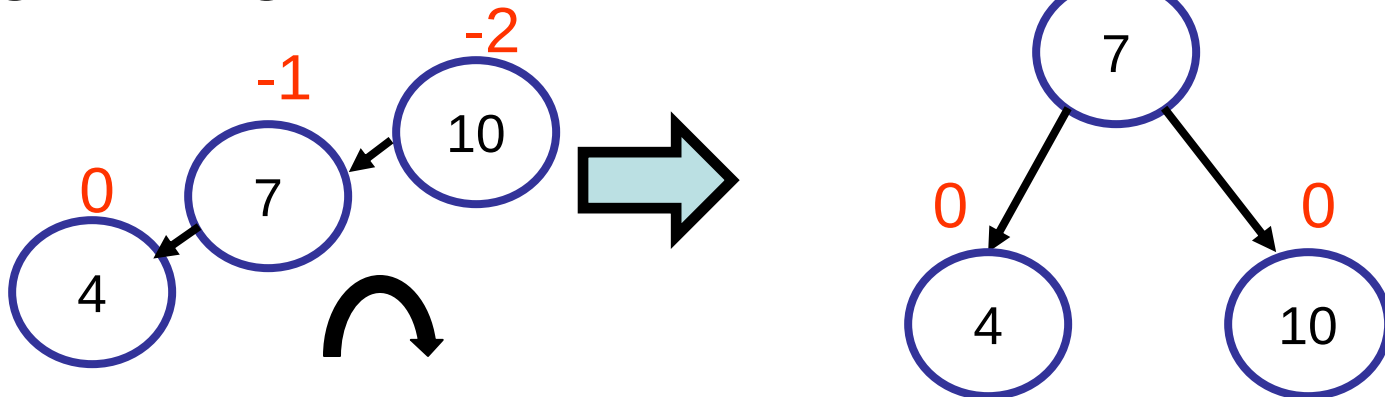
- There are 2 types of rotation: single rotation, or double rotation.
- A rotation can be left or right depends on the direction of rotation.
- Hence there are 4 possible rotations:
 1. Left single rotation
 2. Right single rotation
 3. Left right double rotation
 4. Right left double rotation

Single Rotation

- Left single rotation

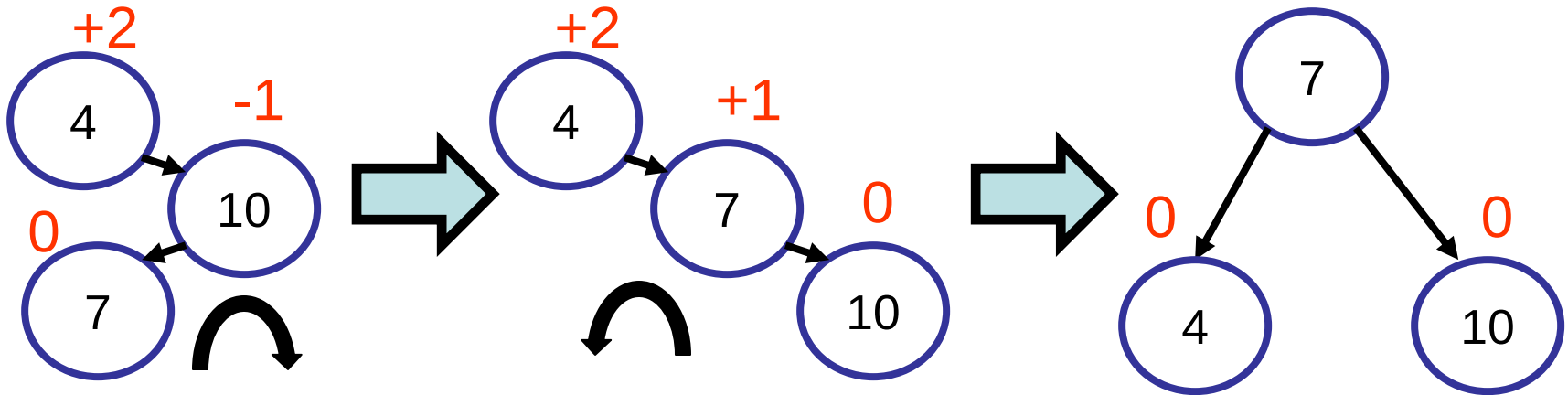


- Right single rotation

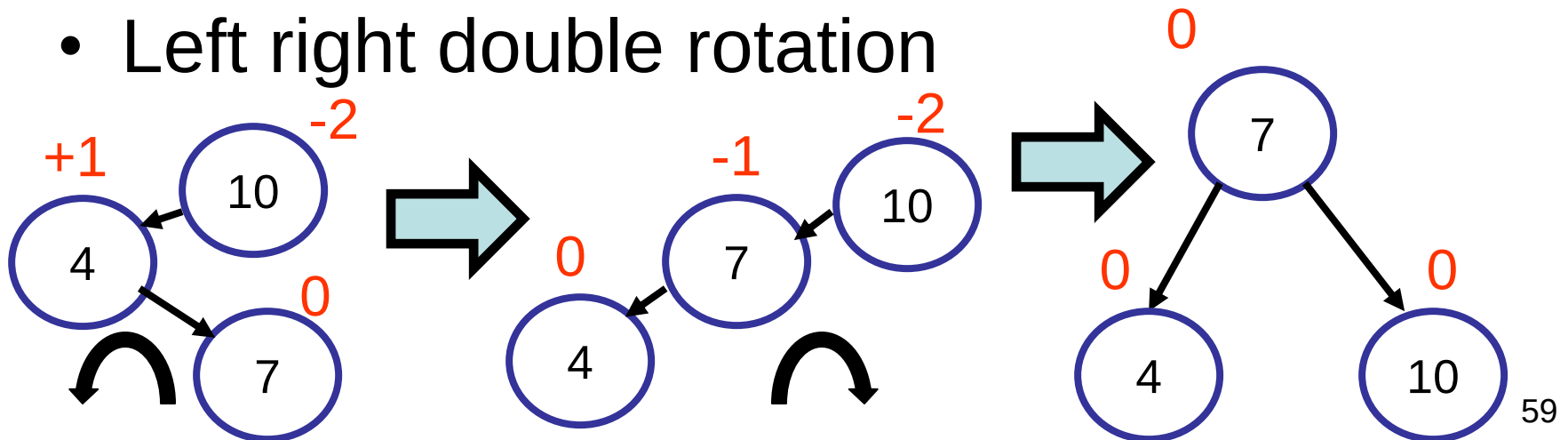


Double Rotation

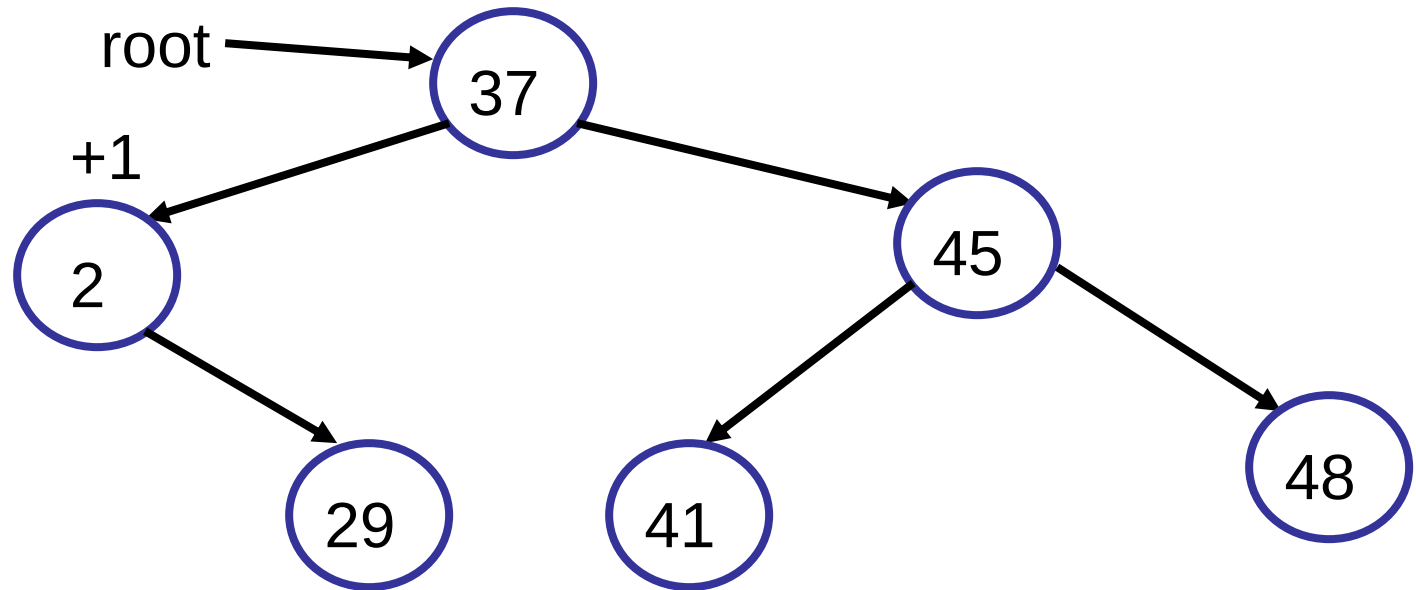
- Right left double rotation



- Left right double rotation



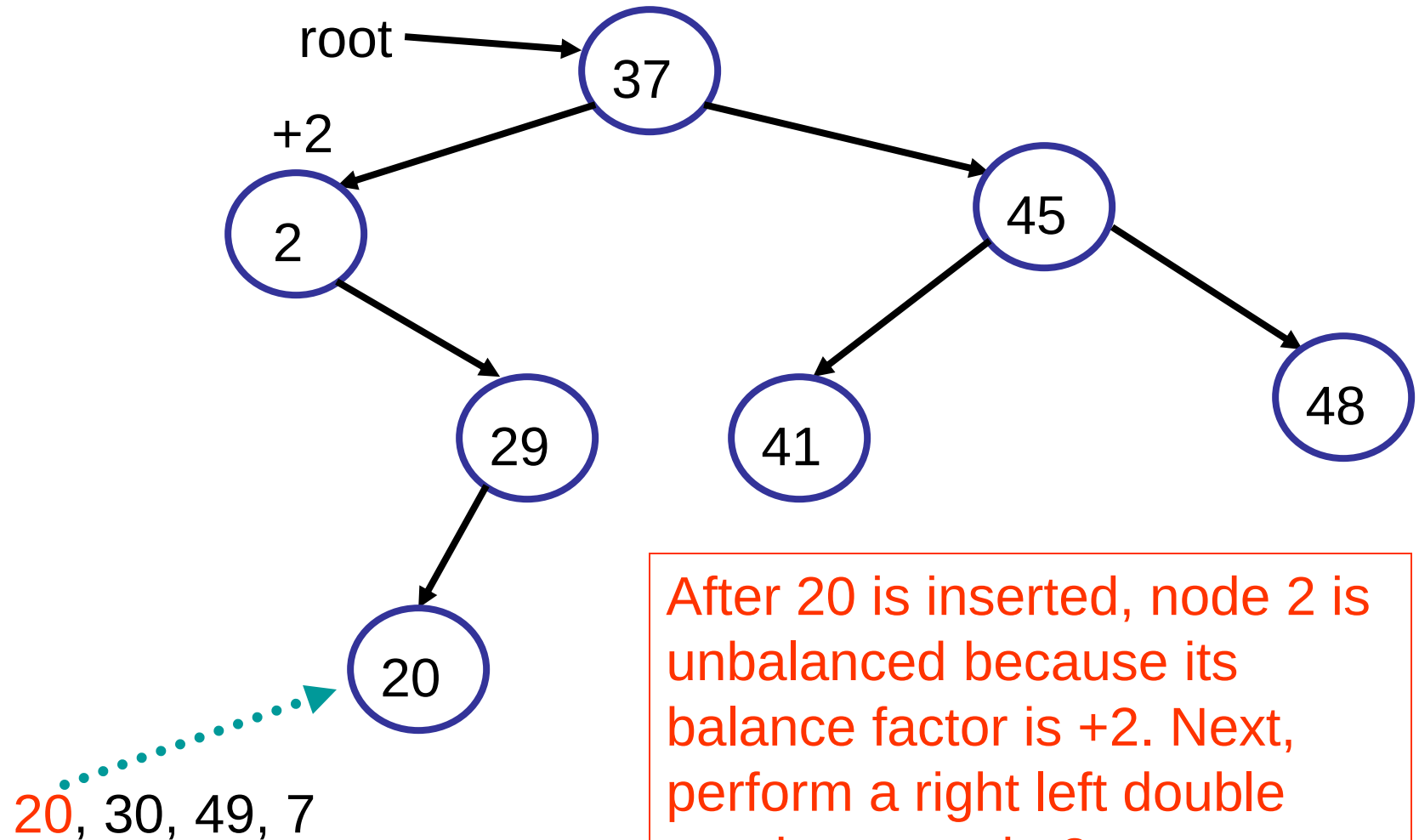
Inserting Nodes Into an AVL



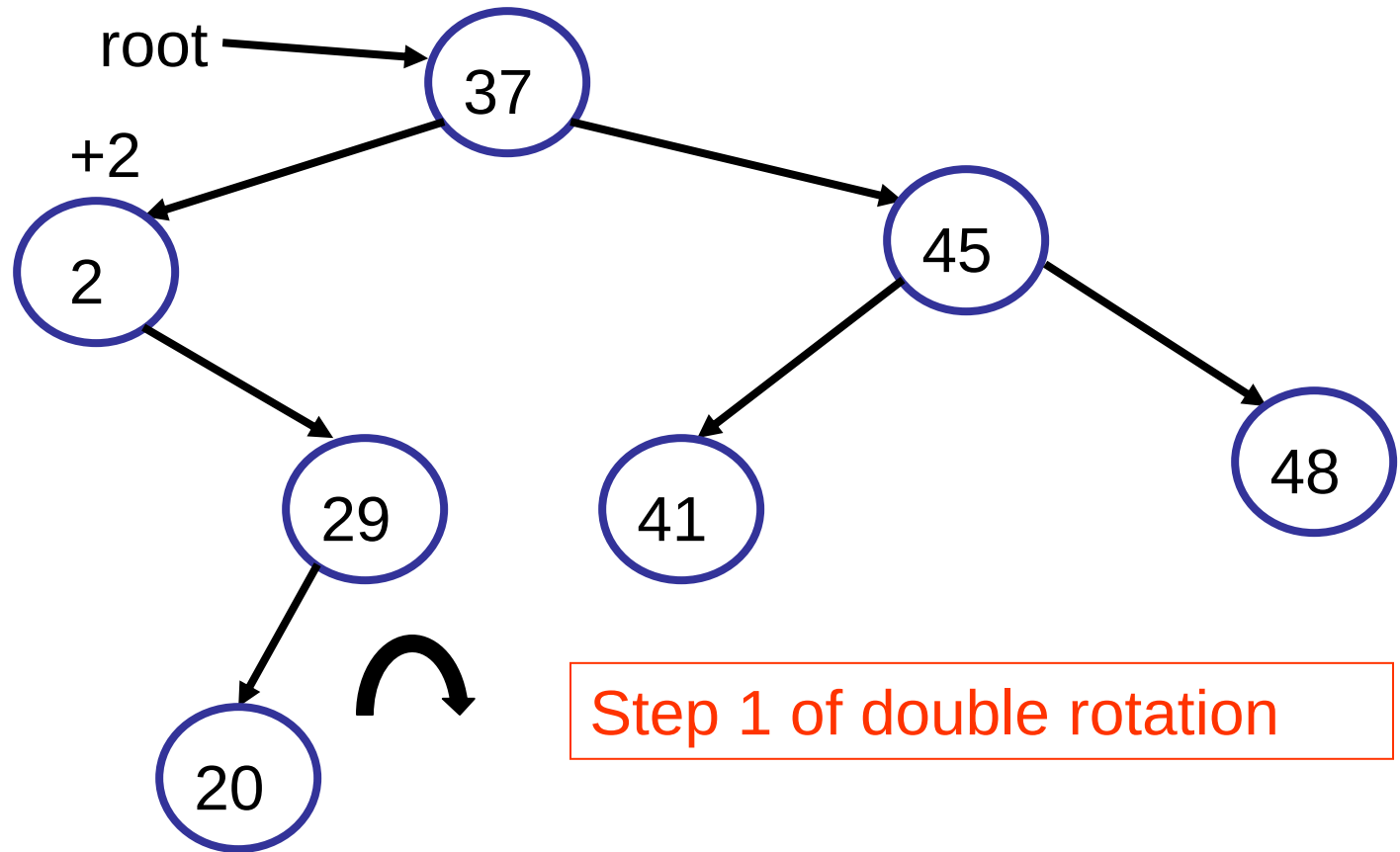
20, 30, 49, 7

Repeat the previous example of BST insertion but with an AVL tree this time.
Node 2 has balance factor of +1 now.

Inserting Nodes Into an AVL (cont.)

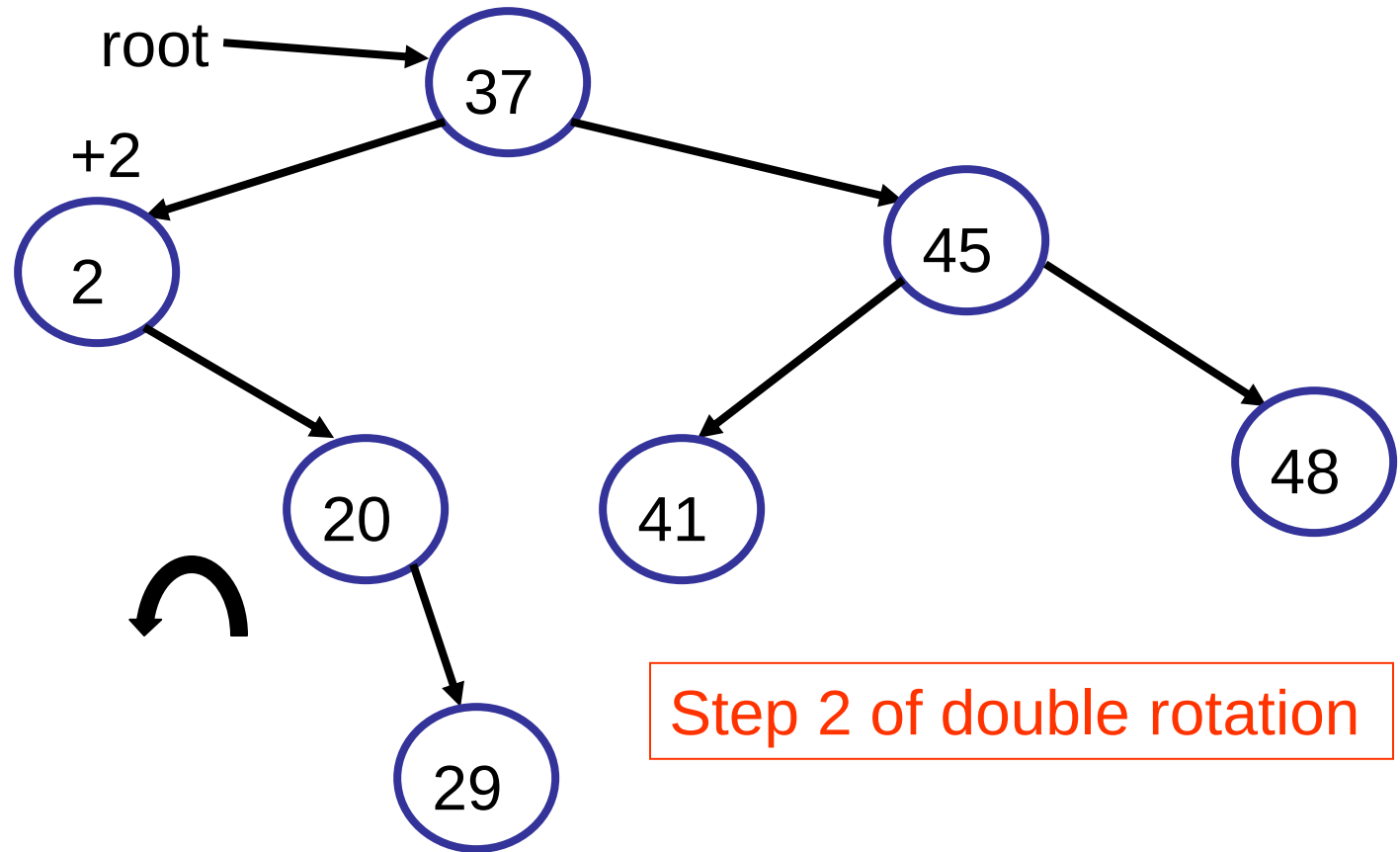


Inserting Nodes Into an AVL (cont.)



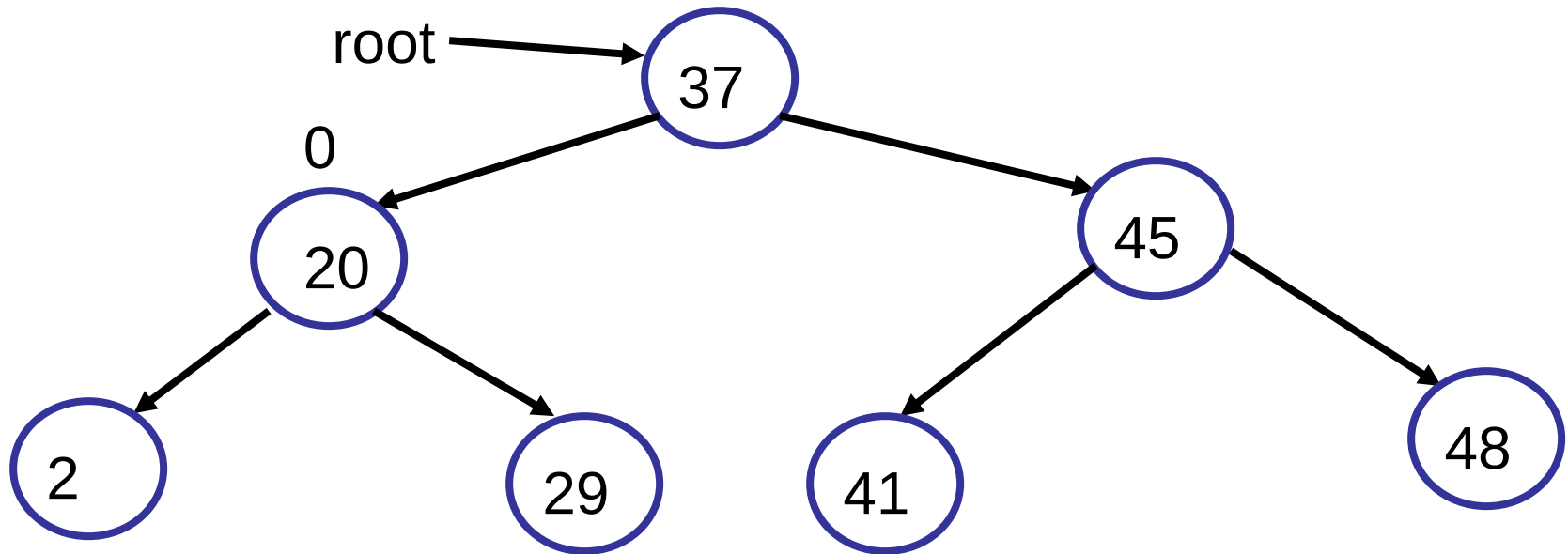
30, 49, 7

Inserting Nodes Into an AVL (cont.)



30, 49, 7

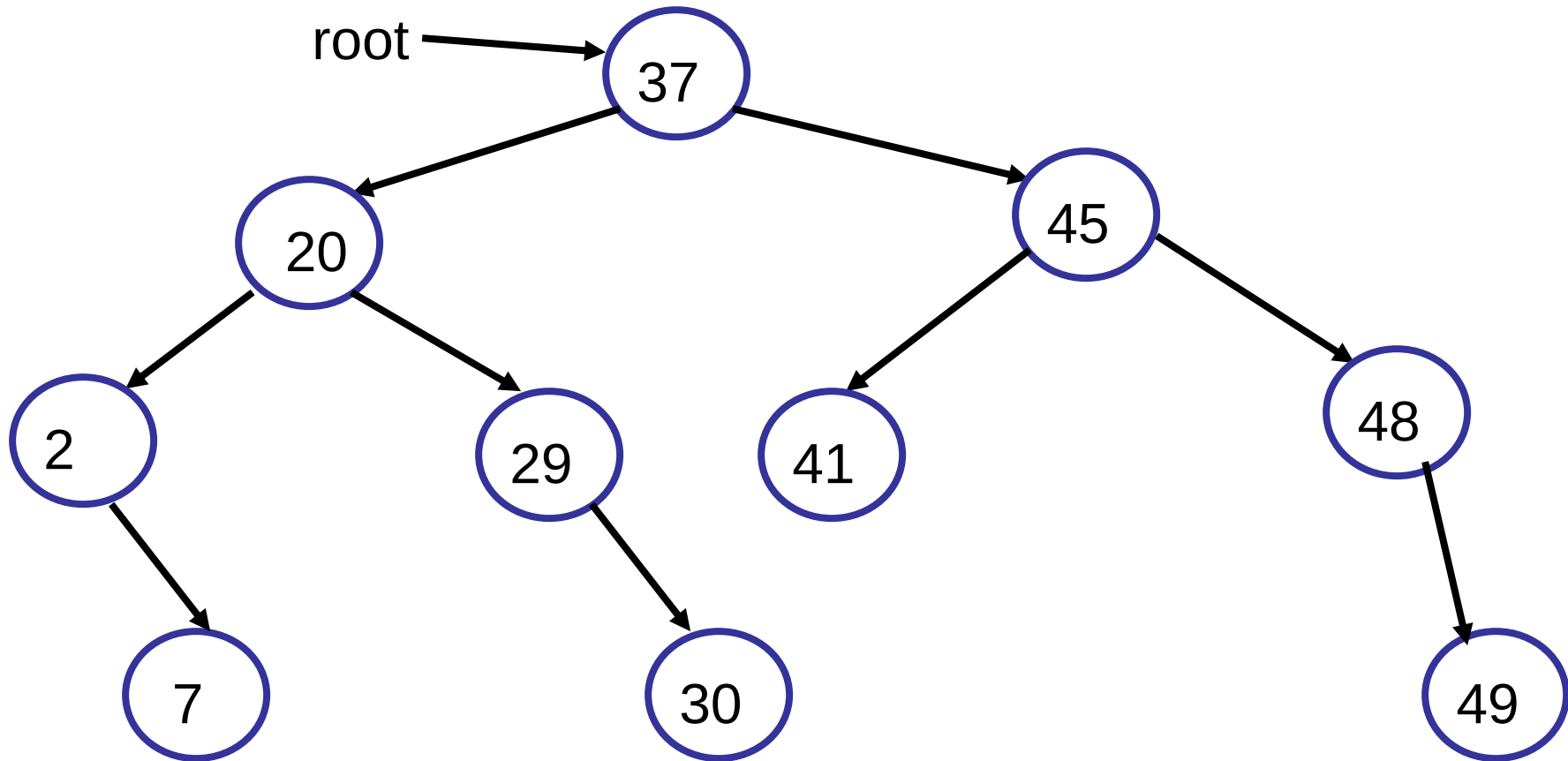
Inserting Nodes Into an AVL (cont.)



Balanced again.
Continue inserting
remaining numbers.

30, 49, 7

Inserting Nodes Into an AVL (cont.)

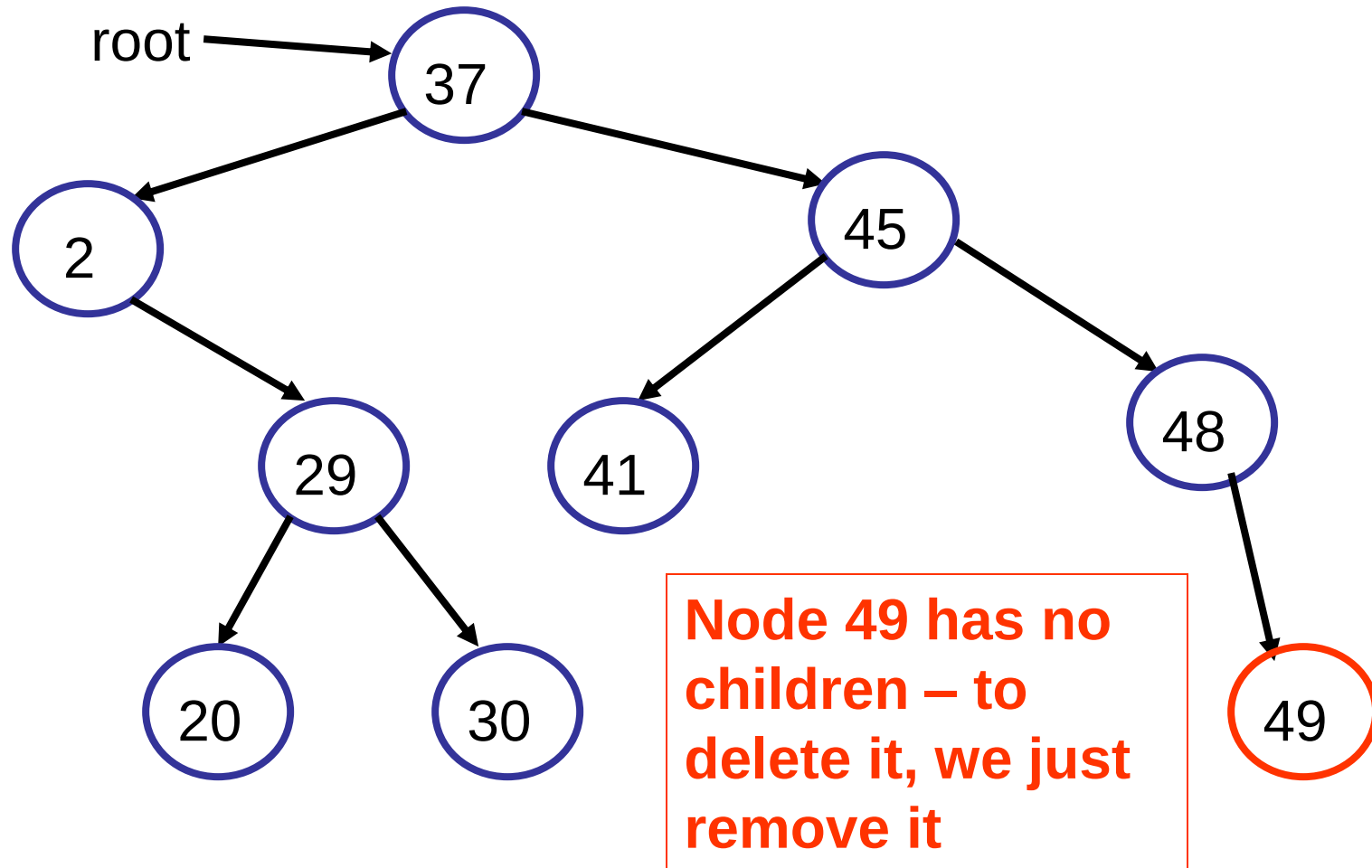


Done.

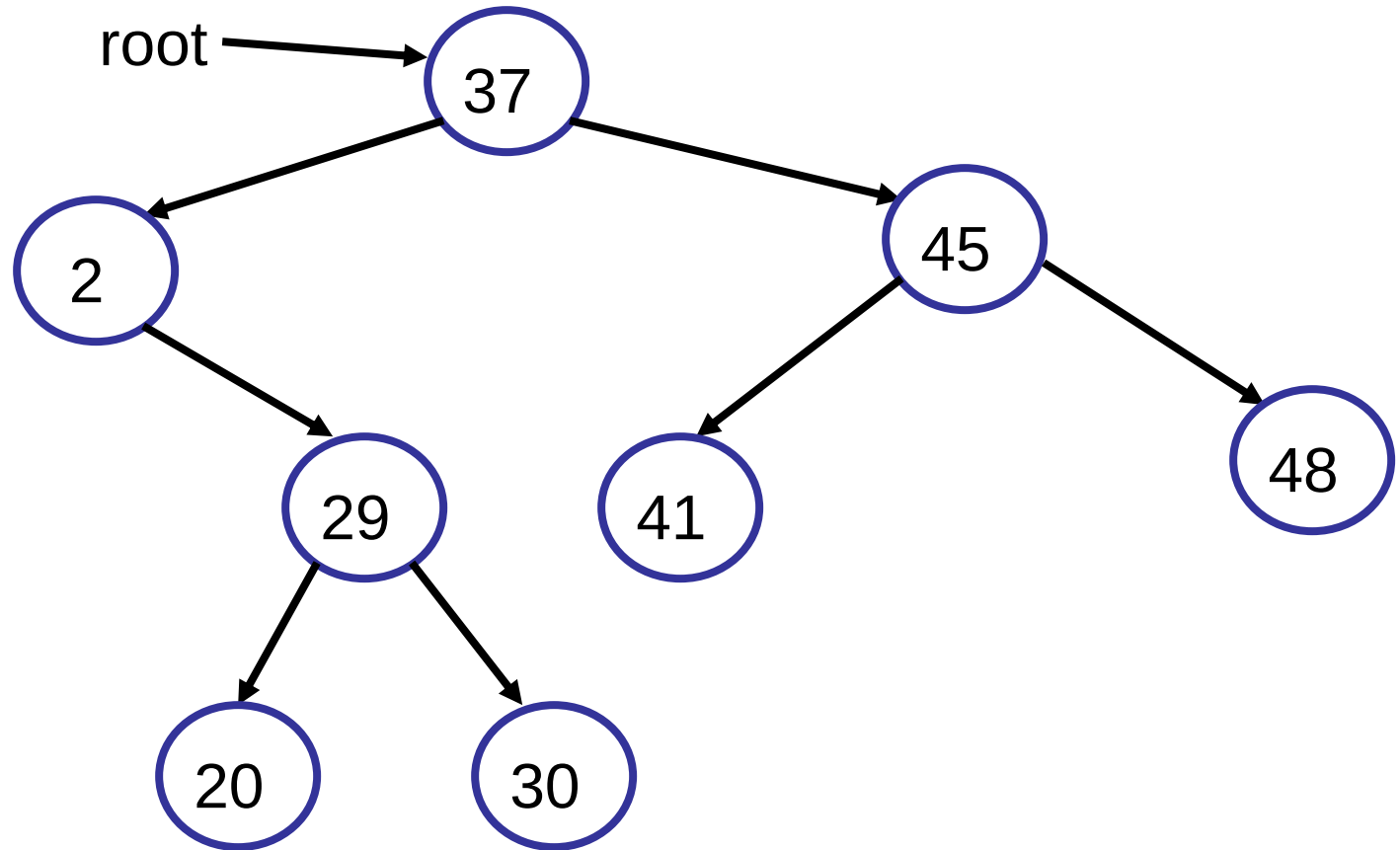
Deleting a BST Node

- Deleting a node in a BST is a little tricky – it has to be deleted so that the resulting structure is still a BST with each node greater than its left child and less than its right child.
- Deleting a node is handled differently depending on whether the node:
 - has no children
 - has one child
 - has two children

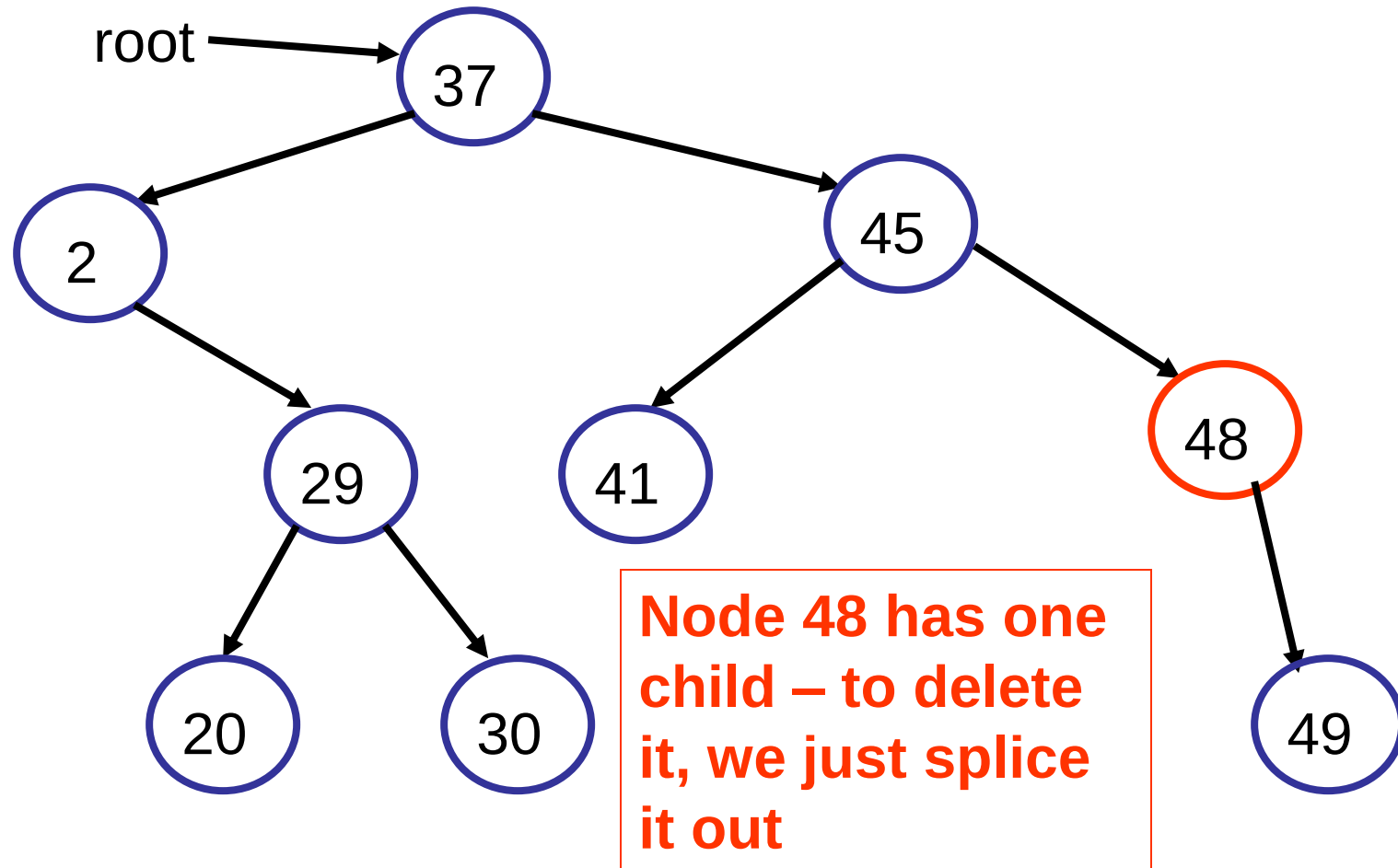
Deletion Case 1: No Children



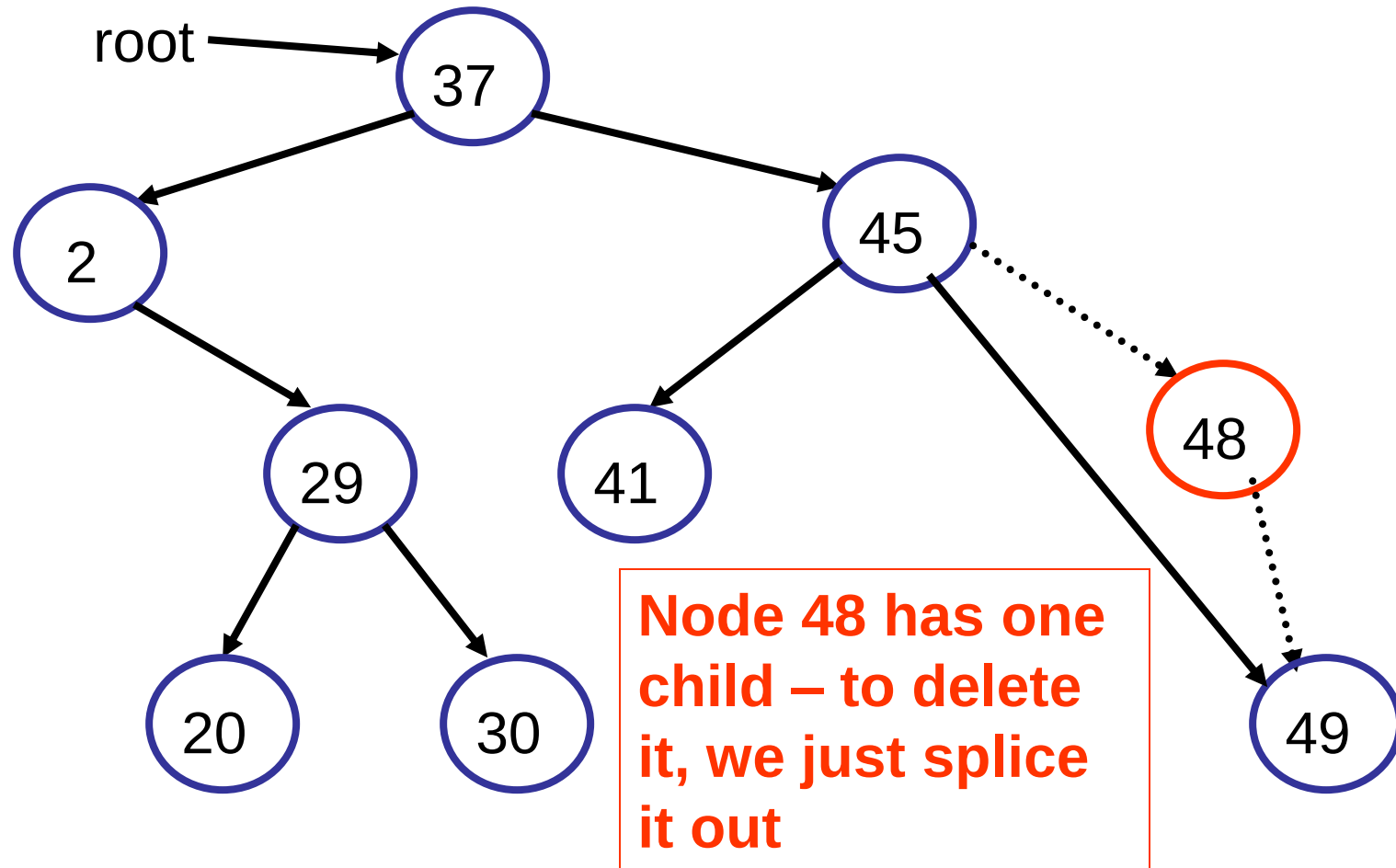
Deletion Case 1: No Children (cont.)



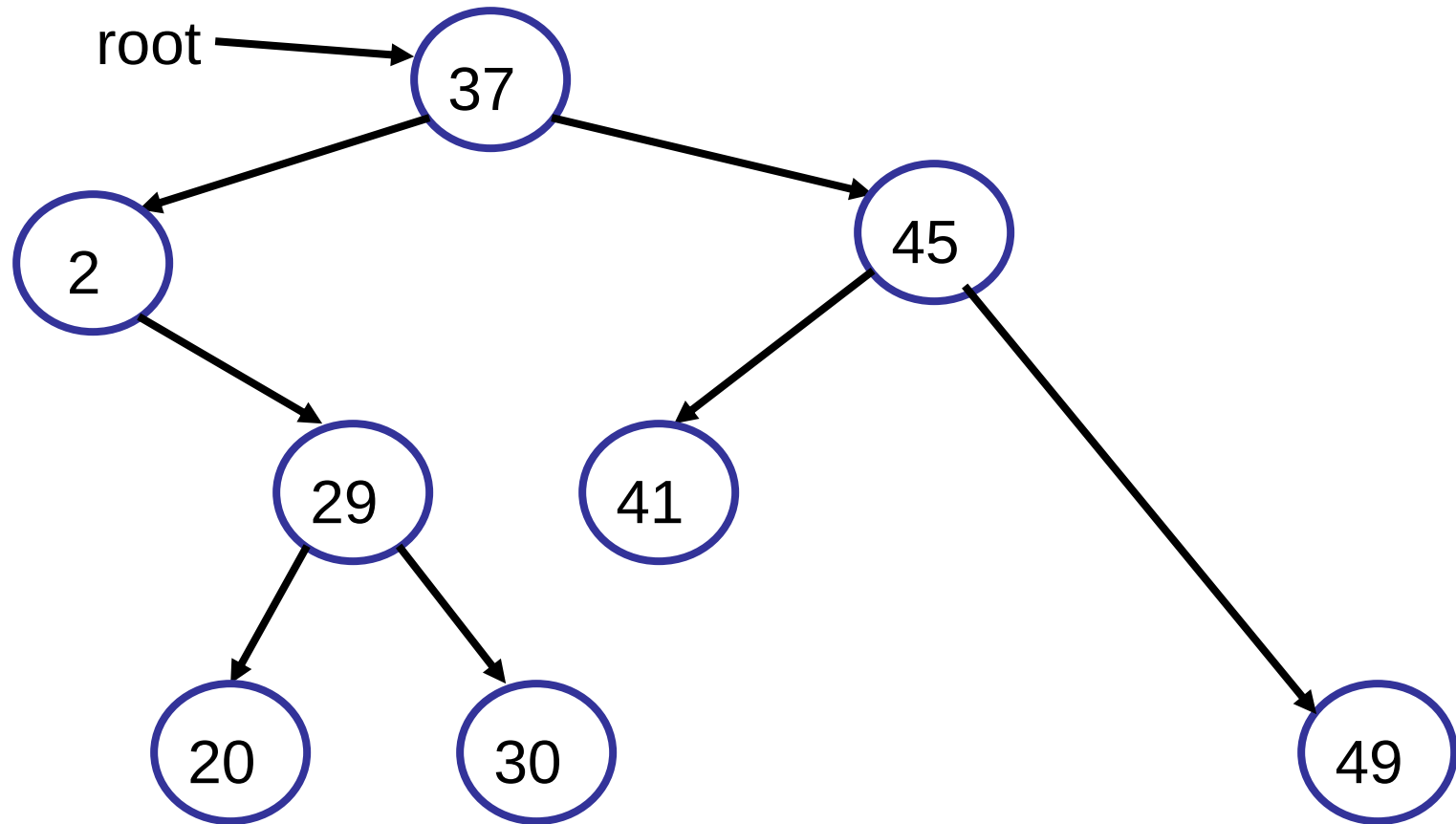
Deletion Case 2: One Child



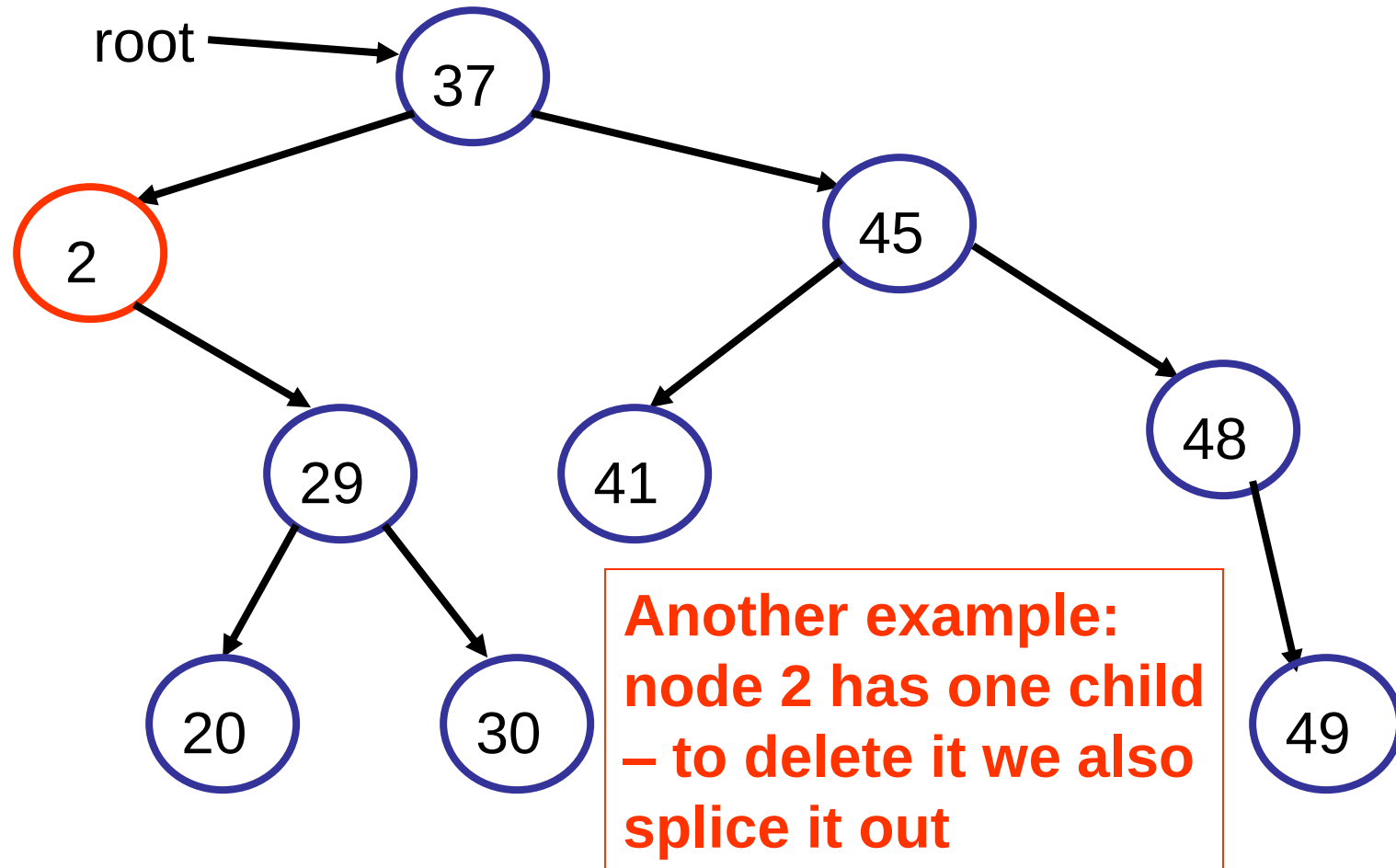
Deletion Case 2: One Child (cont.)



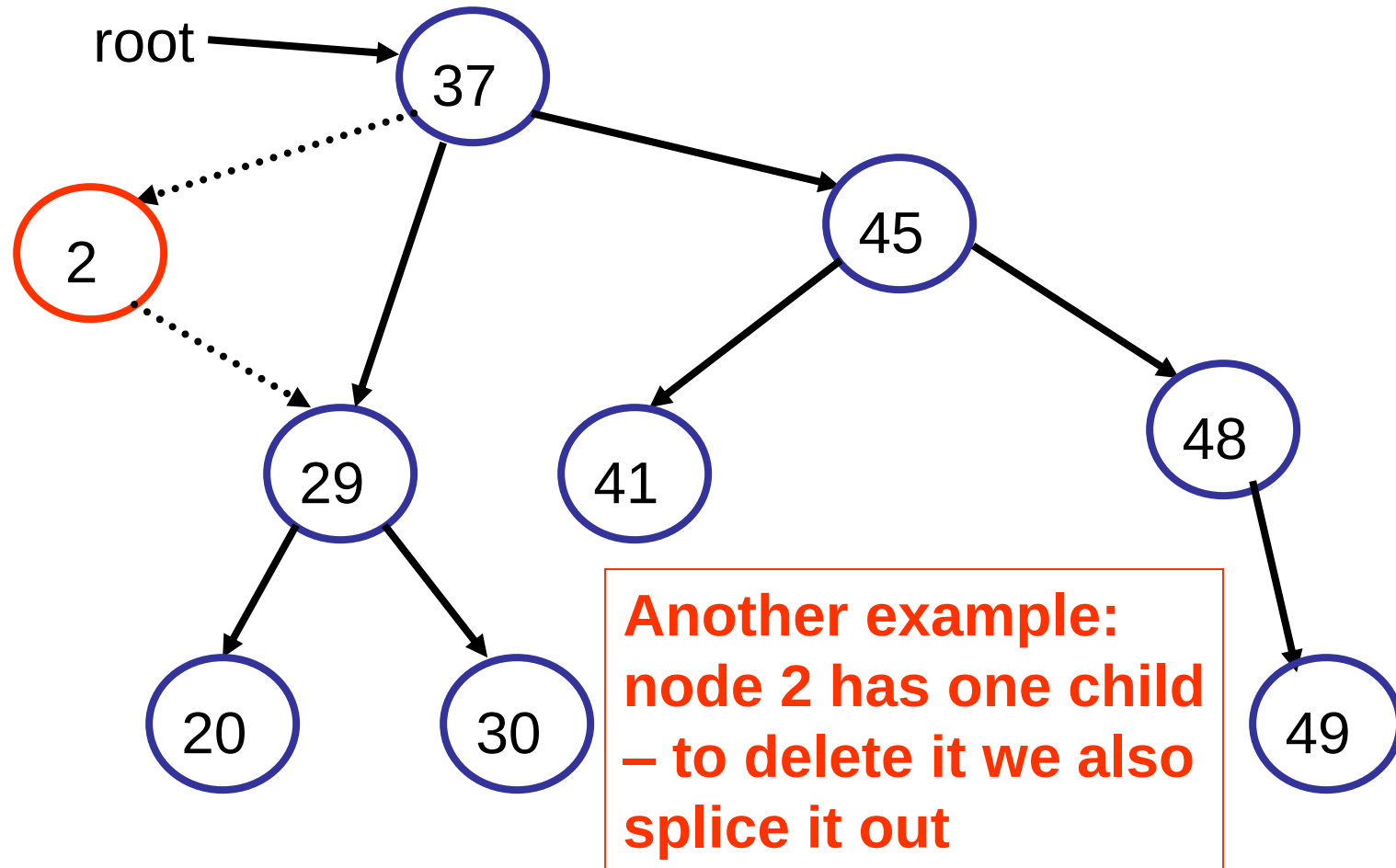
Deletion Case 2: One Child (cont.)



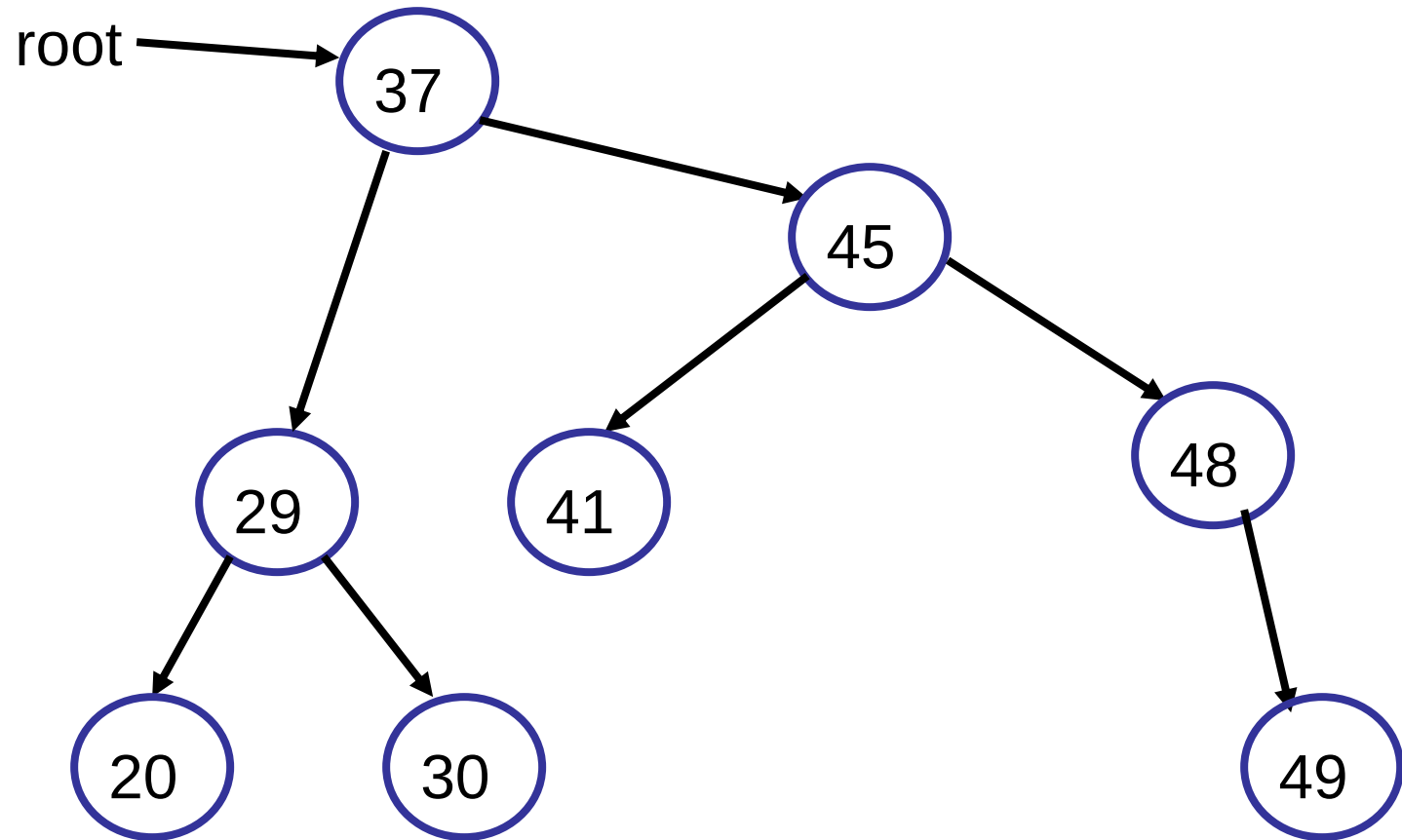
Deletion Case 2: One Child (cont.)



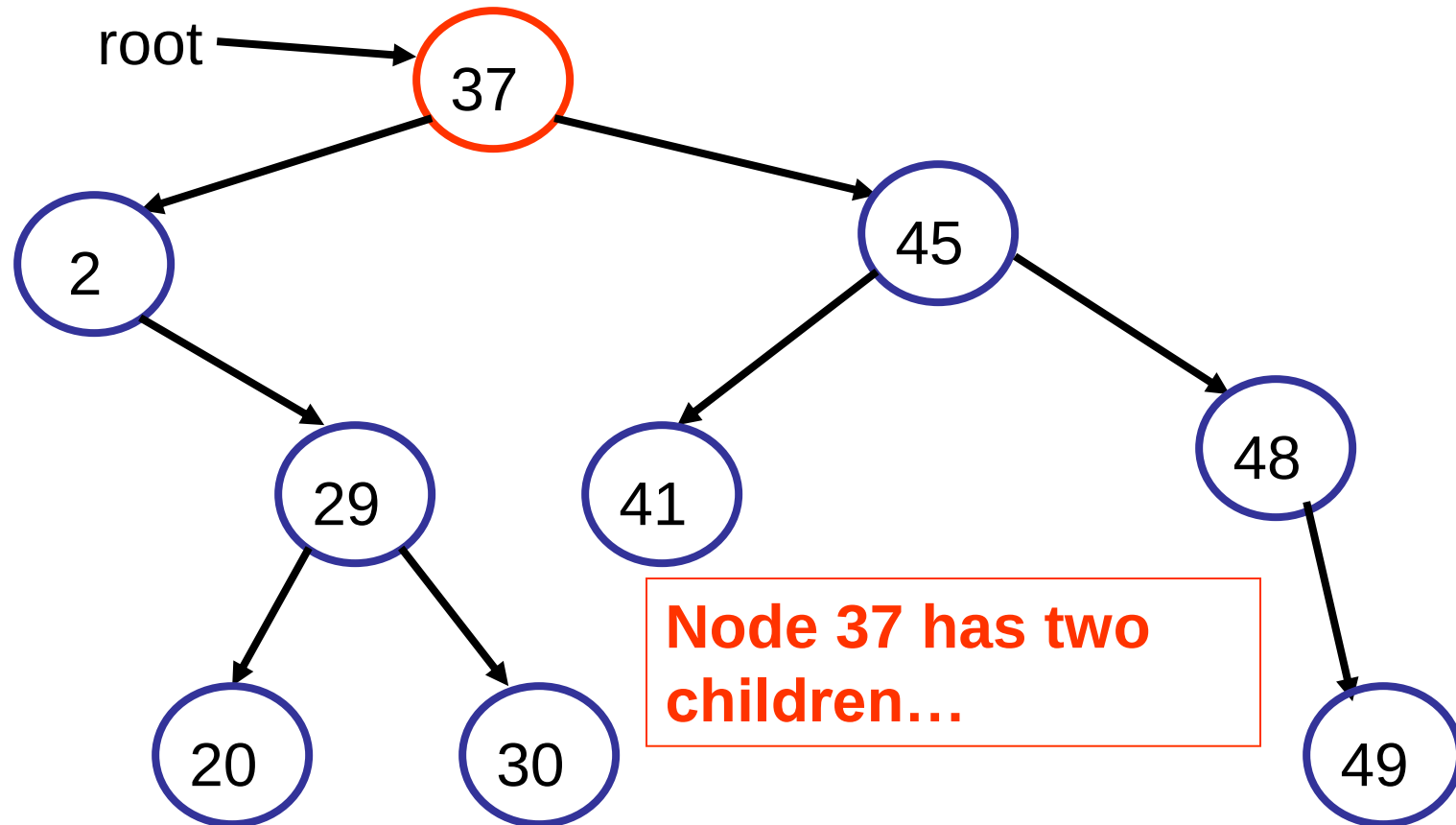
Deletion Case 2: One Child (cont.)



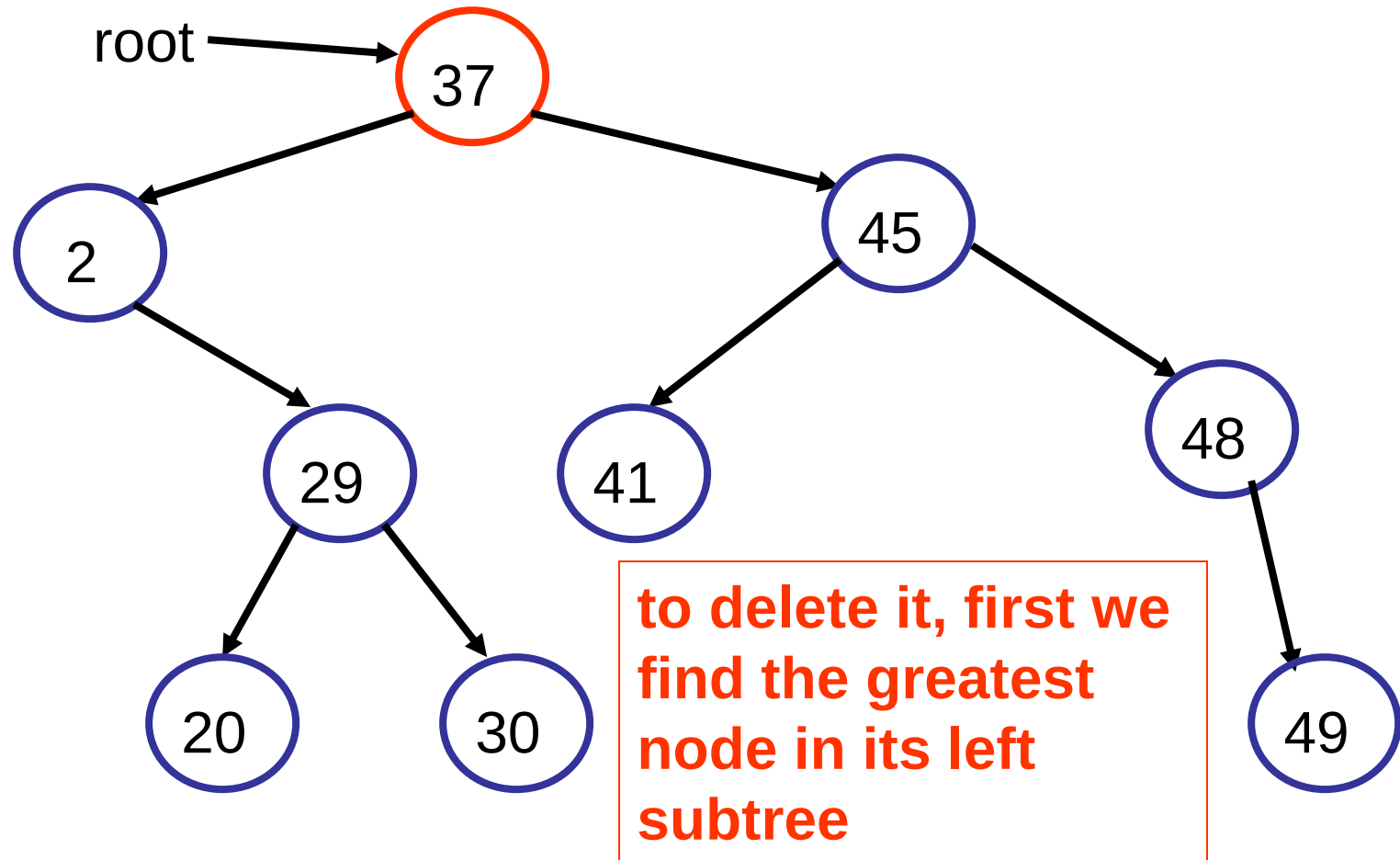
Deletion Case 2: One Child (cont.)



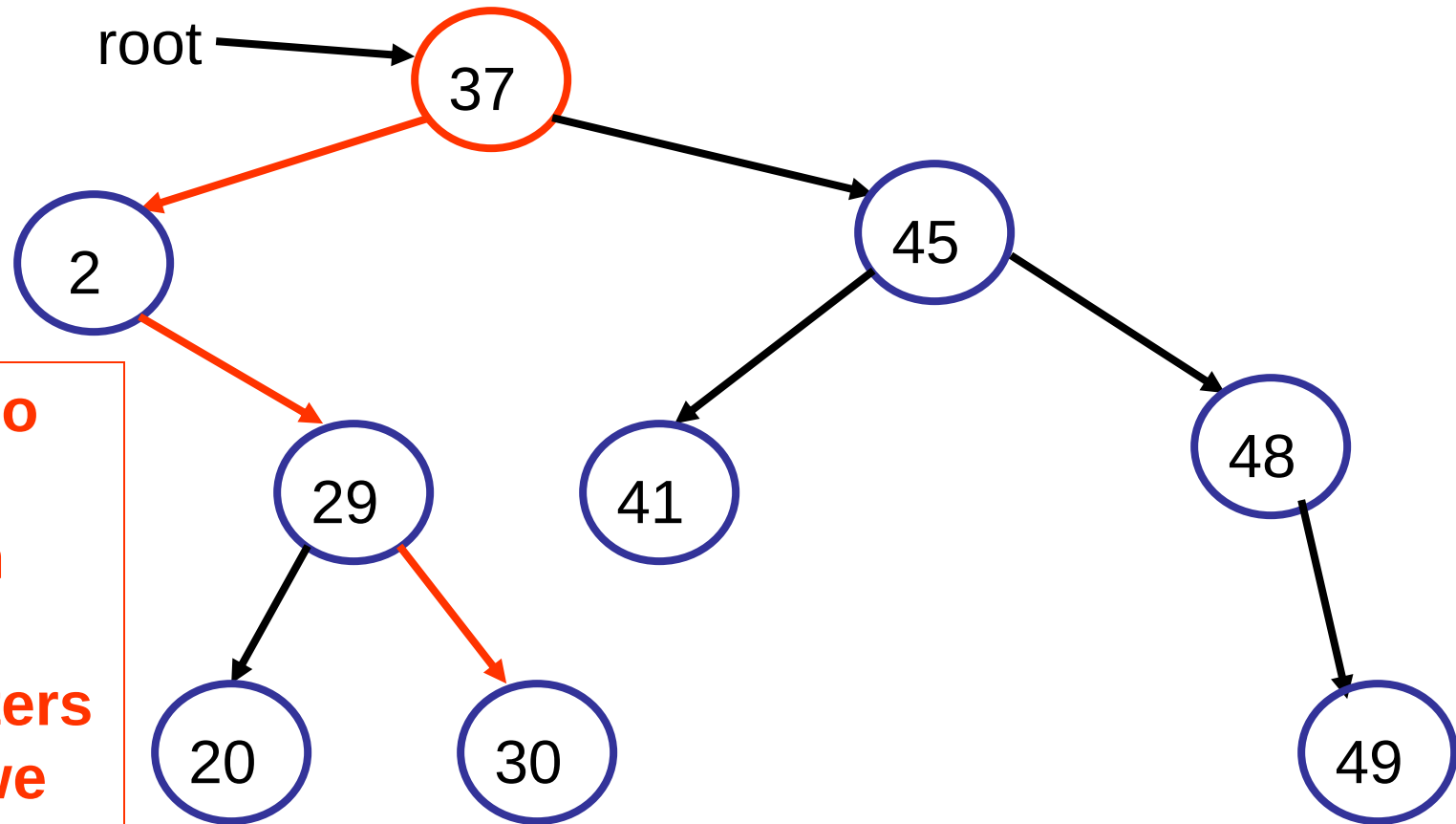
Deletion Case 3: Two Children



Deletion Case 3: Two Children (cont.)

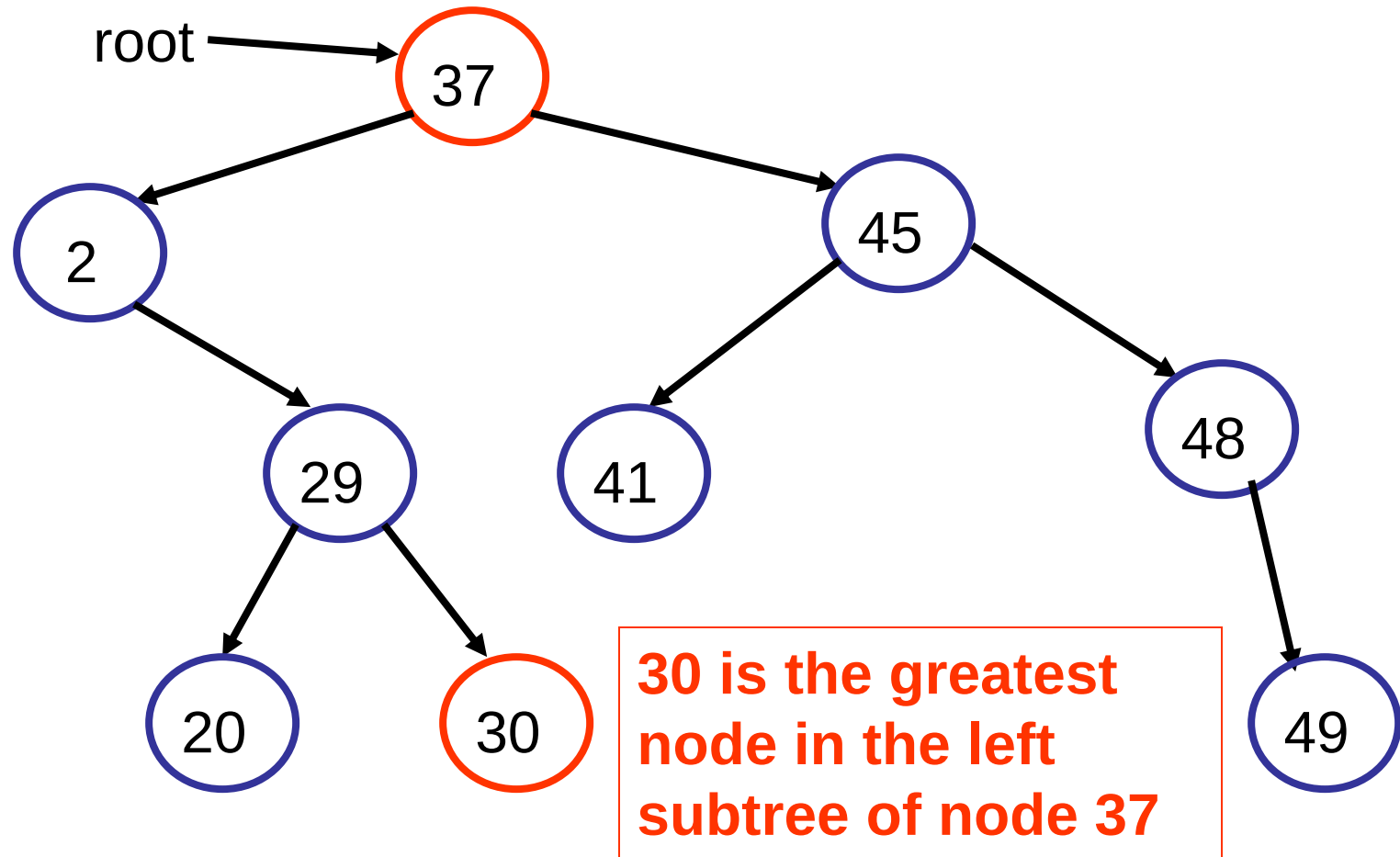


Deletion Case 3: Two Children (cont.)

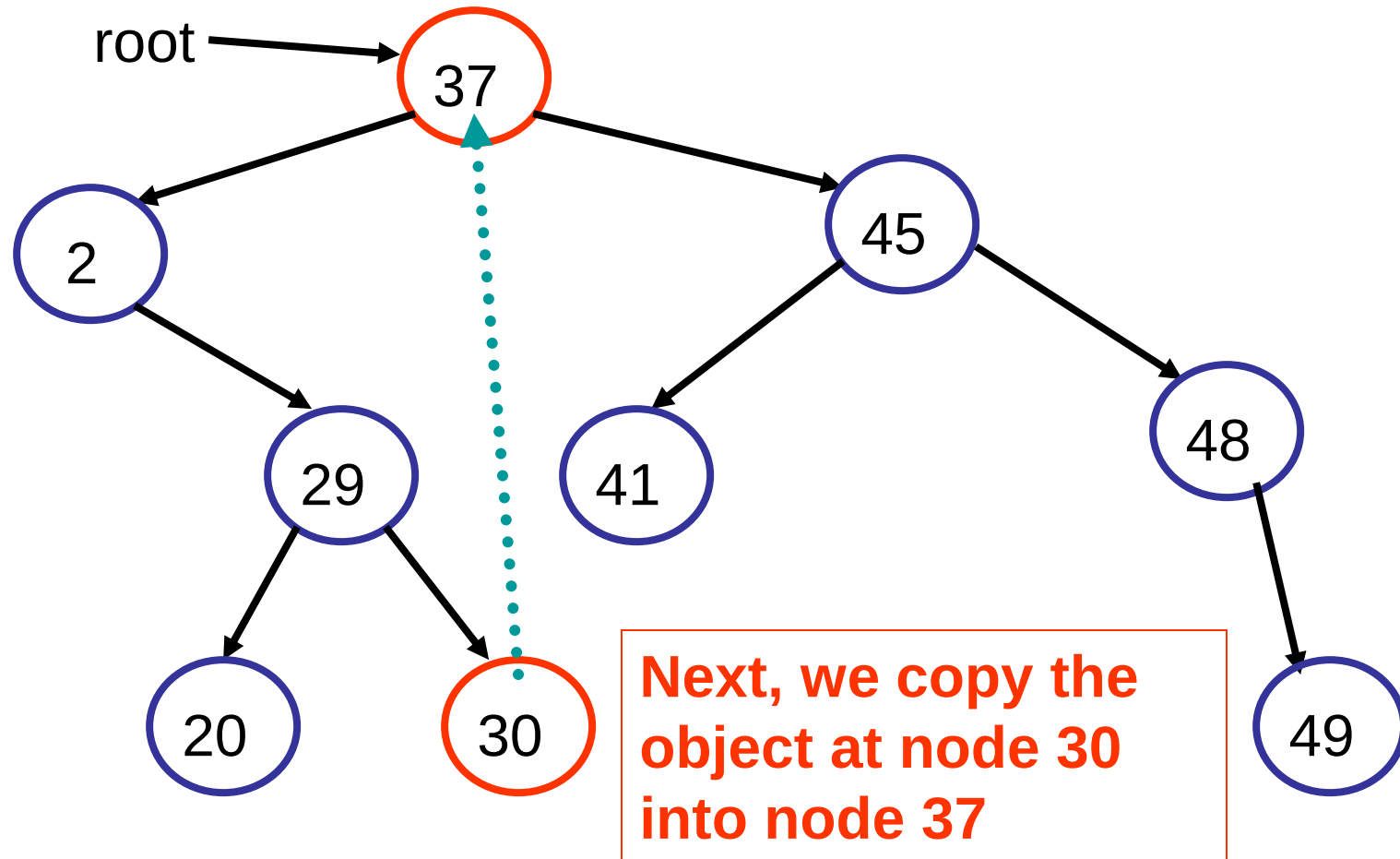


First, we go to the left once, then follow the right pointers as far as we can

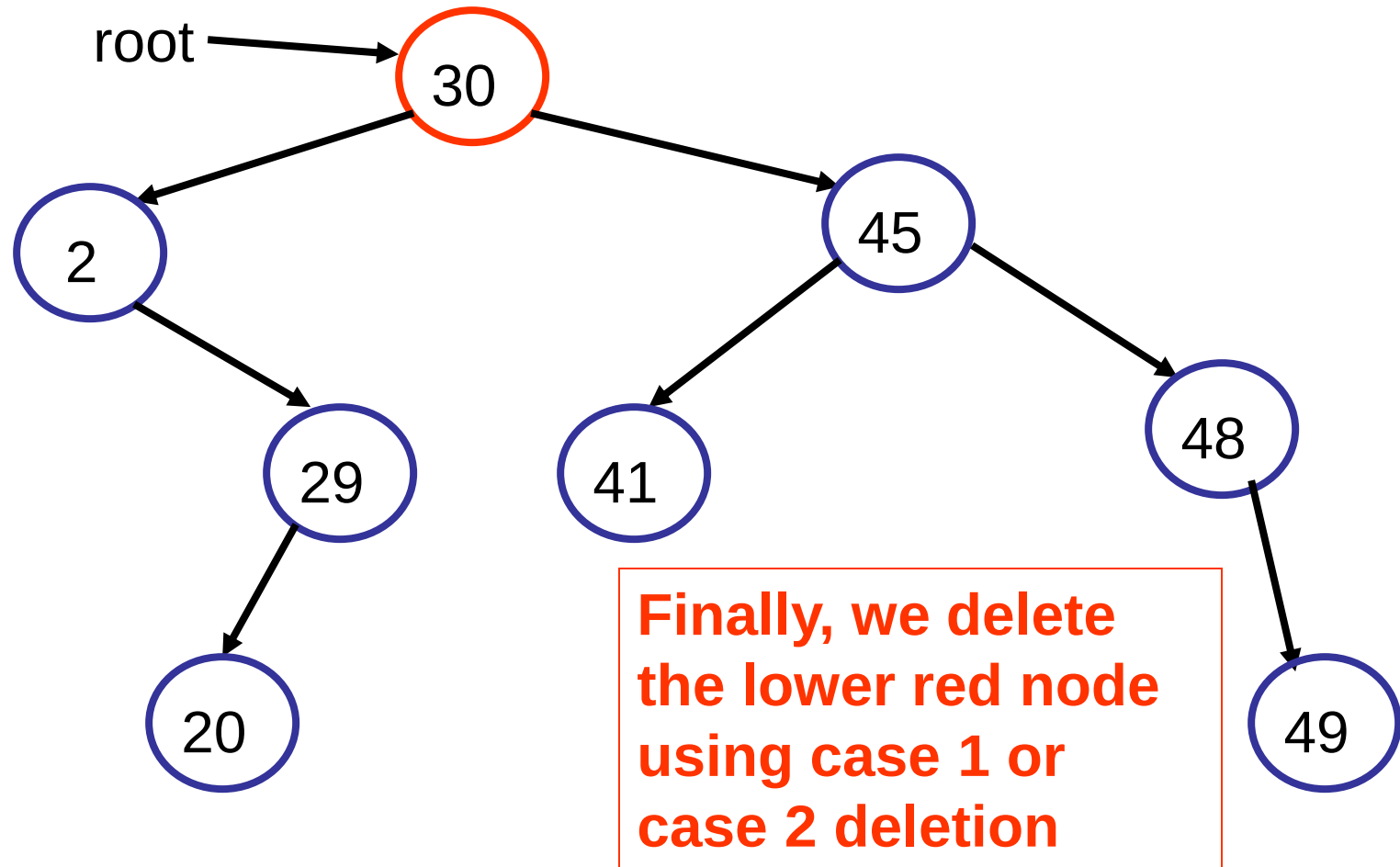
Deletion Case 3: Two Children (cont.)



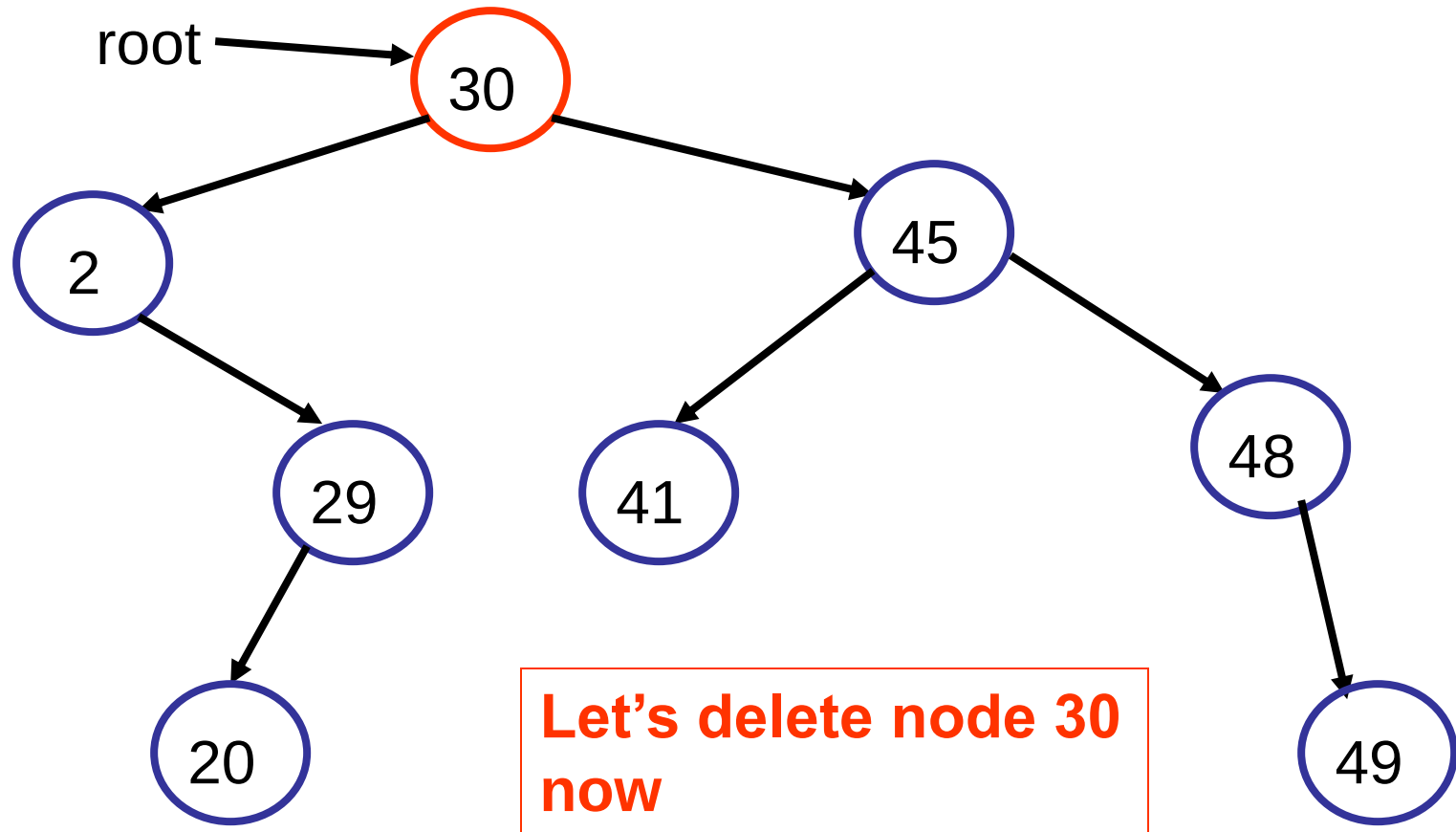
Deletion Case 3: Two Children (cont.)



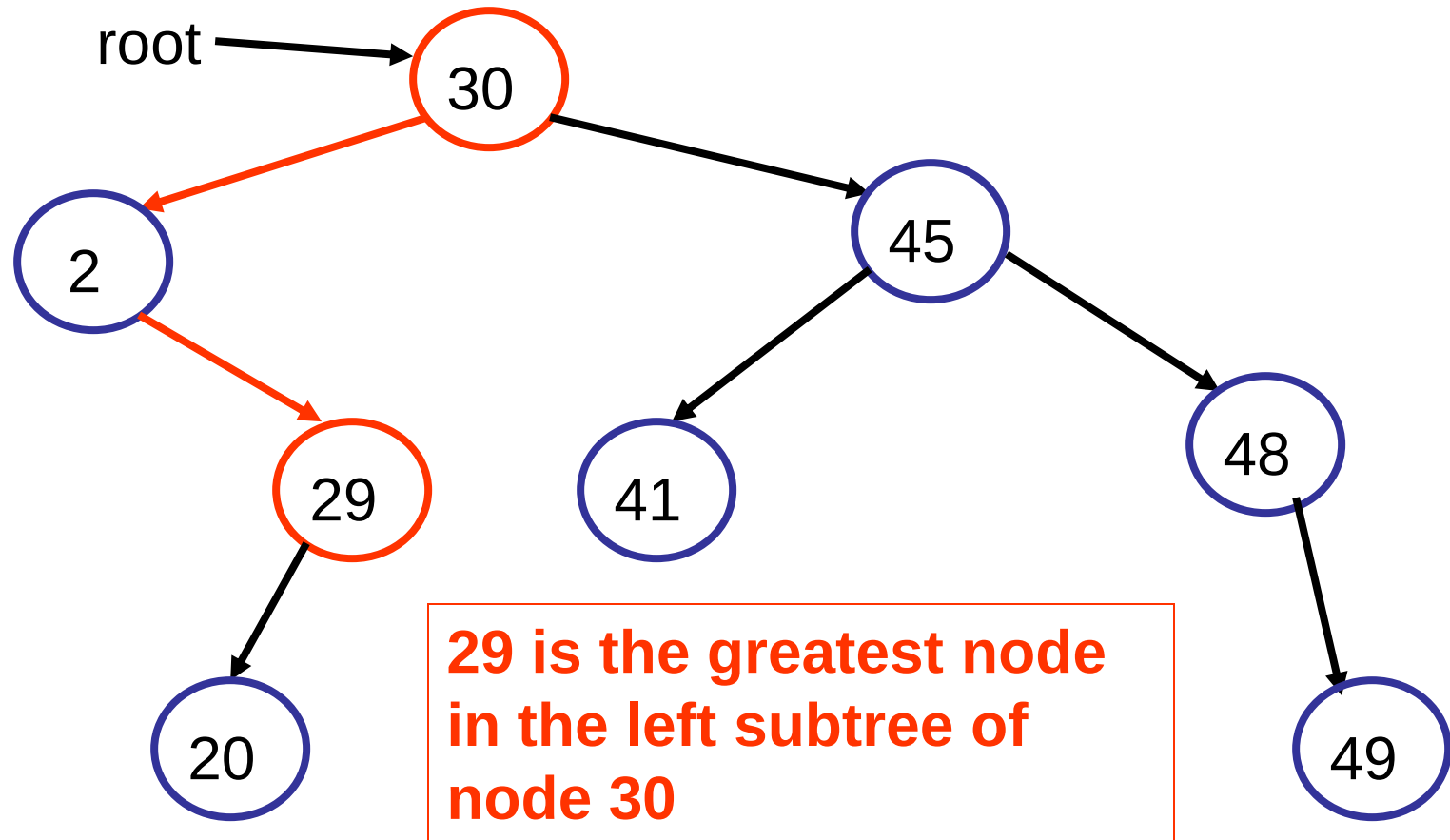
Deletion Case 3: Two Children (cont.)



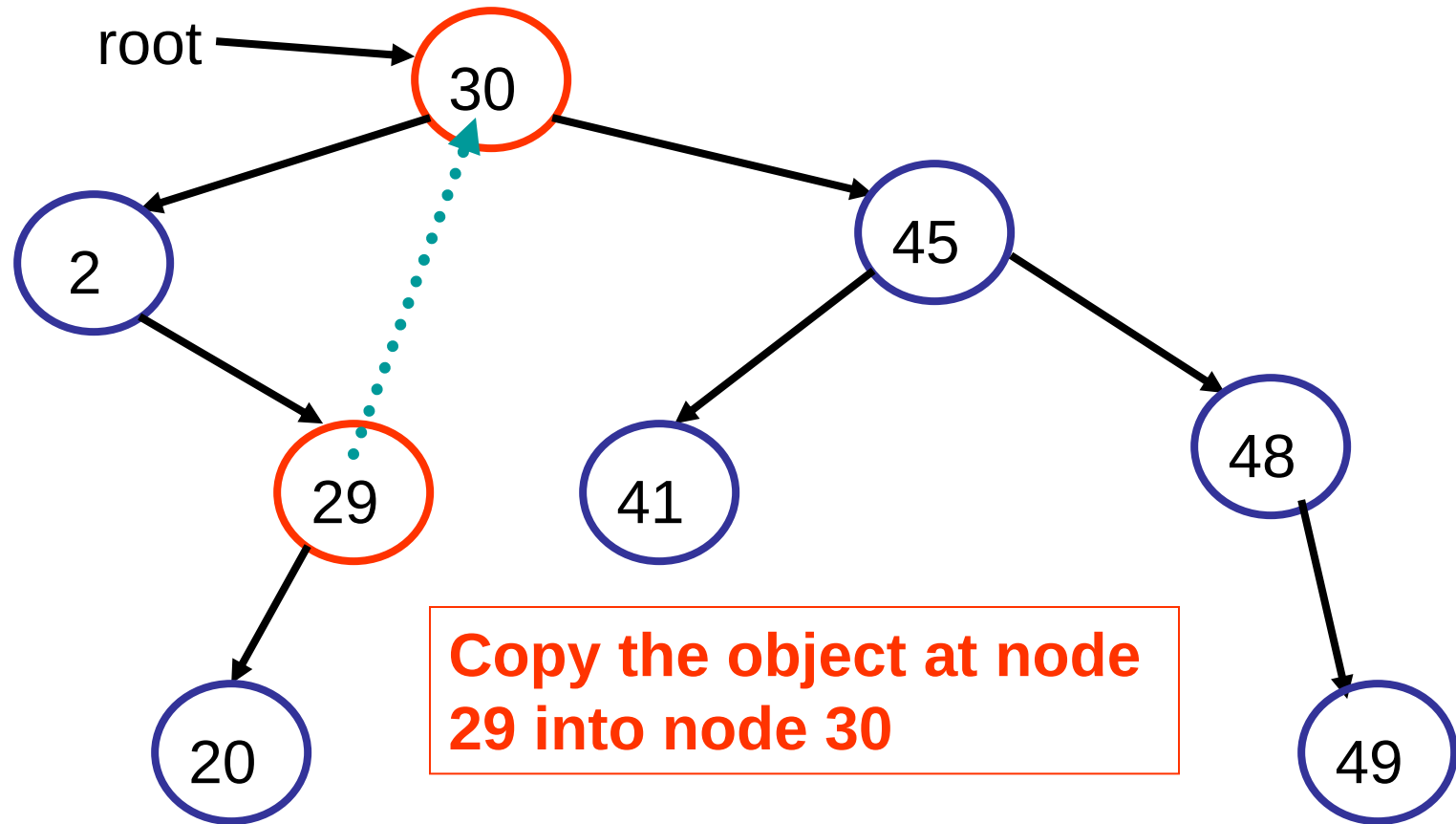
Deletion Case 3: Two Children (cont.)



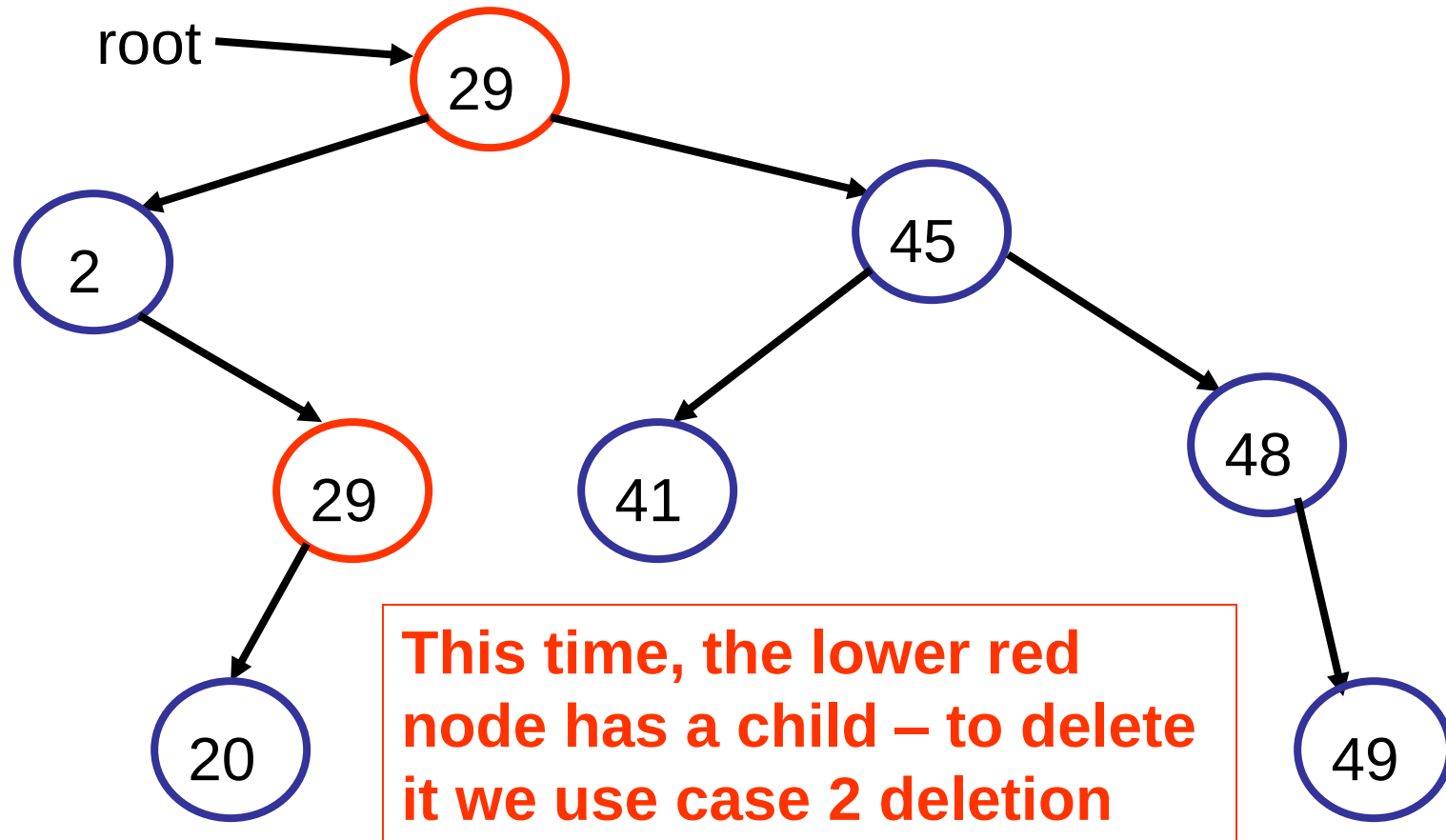
Deletion Case 3: Two Children (cont.)



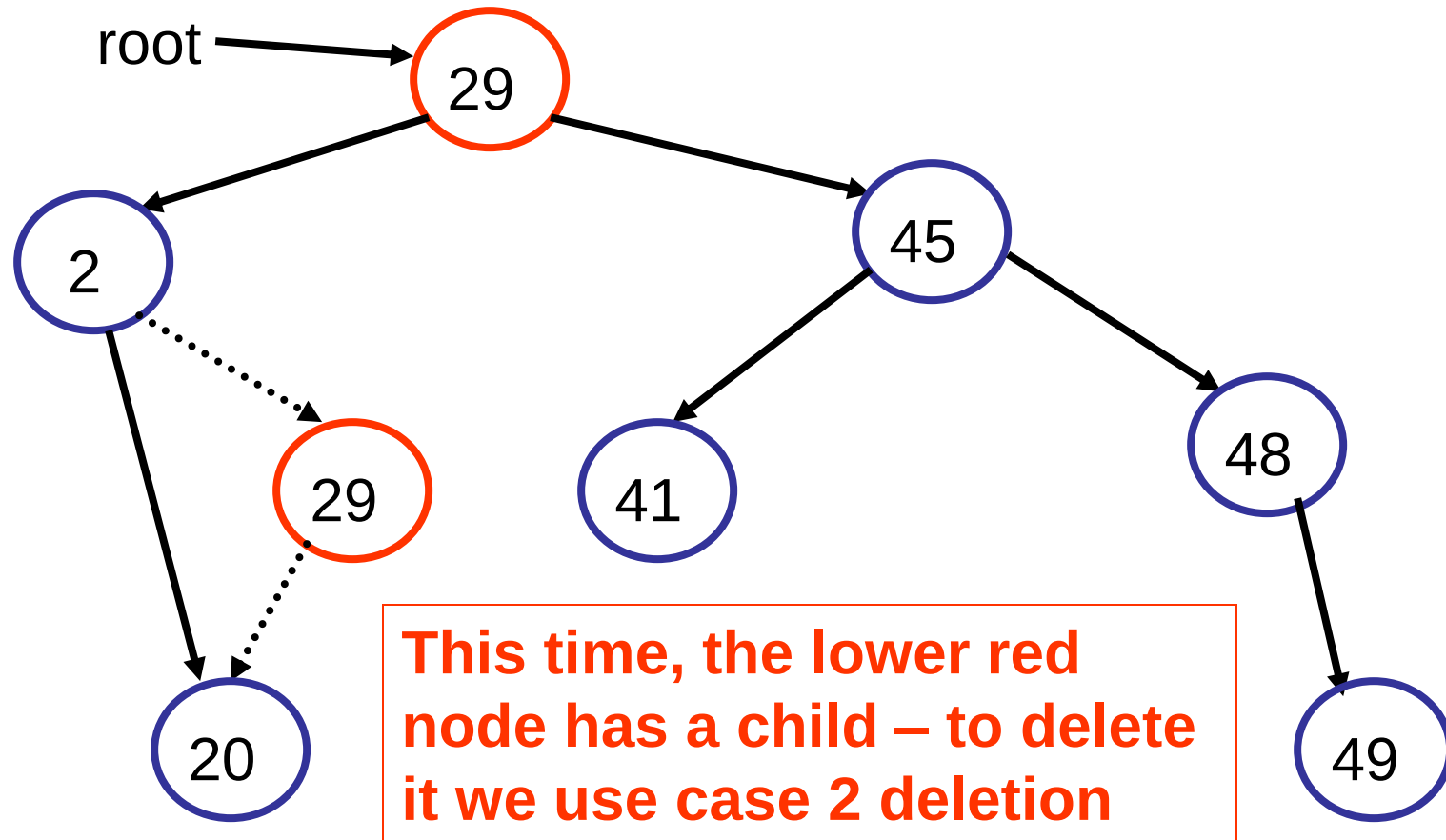
Deletion Case 3: Two Children (cont.)



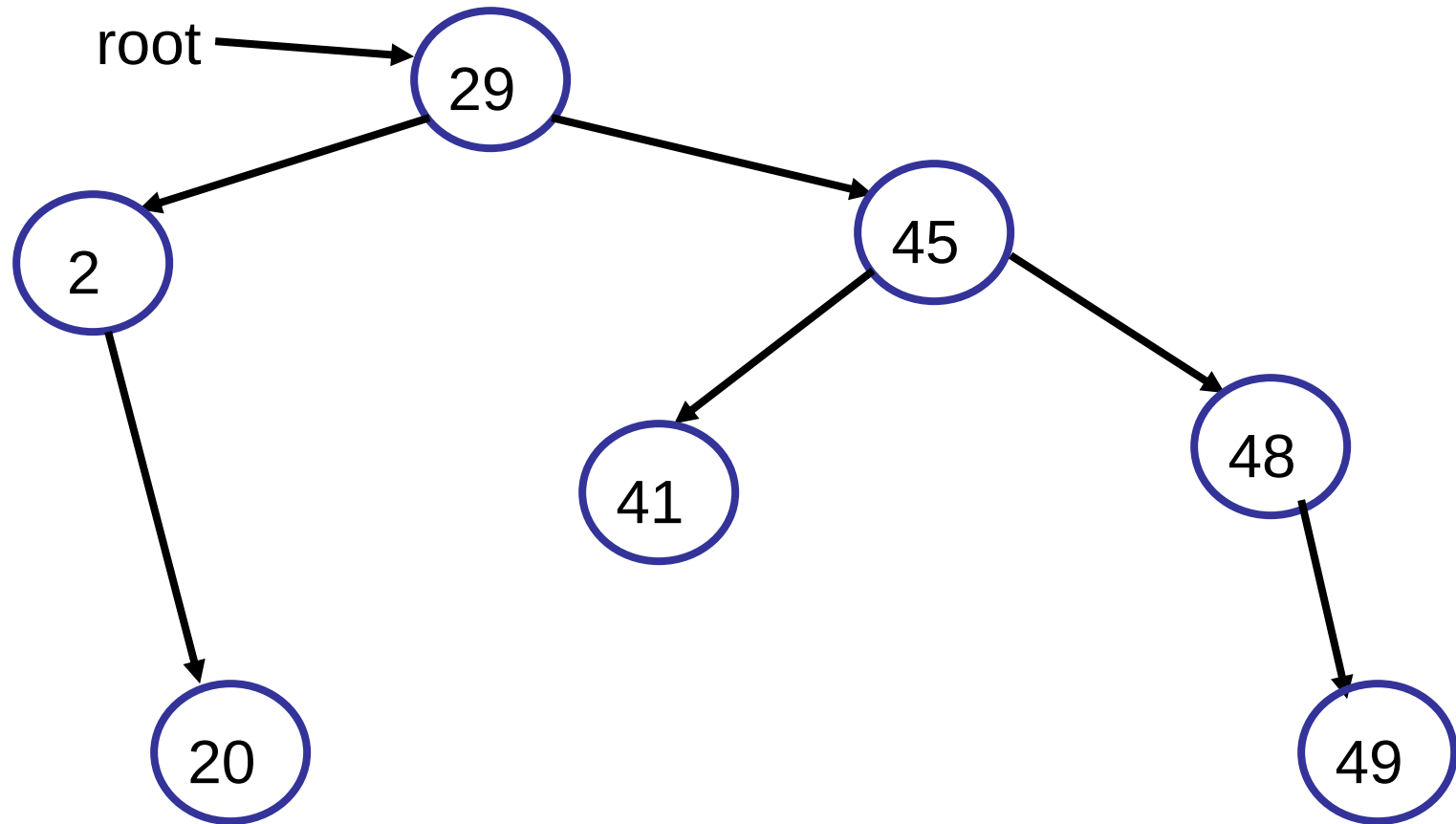
Deletion Case 3: Two Children (cont.)



Deletion Case 3: Two Children (cont.)



Deletion Case 3: Two Children (cont.)



Deletion Time Complexity

- In all cases, we must find the node we wish to delete first, using the standard search method.
- Finding the greatest node in the left subtree is just a continuation of a path down the BST
- For balanced BST's, the time complexity for deletion is $O(\lg n)$ in all 3 cases
- For unbalanced BST's, the time complexity is $O(n)$ in all 3 cases

Traversing a BST

- There are 3 ways to traversal a BST (visit all nodes in BST):
 - 1. Preorder (parent $\rightarrow$ left $\rightarrow$ right)
 - Root is output first
 - 2. Inorder (left $\rightarrow$ parent $\rightarrow$ right)
 - Output is sorted
 - 3. Postorder (left $\rightarrow$ right $\rightarrow$ parent)
 - Root is output last

Traversing a BST (cont.)

- Based on the BST on page 47, the result of traversal:
- 1. Pre-Order (parent $\rightarrow$ left $\rightarrow$ right)
 - 37 (root first) 2 29 20 7 30 45 41 48 49
- 2. In-Order (left $\rightarrow$ parent $\rightarrow$ right)
 - 2 7 20 29 30 37 41 45 46 49 (sorted)
- 3. Post-Order (left $\rightarrow$ right $\rightarrow$ parent)
 - 7 20 30 29 2 41 49 48 45 37 (root last)

BST Traversal

- Pre-Order Traversal

```
Algorithm preOrder (p)
    if p != null
        visit (p)
        preOrder (p.left)
        preOrder (p.right)
```

- In-Order Traversal

```
Algorithm inOrder (p)
    if p != null
        inOrder (p.left)
        visit (p)
        inOrder (p.right)
```

- Post-Order Traversal

```
Algorithm postOrder (p)
    if p != null
        postOrder (p.left)
        postOrder (p.right)
        visit (p)
```

Time Complexities for BST

- Balanced BST (AVL)
 - Insertion is $\Theta(\lg n)$
 - Search is $O(\lg n)$
 - Deletion is $O(\lg n)$
- Unbalanced BST
 - Insertion is $O(n)$
 - Search is $O(n)$
 - Deletion is $O(n)$

AVL vs Hash Table

- If $n = 500,000$,
 - Linear/Sequential search in array takes 500,000 comparisons in the worst case.
 - AVL takes $\lg 500000 \approx 19$ comparisons in the worst case.
 - A well design hashtable may just take 1 comparison.
- Hashtable is faster than AVL in searching and insertion if a good hash function could be obtained. However, finding a good hash function might not be easy.

AVL vs Hash Table (cont.)

- Advantages of AVL over hashtable are:
 - AVL could produce sorted result using inorder traversal.
 - AVL supports finding min or max in $O(\lg n)$ time.